

PEP-AI Validated Quantization for Energy-Efficient Perception in Intelligent Transportation Systems

Tamás Menyhárt^{1*}, András Hajdu¹ and Róbert Lakatos¹

^{1*}Department of Data Science and Visualization, Faculty of
Informatics, University of Debrecen, Debrecen, Hungary.

*Corresponding author(s). E-mail(s): menyhart.tamas@inf.unideb.hu;
Contributing authors: hajdu.andras@inf.unideb.hu;
lakatos.robert@inf.unideb.hu;

Abstract

Background: Post-training INT8 quantization reduces the inference energy of camera-based perception, but quantized models are judged by output accuracy alone, which hides where the error arises and how it propagates.

Methods: Five pretrained detection and classification networks were quantized without retraining into FP16, INT8 and FP8 TensorRT engines on an NVIDIA H200. We prove noise and propagation bounds, test them on layer-wise activations, measure accuracy, energy and robustness, and validate a static analysis.

Results: INT8 was 1.6–11.8× faster than FP32 and used 72–91% less energy; FP16 was lossless, FP8 lost at most 0.4 points and INT8 0.25 mAP to 41 top-1 points. Noise propagation, not network size, explained this range: one measurement of the quantized head input predicted the relative INT8 loss within a factor of about two (tenfold per 12 dB). From the FP32 parameters alone, the distribution-independent FP8 noise was predicted within 0.14 dB, the distribution-dependent INT8 noise only within 1–6 dB. INT8 also lost up to 36% under low contrast and 17 points with the calibration order, and broke the duplicate suppression of an NMS-free detector; FP8 did not. Keeping 20 ranked layers in FP16 restored EfficientNet-B0 from 24.8% to 66.0%.

Conclusion: Quantization error is predictable from FP32 statistics up to a measurable propagation factor, which decides whether INT8 is safe; FP8 is the distribution-independent alternative. For 1 000 vehicles, INT8 saves up to 1 GWh and 251 t CO₂ per year.

Keywords: post-training quantization, INT8, FP8, TensorRT, energy efficiency, explainable AI, sustainable AI, intelligent transportation systems

1 Introduction

Camera-based perception never pauses: it runs on every camera stream of an automated vehicle for as long as the vehicle operates, so its energy grows with fleet size, frame rate and operating hours [1]. If automated driving is widely adopted, the emissions of onboard computing could approach those of today’s data centres [2]. The energy cost of machine learning is already recognised as a sustainability problem [3–6], and for many deployed models it is inference, not training, that dominates [7, 8]; reported savings should therefore be measured rather than estimated [9]. In transport, the European Green Deal asks for a 90% cut of emissions by 2050, at the same time as automated mobility is to be deployed at scale [10, 11]. Choosing the numerical format of a perception network thus means trading energy cost and emissions against perception errors. In AI-based business optimisation (AIBO), model outputs feed managerial decisions whose cost includes both operating expenses and the consequences of errors [12]. For quantization, such a decision needs two numbers that aggregate accuracy does not give: the measured energy per inference and the error rate relative to the reference model.

Post-training quantization (PTQ) converts the weights and activations of a trained network from 32-bit floating point (FP32) to 8-bit integers (INT8), without retraining [13–16]. This cuts memory traffic fourfold, and an 8-bit integer multiplication or addition costs 18–30 times less energy than its FP32 counterpart [17]. On a GPU, however, these savings only materialise with integer kernels generated by an inference compiler [18, 19]; quantization that is merely simulated in floating point saves nothing. Most research on PTQ aims at minimising the output error of the quantized model. Adaptive rounding [20], layer-wise optimisation on small calibration sets [21] and block reconstruction [22, 23] optimise how the weights are rounded; cross-layer equalisation balances per-channel ranges [24]; outlier channel splitting reduces clipping [25]; and mixed-precision methods allocate bit widths from second-order sensitivities [26, 27].

On the analytical side, the rounding error of a single quantizer is usually treated as additive noise that is uniform within the clipping range. This model underlies analytical range setting [15, 28] and the comparison of integer and floating-point formats [29, 30]. How the noise of many quantizers combines as it passes through a nonlinear network has been bounded under distributional assumptions [31] or kept in check by non-expansive layers [32]. What is missing is a decomposition into quantities that can be estimated from FP32 statistics and then tested on a deployed engine. With such a decomposition, one could tell before deployment which networks tolerate INT8 and which layers have to stay in higher precision.

In practice, PTQ methods are judged by aggregate accuracy. Aggregate accuracy can stay stable while the internal representations change, and compressed models can disagree with the original on a subset of inputs [33]. It does not show where the error arises, whether it predicts the errors that matter, or how it behaves under

distribution shift, and these are exactly the questions that safety standards for road vehicles require to be analysed [34–36]. Some pieces are known: depthwise-separable networks are sensitive to per-tensor activation quantization [18, 37], and quantized classifiers are more sensitive to natural corruptions such as fog or blur [38–40]. For detectors running on integer kernels, however, nobody has yet quantified how the error of individual layers relates to detection failures.

In our conference paper we introduced the Precise, Explainable and Provable (PEP-AI) framework for validating quantized networks at the level of their activations [41]. It consisted of the median relative error of channel-projected activations, symmetric-logarithmic hexbin densities and an attenuation model of layer-wise error propagation. That paper analysed a single backbone in simulated INT8 and showed qualitative failure examples; deployed integer engines, other architectures and a quantitative link between the deviation and detection errors were left for future work. The present article takes up these open points. Its three main contributions are:

- *Noise and propagation are separable, and propagation decides.* The error at the task head splits into the noise the quantizers inject, which follows from FP32 statistics, and a measurable propagation factor $\bar{\Gamma}$ (Section 2.6). Across the networks, $\bar{\Gamma}$, not network size, explains the INT8 loss, and one measurement of the quantized head input on unlabelled images predicts the relative loss within a factor of about two (Section 3.8), which a simulated check on 15 networks confirms (Section 3.9).
- *Floating-point noise is distribution-independent, integer noise is not.* Within its normal range, the FP8 rounding noise does not depend on the activation distribution or the calibrated range, so it can be predicted from the FP32 parameters alone without data; the INT8 noise depends on the range and the tails (Sections 2.5 and 3.9).
- *Two INT8 failure modes that aggregate accuracy hides.* The INT8 accuracy of fragile networks depends on the order of the calibration images, and INT8 breaks the learned duplicate suppression of an NMS-free detector (Section 3.2).

They rest on strict-FP32, FP16, INT8 and FP8 TensorRT engines of five networks on an H200 with paired bootstrap intervals, latency, memory traffic and energy; on a detection-level test of the deviation–error link and a robustness study under eight adverse conditions; on a label-free diagnosis and repair by selective precision; and on a static analysis tool, checked in simulated quantization on further networks. The rounding-noise lemma itself restates classical results [28, 29]; what is new is its test on deployed engines and its use in the decomposition. Although the work is motivated by automated driving, the experiments use general-purpose benchmarks (COCO, ImageNetV2) and a data-centre GPU, and the fleet scenario is an illustrative calculation, not a measurement on vehicle hardware. The complete pipeline, the static analysis tool and all results are openly available at <https://github.com/EgyipTomi425/EchteAI>, and every number, table and figure of this article can be regenerated from it. The step-by-step proofs, implementation details and supplementary analyses are given in Appendices A and B, and the extended tables and figures in Appendix C.

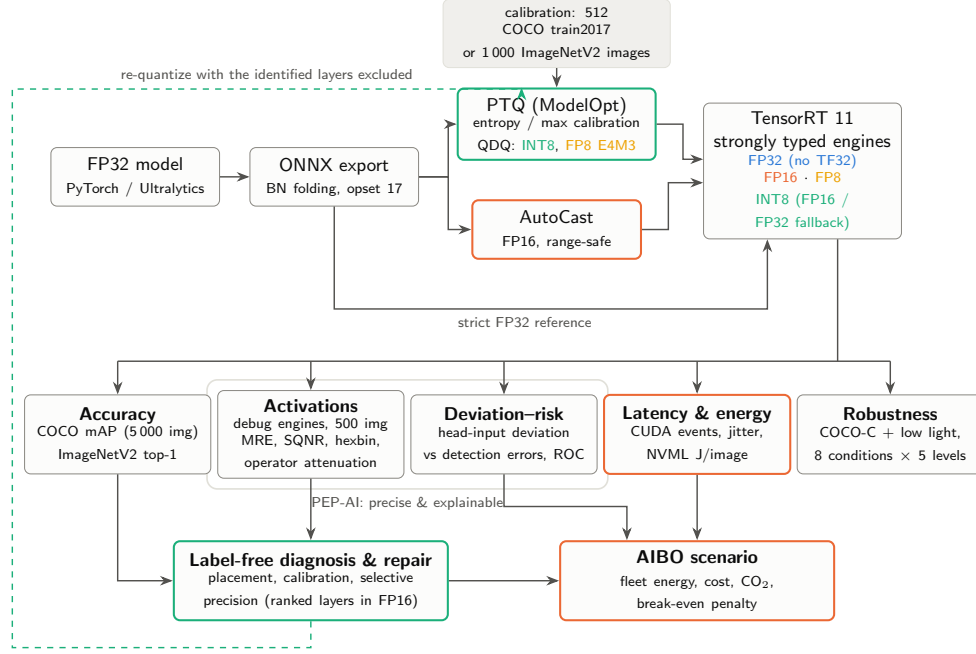


Fig. 1 PEP-AI validation pipeline. The FP32 model is exported once and turned into an FP16 variant and into INT8 and FP8 variants that share one calibration; every variant is compiled into a strongly typed TensorRT engine and evaluated along five measurement branches. The activation analysis feeds back into quantization (label-free diagnosis and repair, dashed), and the measured energy, accuracy and error rates feed the AIBO scenario.

2 Methods

2.1 PEP-AI validation pipeline

PEP-AI compares the activations of a quantized network with those of its FP32 reference. The three letters stand for three requirements. *Precise* means per-tensor, per-image deviation statistics for every analysed layer of the deployed engine. *Explainable* means an analytical model of where the deviation is injected and how it propagates, supported by activation maps and density plots. *Provable* means that statements are either proved under explicit assumptions (Lemma 1, Corollary 2, Propositions 3 and 4) or reported as empirical claims with a stated sample and uncertainty. The pipeline (Fig. 1) exports each FP32 model to ONNX, quantizes it into explicit quantize-dequantize (QDQ) graphs and compiles strongly typed TensorRT engines. It then measures accuracy, activation deviation, the link between deviation and errors, latency, energy and robustness, and the layer-level diagnosis is fed back into quantization. Every stage is scripted and seeded, so that the pipeline regenerates every number, figure and table of this article.

2.2 Models and data

We evaluated five convolutional architectures that cover two-stage detection, one-stage detection and classification (Table 1). The two-stage detector is Faster R-CNN with a ResNet-50 FPN backbone [42], with the torchvision COCO weights and its frozen batch normalisation folded into the convolutions before export. The one-stage detectors are YOLOv10-S and YOLOv10-X [43] with their NMS-free one-to-one head. The classifiers are EfficientNet-B0 [44] and DenseNet-121 [45], both with torchvision ImageNet weights.

Table 1 Evaluated architectures

Model	Task	Input	Params (M)	GFLOPs	Convs	Weights (MB)		
						FP32	FP16	INT8
Faster R-CNN R50-FPN	det.	800×1088	26.8	240.4	61	107.3	53.7	27.0
YOLOv10-S	det.	640×640	8.1	24.9	86	29.0	14.5	7.4
YOLOv10-X	det.	640×640	31.8	170.8	174	117.9	58.9	29.9
EfficientNet-B0	cls.	224×224	5.3	0.8	81	21.1	10.5	6.8
DenseNet-121	cls.	224×224	8.0	5.7	120	32.2	16.1	9.3

Faster R-CNN: backbone and FPN only (the part that is quantized); GFLOPs at the benchmark input size. INT8 weight size counts quantized weights at 1 byte and the remaining tensors in FP16.

The detectors were evaluated on the complete COCO 2017 validation split (5 000 images) [46]; the activation analysis and the robustness study use the first 500 images of a seeded permutation of it. For calibration we took 512 seeded images from the COCO 2017 *training* split, so that calibration and evaluation data never overlap. The classifiers were evaluated on ImageNetV2 (matched frequency) [47], which we split with a fixed seed into 1 000 calibration and 9 000 evaluation images. Pre-processing follows the reference implementation of each model (Table 2). About 40% of all COCO objects belong to traffic-relevant categories.

Table 2 Data splits

Split	Purpose	Images	Objects	Obj./img	S/M/L (%)	Traffic (%)
train2017 subset	calibration (detectors)	512	3 401	6.6	43 / 32 / 25	44
val2017	mAP, deviation-risk	5 000	36 335	7.3	42 / 34 / 24	41
val2017 first 500	activations, robustness	500	3 787	7.6	44 / 33 / 23	41
INV2 calibration	calibration (classifiers)	1 000	—	—	—	—
INV2 evaluation	top-1, activations	9 000	—	—	—	—

Traffic: share of objects in the classes person, bicycle, car, motorcycle, bus, truck, traffic light and stop sign. S/M/L: COCO size classes by box area ($< 32^2$, 32^2 – 96^2 , $> 96^2$ pixels). All subsets are drawn with a fixed seed.

2.3 Quantization and deployment

Post-training quantization used NVIDIA TensorRT Model Optimizer [48] without fine-tuning [13–15]. For a tensor x and a scale $s > 0$, the symmetric INT8 quantizer is

$$Q_s(x) = s \cdot \text{clip}(\lfloor x/s \rfloor, -127, 127), \quad (1)$$

with one scale per output channel for weights and one scale $s = \alpha/127$ per tensor for activations. The clipping range α is set on the calibration images by entropy calibration [49] (Eq. (A1)) or, as an ablation, by the observed maximum. Small calibration sets can make the result depend on the sample [21], so we recalibrated every model on seeded random subsets of its calibration set and, as a control, with an order-independent implementation of entropy calibration (Section A.1). The FP8 variants [50] use $s = \alpha/448$, with α set to the observed maximum (the toolchain default for FP8), and round to the nearest E4M3 number; they use the same calibration data and the same exclusions. FP16 was produced by automatic mixed precision [51]. From one calibration we derive two INT8 graphs that share every scale: *INT8*, in which operations without QDQ nodes run in FP16 (this is the deployed variant), and *INT8 (FP32 fallback)*, which isolates the effect of quantization in the activation analysis. Decoding and post-processing stay in high precision. Quantizing the TopK of the YOLOv10 head would restrict the class scores to 127 levels, and quantizing the distribution-focal-loss (DFL) box decoder distorts large objects; we report both of these deployment artefacts as ablations (Fig. C2). INT4 was not evaluated, because the H200 has no INT4 tensor-core arithmetic and TensorRT supports INT4 only for the weights of matrix multiplications [52, 53].

All variants were compiled with TensorRT 11.3 [19] into *strongly typed* engines, in which the precision of every layer follows the graph. We disabled TF32, so that the FP32 baseline is true single precision. Static-shape engines were built for batch sizes 1, 6 and 8. For evaluation, Faster R-CNN uses a dynamic-shape engine (input sides 256–1344), while its proposal network and ROI heads run in FP32 PyTorch. We read back from the engine inspector the precision in which every convolution is actually executed. The environment is listed in Table 3, and implementation details are given in Section A.1.

Table 3 Hardware and software environment

Component	Version / specification
GPU	NVIDIA H200, power limit 700 W
GPU driver	590.48.01
CPU	Intel Xeon Platinum 8480C
TensorRT	11.3.0.99
NVIDIA ModelOpt	0.47.0
ONNX Runtime	1.24.4
PyTorch / CUDA	2.11.0+cu128 / 12.8

2.4 Activation-level deviation metrics

We mark every output tensor of the analysed operators (convolutions, activation functions, additions, pooling, concatenation, resizing and non-foldable batch normalisation) as an engine output and compare the strict-FP32 engine with the INT8 (FP32 fallback) engine on identical inputs. Marking outputs prevents some fusions, but it leaves the quantized arithmetic unchanged. With f an FP32 tensor, q its quantized counterpart and $e = q - f$, the metrics of Table 4 are

$$\text{MRE}_{\text{elem}} = \text{median}_i \frac{|e_i|}{|f_i| + \eta}, \quad (2)$$

$$A(x, y) = \max_c F(c, x, y), \quad \text{MRE}_{\text{proj}} = \text{median}_{x, y} \frac{|A_q(x, y) - A_f(x, y)|}{|A_f(x, y)| + \eta}, \quad (3)$$

$$r = \frac{\|e\|_2}{\|f\|_2}, \quad \text{SQNR} = 10 \log_{10} \frac{\|f\|_2^2}{\|e\|_2^2} \text{ dB}, \quad (4)$$

with $\eta = 10^{-6}$. To give a feeling for the scale: an SQNR of 40, 20 and 0 dB corresponds to a relative error r of 1%, 10% and 100%, and every 6 dB is a factor of two in r . Task accuracy is the COCO box mAP@[.5:.95] for the detectors [46, 54], complemented for the road-user classes by the AP at IoU 0.5 and by recall and precision at the operating threshold of 0.5, and the top-1 accuracy for the classifiers. MRE_{proj} is the metric of the conference paper; r and SQNR are scale-free and standard in the quantization literature [15]. Per-image values are aggregated over the 500 analysis images by their median and interquartile range. By *head-input* tensors we mean the tensors consumed by the task head: the five FPN levels, the three YOLOv10 head inputs and the last feature map of the classifiers. For the detectors, we also compute MRE_{proj} separately inside and outside the ground-truth boxes.

2.5 Quantization noise model

To interpret the deviations we use the additive noise model that also underlies analytical range setting [15, 16, 28]. It rests on two assumptions. (A1) Inside the clipping range, the INT8 rounding error $e = Q_s(x) - x$ is uniform on $[-s/2, s/2]$ and uncorrelated with x ; this holds when s is small compared with the spread of x . (A2) Under the power-weighted measure $x^2 dP(x)$, the significand $m = |x|/2^{\lfloor \log_2 |x| \rfloor}$ of an activation is log-uniform on $[1, 2)$; this holds approximately for any distribution whose logarithm spreads over several octaves. Lemma 1 collects known results [15, 28, 29] in the form used in this article; its role here is to provide the per-quantizer term of the decomposition in Section 2.6 and a prediction that can be tested on deployed engines.

Lemma 1 (Rounding noise) *Let x be an activation with root-mean-square value σ and normalised range $\kappa = \alpha/\sigma$. Under (A1), the INT8 quantizer of Eq. (1) with $s = \alpha/127$ has the error power*

$$\mathbb{E} e^2 = \frac{s^2}{12} \Pr(|x| \leq \alpha) + \mathbb{E} \left[(|x| - \alpha)^2 \mathbb{1}_{\{|x| > \alpha\}} \right], \quad (5)$$

Table 4 PEP-AI metrics. f : FP32 tensor, q : quantized tensor, $e = q - f$, A : channel-wise maximum projection.

Metric	Definition	PEP-AI role	Interpretation
MRE_{proj}	Eq. (3)	precise, explainable	conference-paper metric; spatial, on the dominant response per location
MRE_{elem}	Eq. (2)	precise	element-wise median relative error over all channels
r	$\ e\ _2 / \ f\ _2$	precise	scale-free relative error; $r \geq 1$ means no usable signal
SQNR	$-20 \log_{10} r$	precise	signal-to-quantization-noise ratio in dB
a_n	$r_{\text{out}} / \max r_{\text{in}}$	explainable	operator-level attenuation (< 1) or amplification (> 1)
s_n	SQNR drop across node n	explainable	error injected by a layer; drives selective quantization
AUC	ROC of deviation vs. errors	provable	how well internal deviation predicts detection errors

and, without clipping,

$$\text{SQNR}_{\text{INT8}} = 10 \log_{10} (12 \cdot 127^2) - 20 \log_{10} \kappa \approx 52.9 \text{ dB} - 20 \log_{10} \kappa. \quad (6)$$

Under (A2), round-to-nearest in a floating-point format with p significand bits has, in its normal range, the error power $\mathbb{E} e^2 = \mathbb{E} x^2 2^{-2p} / (8 \ln 2)$, so that, independently of κ ,

$$\text{SQNR}_{\text{FP}} = 20p \log_{10} 2 + 10 \log_{10} (8 \ln 2) \approx 6.02p + 7.44 \text{ dB}. \quad (7)$$

The idea behind both results is simple. INT8 has a fixed grid with step $s = \alpha/127$, and a rounding error that is uniform over one step has the mean square $s^2/12$; dividing the signal power σ^2 by it leaves only the ratio $\kappa = \alpha/\sigma$. A floating-point grid, by contrast, is spaced in proportion to the value itself, so the relative error is the same for small and large values and κ drops out; averaging over where a value falls within its octave gives the constant $8 \ln 2$. For E4M3 ($p = 4$), Eq. (7) gives 31.5 dB, and for FP16 ($p = 11$) 73.7 dB. The two formats behave very differently: the relative precision of a floating-point format does not depend on the calibrated range [29, 30], whereas INT8 loses 20 dB per decade of κ . Eq. (5) expresses the familiar trade-off between the granular noise of a wide range and the clipping of a narrow one [25, 28]. A Monte Carlo simulation reproduces Eqs. (5)–(7) within 0.1 dB (Fig. 2), and the proofs are given in Section A.2. Because TensorRT uses static scales, Lemma 1 also tells what happens when the signal level changes after calibration.

Corollary 2 (Static scales) *Let the scale s be calibrated for an activation with root-mean-square value σ_{cal} , and let the activation at inference be c times larger in distribution, $c > 0$. If no value is clipped and (A1), (A2) hold, then*

$$\Delta \text{SQNR}_{\text{INT8}} = 20 \log_{10} c, \quad \Delta \text{SQNR}_{\text{FP8}} = 0 \quad \text{for } c \sigma_{\text{cal}} \gg 2^{-6} s, \quad (8)$$

where $2^{-6} s$ is the smallest normal E4M3 number.

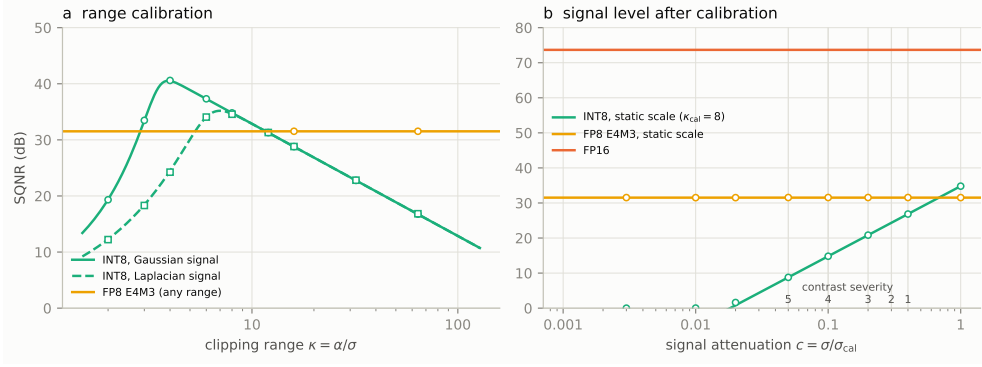


Fig. 2 Quantization noise model (lines: Eqs. (5)–(7); markers: Monte Carlo simulation with 4×10^5 samples). **a** SQNR of INT8 versus the normalised clipping range $\kappa = \alpha/\sigma$ for Gaussian and Laplacian signals, and of FP8 E4M3. **b** SQNR of a signal attenuated by the factor c after the scale was calibrated at $\kappa_{\text{cal}} = 8$ (static per-tensor scale); vertical lines: contrast factors of the five COCO-C contrast severities.

In words: INT8 loses 6 dB each time the signal is halved, while FP8 loses nothing. A signal that grows ($c > 1$) enters the clipping regime of Eq. (5), which the tight INT8 range cannot absorb, unlike the maximum-calibrated FP8 range. Reduced contrast and fog are shifts of this kind. The attenuation c has to be measured at the quantizers, however, because biases and mean responses do not scale with the pixel contrast (Section A.2).

2.6 Error propagation model

The conference paper modelled the propagation of quantization error between layers l and $l+1$ as

$$\epsilon^{(l+1)} = \mathcal{A} \left(\mathcal{F}(y_f^{(l)} + \epsilon^{(l)}) - \mathcal{F}(y_f^{(l)}) \right) + \delta^{(l+1)}, \quad (9)$$

with layer transformation $\mathcal{F}$, local rounding noise $\delta^{(l+1)}$ and an attenuation operator $\mathcal{A}$. Other analytical treatments propagate quantization noise under distributional assumptions [31] or constrain the layers to be non-expansive [32]. Here we linearise every node n around its FP32 operating point. With FP32 output x_n , deviation e_n , inputs $\pi(n)$, Jacobian J_n and injected rounding noise δ_n ($\delta_n = 0$ for nodes without quantizers),

$$e_n = J_n e_{\pi(n)} + \delta_n, \quad (10)$$

where $e_{\pi(n)}$ stacks the deviations of all inputs. Squared norms of deviations are understood as expectations over the rounding noise, $\|e\|^2 := \mathbb{E} \|e\|^2$. If δ_n is uncorrelated with the propagated deviation, the cross term $2 \mathbb{E} \langle J_n e_{\pi(n)}, \delta_n \rangle$ vanishes, and the relative error $r_n = \|e_n\|/\|x_n\|$ obeys

$$r_n^2 = g_n^2 r_{\pi(n)}^2 + \rho_n^2, \quad g_n = \frac{\|J_n e_{\pi(n)}\|}{\|e_{\pi(n)}\|} \cdot \frac{\|x_{\pi(n)}\|}{\|x_n\|}, \quad \rho_n = \frac{\|\delta_n\|}{\|x_n\|}. \quad (11)$$

The propagation gain g_n is the attenuation operator $\mathcal{A}$ in scalar form. For every analysed node we measure the amplification ratio

$$a_n = \frac{r_n}{\max_{m \in \pi(n)} r_m} \leq \sqrt{g_n^2 + \rho_n^2 / r_{\pi(n)}^2}, \quad (12)$$

with equality for single-input nodes, because $r_{\pi(n)} \leq \max_m r_m$ for stacked inputs; for single-input operators without quantizers, $a_n = g_n$. Section A.3 derives closed forms of g_n for sigmoid gates, max pooling and non-folded batch normalisation (Eq. (A3)) and proves the following two statements.

Proposition 3 *Assume the linearisation of Eq. (10) and zero-mean, mutually uncorrelated injected noises δ_n . Then the relative error at the head input h is*

$$r_h^2 = \sum_n \Gamma_{n \rightarrow h} \rho_n^2, \quad \Gamma_{n \rightarrow h} = \frac{\mathbb{E} \|J_{n \rightarrow h} \delta_n\|^2}{\mathbb{E} \|\delta_n\|^2} \cdot \frac{\|x_n\|^2}{\|x_h\|^2}, \quad (13)$$

where $J_{n \rightarrow h}$ is the Jacobian from node n to h . If a set S of nodes is kept in FP16, whose injected noise is negligible by Eq. (7), then $r_h^2(S) = r_h^2 - \sum_{n \in S} \Gamma_{n \rightarrow h} \rho_n^2$, and a set of k nodes that minimises $r_h^2(S)$ consists of the k largest contributions $\Gamma_{n \rightarrow h} \rho_n^2$.

Proposition 4 *Let a chain of nodes $1, \dots, n$ satisfy Eq. (11) with gains g_m and injected noise ρ_m . Then*

$$r_n^2 = r_0^2 \prod_{m=1}^n g_m^2 + \sum_{k=1}^n \rho_k^2 \prod_{m=k+1}^n g_m^2. \quad (14)$$

If $g_m \leq \bar{g} < 1$ and $\rho_m \leq \bar{\rho}$ for all m , the relative error stays bounded by $\max\{r_0, \bar{\rho}/\sqrt{1 - \bar{g}^2}\}$ at any depth, and for constant $g_m = g < 1$ and $\rho_m = \rho$ it converges to the fixed point

$$r_\infty = \frac{\rho}{\sqrt{1 - g^2}}, \quad \text{SQNR}_\infty = -20 \log_{10} \rho + 10 \log_{10} (1 - g^2). \quad (15)$$

If instead $g_m \geq g > 1$, the propagated part grows at least as $g^{2n} r_0^2$.

Proposition 4 puts the attenuation effect of the conference paper on a formal footing. The error does not have to shrink from layer to layer; it is enough that its propagated part contracts ($g < 1$), and the error then settles on a plateau r_∞ that is set by ρ and g . Proposition 3 motivates selective precision, and it also suggests how the head-input fidelity can be predicted from FP32 statistics alone. Applying Eq. (1) with the calibrated scales to the FP32 activations gives the injected noise ρ_n^2 of every quantizer exactly, and with unit propagation factors we obtain

$$\text{SQNR}_{\text{add}} = -10 \log_{10} \sum_n \rho_n^2, \quad \bar{\Gamma} = \frac{r_h^2}{\sum_n \rho_n^2} = 10^{(\text{SQNR}_{\text{add}} - \text{SQNR}_h)/10}, \quad (16)$$

where SQNR_h is the measured head-input SQNR. By Proposition 3, $\bar{\Gamma} = \sum_n \Gamma_{n \rightarrow h} \rho_n^2 / \sum_n \rho_n^2$ is the mean propagation factor, weighted by the injected noise. It is below one for a network that attenuates the injected noise and above one for a network that amplifies it (Fig. C3).

2.7 Deviation–risk analysis

We run both engines on all 5 000 validation images with the head-input tensors exposed and compare them at a score threshold of 0.5. As in the conference paper, an image contains a *detection error* if INT8 misses a ground-truth object that FP32 found, or if an FP32 detection has no INT8 counterpart of the same class with $\text{IoU} \geq 0.5$. A *score-tolerant* variant accepts INT8 counterparts down to a score of 0.3; an FP32 detection that still has no counterpart has then genuinely vanished. At the image level, we compare the ROC AUC of the head-input MRE_{proj} with that of a trivial covariate, the number of FP32 detections. For the local deviation defined below, we derive an architecture-specific threshold from the data as the cut that maximises Youden’s index $J = \text{TPR} - \text{FPR}$ for vanishing detections. At the detection level, the *local deviation* d_j of detection j is the median MRE_{proj} of the head-input maps inside its box, and it enters the logistic model

$$\text{logit } \Pr(v_j = 1) = \beta_0 + \beta_1 \log_2 d_j + \beta_2 m_j + \beta_3 \log A_j \quad (17)$$

for the vanishing v_j of the detection, with score margin $m_j = -|s_j - 0.5|$ and box area A_j ; e^{β_1} is then the odds ratio per doubling of the local deviation. These analyses use the INT8 engines with FP32 fallback. What box shifts mean for monocular distance estimation follows from a flat-ground camera model (Section A.4).

2.8 Latency, jitter and energy

Latency was measured on an otherwise idle GPU, one engine at a time, with CUDA events around each of 2000 executions after 100 warm-up runs, with and without CUDA graphs and with inputs already on the device. Pre-processing, host transfers and the Faster R-CNN detection heads are excluded, so the measurement isolates the compiled engine. We summarise latencies by the median (p50), the 99th percentile (p99), the maximum and the jitter $Q_{0.99} - Q_{0.5}$. Energy was read from the GPU energy counter $E(t)$ (NVML) during at least 20 s of sustained execution of n batches of size B , replayed as CUDA graphs as in the latency measurements, after 10 s of idling at power P_{idle} :

$$E_{\text{img}} = \frac{E(t_1) - E(t_0)}{nB}, \quad E_{\text{img}}^{\text{net}} = E_{\text{img}} - \frac{P_{\text{idle}}(t_1 - t_0)}{nB}. \quad (18)$$

Sustained runs average out the partial sampling of the built-in power sensor [55]. We logged clock and temperature and repeated every measurement three times, reporting the median. Memory traffic and the precision in which each layer is executed were read from the engine inspector (Eq. (A2)), and reference energies per operation [17] are listed in Table C1.

2.9 Robustness under adverse imaging conditions

Fog, snow, frost, brightness (glare), contrast, Gaussian sensor noise and motion blur follow the COCO-C protocol [39, 56] at severities 1–5 with fixed per-image seeds.

Low light, which COCO-C does not include, is simulated by gamma darkening with Poisson shot noise (Fig. C4). For every condition, severity and precision we report box mAP, head-input deviation and detection-error rate on the 500 analysis images. For the YOLOv10 detectors we also tried two label-free remedies: a robust calibration set in which half of the images carry a random condition, and keeping the input or the top-ranked convolution in FP16.

2.10 PEP-AI-guided diagnosis and repair

The main INT8 results use the default placement and entropy calibration of the toolchain. In addition, we re-quantized every model in a configuration that was chosen only with label-free criteria on the calibration split, namely the head-input SQNR and the output consistency with FP32 (Eq. (A6)). Three things could be changed: (i) the calibration method, (ii) the placement of operators whose measured amplification shows that they magnify upstream noise, such as the pre-activation batch normalisation of DenseNet, and (iii) selective precision. For selective precision, the sensitivity of a convolution is the median SQNR drop across it, $s_n = 20 \log_{10} a_n$ (for the input convolution, $\max(0, 40 \text{ dB} - \text{SQNR}_n)$). Either the k most sensitive convolutions are kept in FP16 at once (one-shot), or the network is re-measured after every exclusion (iterative; Algorithm 1). All variants share the calibration scales of the base model (Section A.5). In contrast to Hessian-based allocation [26, 27] or reconstruction-based PTQ [22, 23], no gradients are needed. We compare the ranking with random subsets (three seeds), with the heuristic of keeping all depthwise convolutions in higher precision [24, 37] and with a ranking predicted from FP32 statistics alone. Task accuracy is measured only afterwards.

2.11 Illustrative fleet scenario (AIBO)

We take the measured energy per image at batch size 6, which corresponds to the synchronised frames of the six cameras of a vehicle, and scale it to a fleet of $N = \text{vehicles} \times \text{cameras} \times \text{frame rate} \times \text{operating hours inferences per year}$. As reference values we use the EU non-household electricity price of 0.1837 EUR/kWh (second half of 2025) [57] and the EU grid intensity of 242 g CO₂/kWh (2023) [58], bracketed by 0.10–0.30 EUR/kWh and 100–400 g/kWh. Following the AIBO view of AI as an input to managerial decisions [12], the annual cost of a precision p is

$$C(p, \lambda) = N E_{\text{img}}(p) c_{\text{el}} + N K(p) \lambda, \quad (19)$$

with electricity price c_{el} , penalty λ per critical error and $K(p)$ critical errors per image relative to FP32. For the detectors, a critical error is an FP32 detection of a road user (person, bicycle, car, motorcycle, bus, truck) that vanishes in the deployed engine under the score-tolerant definition; for the classifiers, it is a top-1 disagreement. Because $K(\text{FP32}) = 0$, the *break-even penalty*

$$\lambda^*(p) = \frac{(E_{\text{img}}(\text{FP32}) - E_{\text{img}}(p)) c_{\text{el}}}{K(p)} \quad (20)$$

is the cost per critical error above which the saving of p no longer pays off, and the lower envelope of $C(\cdot, \lambda)$ gives the cost-optimal precision for every λ (Fig. C1). We count only the GPU inference energy (no host, cooling or idle power), and we do not count false positives, such as duplicate boxes, as critical errors. The absolute figures describe a fleet whose onboard accelerator has the measured energy per image of the H200. For an embedded GPU they scale with that GPU’s own energy per image, while the relative savings and the structure of Eqs. (19)–(20) carry over.

2.12 Statistical analysis

Accuracy differences to FP32 carry 95% confidence intervals from 1 000 paired bootstrap resamples of the evaluation images (Eq. (A7)). The discriminative power of a deviation score is measured by the ROC AUC, i.e. the Mann–Whitney probability that an erroneous case scores higher than a correct one, with bootstrap intervals; odds ratios carry bootstrap intervals over detections. Where the spread across images matters, medians are reported with interquartile ranges. The network-level relations of Section 3.8 have only one point per network ($n = 5$). For them, rank agreement is the Spearman correlation with the exact one-sided permutation p -value over all 5! orderings; it is one-sided because the model fixes the direction (lower SQNR, higher loss). The relation between head-input SQNR and relative loss is an ordinary least-squares fit of $\log_{10}$ of the relative loss, with t -based 95% confidence intervals of the slope, and we assess its predictive accuracy by leave-one-out, predicting each network from a fit to the other four. All random choices are seeded.

2.13 Static analysis of the FP32 parameters

The first steps of the procedure in Section 3.8 need calibration images. A coarser version needs none at all: the weights and the batch-normalisation (BN) buffers of the FP32 model already determine most of the quantities involved, without an image and without a forward pass (`43_static_analysis.py`). The analysis is static by design. Executing the model on an image folder (`44_validate_static.py`) serves only to check it (Section 3.9).

Weights. For every convolution and linear layer, with BN folded in as in deployment, the weights w_c of output channel c are quantized exactly: to INT8 with $s_c = \max |w_c|/127$ (one scale per output channel, as deployed) and, for comparison, with a single scale for the whole tensor, and to E4M3 with $s_c = \max |w_c|/448$. The SQNR is computed from the quantized and the original weights, without any approximation. By Lemma 1 it is about $52.9 \text{ dB} - 20 \log_{10} \kappa_w$ with $\kappa_w = \max |w_c|/\text{RMS}(w_c)$, so outlier weights lower it, whereas the number of weights does not matter.

Activations. A BN layer stores the running mean μ_c and variance v_c of its input, so its output channel c has mean β_c and standard deviation $\sigma_c = |\gamma_c| \sqrt{v_c/(v_c + \epsilon)}$. We model each channel as a Gaussian, $N(\beta_c, \sigma_c^2)$, passed through the activation function that follows. Which activation follows a BN, and whether its output enters a residual addition instead, is read from the symbolic graph of the model (`torch.fx`, which traces the computation without executing it); for models that cannot be traced, the order of the modules is used. A ReLU turns it into a rectified Gaussian with a point mass

at zero, which uses only the positive half of the symmetric INT8 grid; SiLU leaves a small negative tail bounded by -0.28 . The tensor is the mixture of its channels. If the channels differ in width, this mixture is heavy-tailed: the per-tensor range α is then set by the widest channels, and the narrow ones are quantized coarsely. We choose the range that minimises the error power of Eq. (5) on this mixture. It is evaluated on 2048 synthetic samples per channel, drawn with a fixed seed, by a search over 60 logarithmically spaced candidates between 5% and 100% of the largest sample magnitude. This gives $\kappa = \alpha/\sigma$ and, by Lemma 1, the injected SQNR; for FP8 the maximum is used. The channel imbalance is reported as the ratio of the 90th to the 10th percentile of the channel RMS. Summing the injected noise of all N modelled tensors, and optionally of the weights, gives a static SQNR_{add} by Eq. (16), and the gain of every BN that cannot be folded follows from Eq. (A3). The analysis reports every SQNR also as a relative error $r = 10^{-\text{SQNR}/20}$ in percent of the signal, and it lists the tensors it cannot model (convolution outputs without a BN, other normalisation layers), whose noise is then missing from SQNR_{add} .

How the network enters. By the decomposition of Section 3.8, the static SQNR_{add} falls by 3 dB each time the number of quantized tensors doubles, that is, with depth. Width enters only through the channel imbalance, and spatial size and parameter count do not enter at all. The sum is dominated by the weakest tensors, those with a large κ from outlier channels, depthwise inputs, gates or attention blocks. There are also things the static analysis cannot see: the propagation factor itself, tensors without a BN (such as the outputs of residual additions), the heavier-than-Gaussian tails of real activations, and errors created by decoders and heads.

Example. Take MobileNetV2, which is not used elsewhere in this article. The BN after the projection convolution of block 6 (`features.6.conv.3`, no activation) has $C = 32$ channels with $\beta_c \approx 0$ and σ_c between 1.04 and 12.36, a tensor RMS of 3.60 and a channel imbalance of 3.8. The MSE-optimal range is $\alpha = 40.9$, i.e. $\kappa = 11.36$, and Eq. (6) gives $52.87 - 20 \log_{10} 11.36 = 52.87 - 21.11 = 31.8$ dB; quantizing the samples directly gives 31.2 dB (no clipping). The step $s = \alpha/127 = 0.32$ is only 0.03 of the widest channel but 0.31 of the narrowest. The narrowest channel therefore has an SQNR of only $10 \log_{10}(12 \sigma_c^2/s^2) = 10 \log_{10}(12/0.31^2) = 20.9$ dB, against 42.5 dB for the widest: the shared per-tensor scale makes the narrow channels about 22 dB noisier. Over the whole network, the mean injected noise power of the 52 modelled tensors corresponds to $-10 \log_{10} \langle \rho_n^2 \rangle = 39.1$ dB. This gives $\text{SQNR}_{\text{add}} = 39.1 - 10 \log_{10} 52 = 39.1 - 17.2 = 22.0$ dB without the weights and 20.3 dB with the 53 weight layers. FP8 gives 12.0 dB, because the E4M3 weights (about 32 dB) contribute as much noise as the activations.

3 Results

3.1 Deployment efficiency on the GPU

INT8 is the fastest deployment format for Faster R-CNN, YOLOv10-X and EfficientNet-B0 at both batch sizes and for YOLOv10-S at batch size 8. At batch size 1, YOLOv10-S runs just as fast in FP8 (within 1%), and DenseNet-121 is faster in FP8 (Fig. 3, Table 5). Compared with strict FP32, INT8 lowers the median latency

1.6–8.5-fold at batch size 1 and 2.3–11.8-fold at batch size 8 (Faster R-CNN: from 40.7 to 3.5 ms per batch). It is also 1.12–1.27 and 1.24–1.47 times faster than FP16, so its advantage does not rest on the FP32 baseline alone. At batch size 8, INT8 needs 72–91% less energy per image than FP32 and 37–49% less than FP16 (Faster R-CNN 281 instead of 3 184 mJ, YOLOv10-X 295 instead of 2 477 mJ; 80–91% less if only the energy above idle is counted), and its weights take up 25–32% of their FP32 size. With CUDA graphs, latency is highly deterministic. The jitter is at most 0.015 ms for every INT8 engine and 0.08 ms for any engine, and the slowest of 2 000 executions exceeds the median by at most 23% (EfficientNet-B0 in FP8, 0.64 versus 0.52 ms), which bounds the observed worst-case execution time. The three repeats of each measurement differ by a median of 0.1% in latency and 0.5% in energy (at most 8.8% and 6.0%), far less than the differences between formats. The engine inspector shows that the gains really come from the number format (Table C4): every quantized convolution runs on INT8 kernels, and INT8 cuts the memory traffic per inference to 14–26% of FP32 (FP16: 40–62%, FP8: 26–39%). FP8 is slower than INT8 wherever its activation functions run as separate kernels, which doubles the activation traffic of YOLOv10-X. It is faster for DenseNet-121 (1.23 versus 1.85 ms at batch size 8), because the conventional engines implement the concatenations of this network as hundreds of copies. Power-limit throttling, launch overhead and the cost of output conversion are discussed in Section B.1.

3.2 Task accuracy of the quantized engines

The FP32 engines reproduce the published accuracies (Faster R-CNN 36.95 mAP versus 37.0, YOLOv10-S 46.0 versus 46.3, YOLOv10-X 54.0 versus 54.4). These values may look low to a reader outside object detection, but they are competitive: COCO mAP averages the precision over 80 classes and ten IoU thresholds up to 0.95 [46, 54], and current real-time detectors reach 51–55 [43, 59, 60]. For road users, the FP32 detectors reach an AP₅₀ of 82–86% for persons and 68–76% for cars, and they find 88–93% of the large ones at the operating threshold (Table C6). The ImageNetV2 accuracies of the classifiers, 65.8% and 62.0%, correspond to 77.7% and 74.4% on the original ImageNet validation set [61]; the gap is the known shift of ImageNetV2 [47, 62]. FP16 matches FP32 within 0.02 points, with paired 95% confidence intervals that include zero (Table 5). Every INT8 loss, by contrast, is significant, and its size depends strongly on the architecture. Faster R-CNN loses 0.25 mAP (CI 0.13–0.36), YOLOv10-S and YOLOv10-X lose 2.2 and 6.2 mAP, DenseNet-121 loses 4.7 top-1 points, and EfficientNet-B0 collapses from 65.8% to 24.8%. FP8 loses at most 0.4 points. This is consistent with Eq. (7), by which the relative precision of FP8 does not depend on the range. Only for Faster R-CNN, whose INT8 quantizers operate at moderate ranges ($\kappa \approx 18$), is FP8 marginally worse than INT8 (36.58 versus 36.71). The head-input SQNR follows the accuracy: 15.0, 13.4 and 16.7 dB for FP8 against 11.6, −0.9 and 6.6 dB for INT8 in YOLOv10-S, EfficientNet-B0 and DenseNet-121.

The YOLOv10 losses concentrate on large objects (mAP_{large} −7.7 and −13.2), even though their localisation is unchanged (best IoU 0.947 versus 0.950 for YOLOv10-X). For YOLOv10-X the cause turned out to be structural. Its NMS-free one-to-one head suppresses duplicates implicitly [43], and INT8 breaks this suppression: 15.2% of its

Table 5 Task accuracy, latency, throughput and energy per precision (TensorRT 11.3, NVIDIA H200)

Model	Precision	Accuracy (%)	Δ vs FP32 [95% CI]	p50 bs1 (ms)	p99 bs1 (ms)	Images/s bs8	mJ/image bs8
Faster R-CNN R50-FPN	FP32	37.0	-	6.29	6.32	197	3184
	FP16	36.9	-0.01 [-0.04, +0.02]	0.91	0.92	1571	443
	INT8	36.7	-0.25 [-0.36, -0.13]	0.74	0.74	2312	281
	FP8	36.6	-0.37 [-0.49, -0.25]	0.91	0.92	1770	392
YOLOv10-S	FP32	46.0	-	1.78	1.78	1269	409
	FP16	46.0	+0.01 [-0.03, +0.03]	0.81	0.81	4370	109
	INT8	43.8	-2.24 [-2.56, -1.83]	0.69	0.69	5430	63.3
	FP8	45.6	-0.41 [-0.53, -0.28]	0.68	0.69	4297	96.7
YOLOv10-X	FP32	54.0	-	6.06	6.07	239	2477
	FP16	54.0	-0.01 [-0.07, +0.02]	1.99	2.00	1160	582
	INT8	47.8	-6.21 [-6.52, -5.74]	1.57	1.57	1658	295
	FP8	53.9	-0.15 [-0.31, -0.01]	1.79	1.79	1375	430
EfficientNet-B0	FP32	65.8	-	0.59	0.60	7086	45.7
	FP16	65.8	-0.02 [-0.12, +0.06]	0.42	0.42	12087	20.6
	INT8	24.8	-40.97 [-42.07, -39.77]	0.38	0.38	16564	12.7
	FP8	65.4	-0.39 [-0.79, 0.00]	0.52	0.52	9598	26.0
DenseNet-121	FP32	62.0	-	2.78	2.79	1912	152
	FP16	62.0	-0.02 [-0.11, +0.07]	1.59	1.59	3344	70.5
	INT8	57.4	-4.68 [-5.37, -3.98]	1.39	1.39	4325	40.6
	FP8	61.7	-0.32 [-0.64, +0.02]	0.71	0.71	6510	40.9

Accuracy: COCO val2017 box mAP@[.5:.95] (5000 images) for detectors, ImageNetV2 top-1 (9000 images) for classifiers; 95% CI of the difference from 1000 paired bootstrap resamples of the images. Latency: median (p50) and 99th percentile (p99) of 2000 executions with CUDA graphs; throughput from the mean latency with CUDA graphs. Energy: gross GPU energy per image from the NVML counter during sustained CUDA-graph execution. Median of three repeats. INT8 and FP8 engines keep non-quantized operations in FP16; INT8 uses the entropy calibration of the toolchain on the full calibration set in its stored order (sensitivity: Table C18); Faster R-CNN: backbone and FPN only.

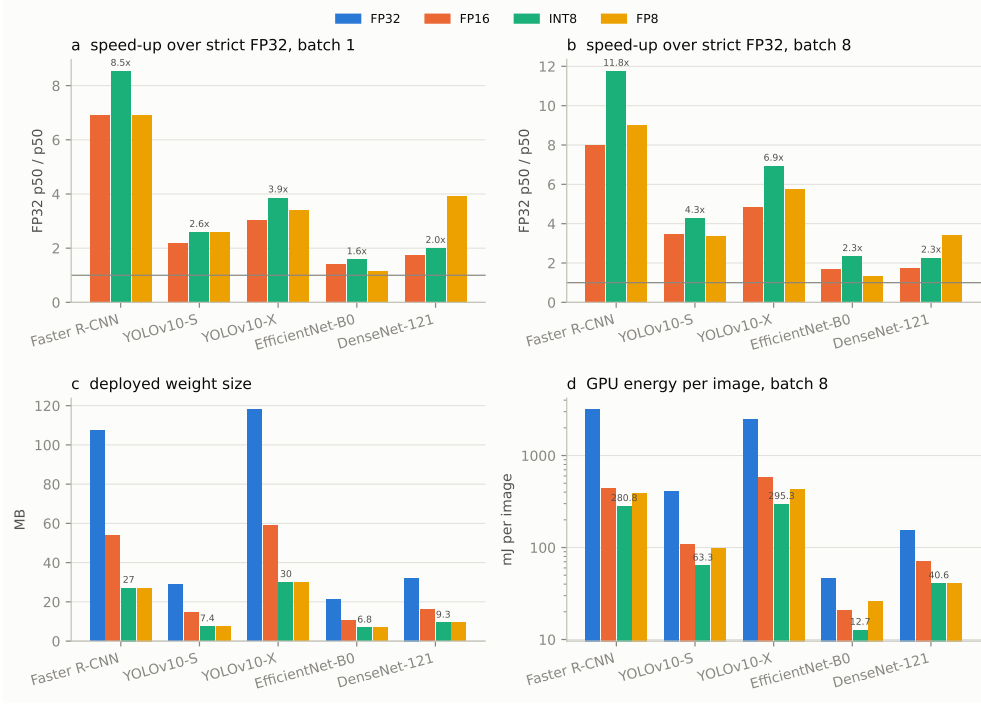


Fig. 3 Deployment efficiency on the NVIDIA H200 (TensorRT 11.3, CUDA graphs). **a, b** Speed-up of the median latency over strict FP32 at batch sizes 1 and 8. **c** Deployed weight size. **d** Gross GPU energy per image at batch size 8 (NVML, sustained CUDA-graph execution; logarithmic axis).

detections above the operating threshold duplicate a higher-scoring box of the same class ($\text{IoU} \geq 0.7$), against 0.25% for FP32 and FP8 (Table C7). Adding class-wise non-maximum suppression, which leaves FP32 and FP8 unchanged, raises the INT8 mAP from 47.8 to 52.4 and $\text{mAP}_{\text{large}}$ from 57.2 to 65.7, at a cost of 0.16 ms per image (10% of the engine latency at batch size 1). The rest of the loss coincides with lower class scores for large objects (median 0.92 in FP32, 0.85 in INT8). YOLOv10-S shows no such effect (0.55% duplicates). Its default quantizer placement, however, had cost it another 0.9 mAP through the quantized TopK and box decoder, a deployment artefact that output-level metrics would have blamed on the network itself (Section A.1).

For the fragile networks, the INT8 accuracy also depends on how the ranges are set (Tables C19 and C18). The toolchain builds its entropy histogram one image at a time, so the first calibration image fixes a lower bound for every clipping range (Section A.1). Calibration subsets that begin with a different image leave Faster R-CNN, the network with the smallest propagation factor, unchanged (± 0.01 mAP). They move YOLOv10-X by up to 2.7 mAP, however, and EfficientNet-B0 from 24.8% to 41.7%. An order-independent entropy calibration raises DenseNet-121 to 60.0% but over-clips the SiLU networks (YOLOv10-X 37.1 mAP, EfficientNet-B0 0.1%). Quantizing only the inputs of convolutions, the placement used by Wu et al. [18],

lifts EfficientNet-B0 to 54.9% with the order-independent calibration but leaves it at 23.8% with the calibration of the toolchain. In short, no INT8 range setting is best for all networks. FP8, whose precision does not depend on the range (Lemma 1), stays within 0.4 points of FP32 for all of them (Section B.2).

3.3 Activation-level deviation

The FP16 engines deviate from FP32 by a head-input SQNR of 36.9–50.5 dB, which is the numerical floor of the comparison. The INT8 deviations are one to two orders of magnitude larger, and they are ordered like the accuracy losses (Table 6): the head-input MRE_{proj} is 2.0% for Faster R-CNN, 5.8% for YOLOv10-S, 13.4% for DenseNet-121, 14.4% for YOLOv10-X and 25.9% for EfficientNet-B0. For every detector, the relative deviation is lower inside the ground-truth object regions than in the background (Faster R-CNN 2.9% vs. 3.3%, YOLOv10-S 3.9% vs. 6.2%, YOLOv10-X 6.4% vs. 13.2%; Fig. 4). This is what the additive model predicts: the same absolute noise is smaller relative to the stronger object responses (Eq. (A4)). The hexbin densities (Fig. 5) show why fragile networks fail. For EfficientNet-B0, a diagonal of deviations equal to minus the FP32 value marks activations that a too coarse per-tensor scale has rounded to zero.

Along the depth (Fig. 6), the SQNR falls within the first layers and then levels off, as Proposition 4 predicts. Fitting the chain recursion to consecutive convolutions gives $g = 0.90, 0.60$ and 0.70 for Faster R-CNN, YOLOv10-S and YOLOv10-X ($R^2 = 0.68, 0.13$ and 0.24 , because branches break the chain order). The corresponding fixed points, 18.5, 13.7 and 5.7 dB, are close to the measured plateaus of 20.3, 13.5 and 6.0 dB. The densely branched EfficientNet-B0 and DenseNet-121 do not form chains; for them the graph form of Proposition 3 is needed. FP8 lies between INT8 and FP16 along the whole depth (median layer SQNR 14.3–21.5 dB, against 5.0–19.8 dB for INT8). The operator ratios (Fig. C5) quantify, per operator type, the attenuation described in the conference paper. Sigmoid gates attenuate ($a_n = 0.30$ – 0.54), max pooling attenuates only where no quantizer precedes it, and non-folded batch normalisation amplifies (median 1.17 per layer in DenseNet). The amplification is predicted by the closed form of Eq. (A3) from the FP32 parameters alone (median 1.24; Spearman $\rho = 0.83$ over 62 layers).

The additive model ties these observations to the fidelity of the network as a whole (Table C8, Fig. C8). On its own, every INT8 quantizer is benign: the median injected SQNR is 26.7–32.5 dB, and Eq. (6) predicts it within 1.1–2.8 dB. Summed with unit propagation factors, the 62–187 quantizers upstream of the head predict head-input SQNRs of 2.6–12.5 dB, without executing a quantized model. The effective propagation factor of Eq. (16) then ranks the networks almost exactly as their relative INT8 loss (Spearman $\rho = 0.9$ over $n = 5$ networks, exact one-sided $p = 0.04$): it is 0.13 for Faster R-CNN, 0.28 and 0.69 for YOLOv10-S and YOLOv10-X, 0.90 for DenseNet-121 and 2.89 for EfficientNet-B0. The injected noise is thus nearly the same across architectures; what decides the INT8 loss is $\bar{\Gamma}$.

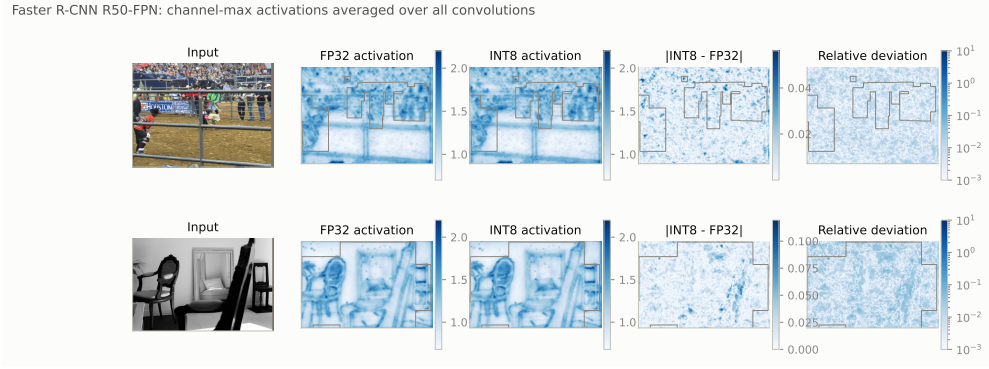


Fig. 4 FP32 and INT8 activation maps of Faster R-CNN. Channel-wise maximum activations of the Faster R-CNN backbone and FPN, averaged over all convolutions, for FP32 and INT8 on a shared colour scale, with the absolute and relative deviation. Grey contours: ground-truth boxes.

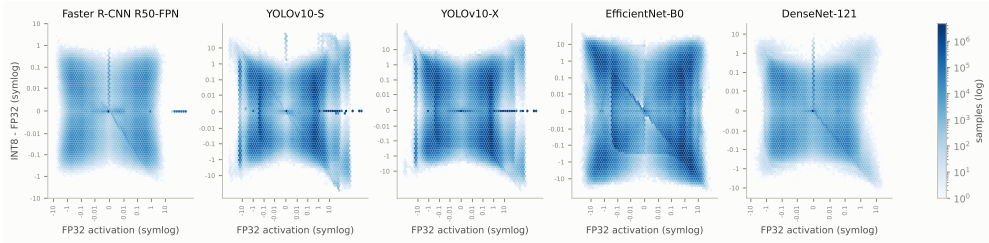


Fig. 5 Density of INT8 deviation versus FP32 activation. Symmetric-logarithmic axes (128 random elements per tensor and image, 500 images per model).

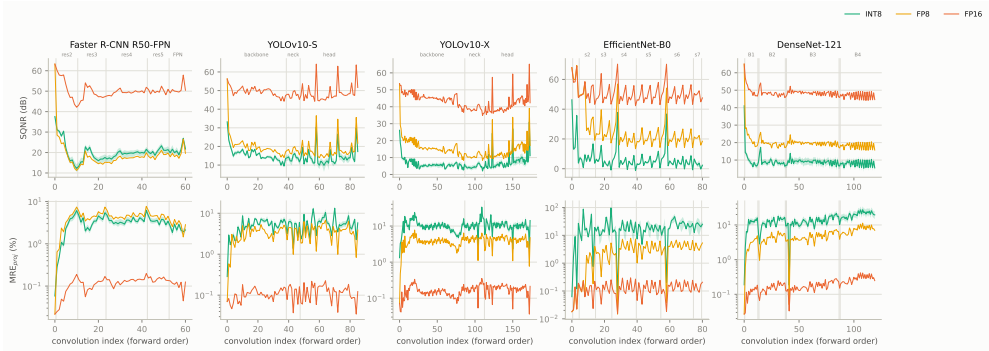


Fig. 6 Layer-wise propagation of quantization deviation. SQNR (top) and median relative error of the channel-max projection (bottom) of every convolution, ordered by depth, for INT8 (median over 500 images, band: interquartile range), FP8 (100 images) and FP16 (500 images; Faster R-CNN 100). Grey lines and labels: network stages.

Table 6 Activation-level deviation of INT8 from strict FP32 (500 images per model)

Model	Head input (INT8)			All convolutions (INT8)			Head SQNR FP16	
	MRE _{proj} (%)	MRE _{elem} (%)		SQNR (dB)	MRE _{elem} (%)	worst SQNR (dB)	(dB)	
Faster R-CNN R50-FPN	2.0 [1.7, 2.3]	7.1 [6.0, 8.3]	21.5 [20.1, 23.2]	8.6 [7.4, 10.1]	12.2	50.5		
YOLOv10-S	5.8 [5.1, 6.6]	16.9 [14.5, 20.6]	11.6 [10.1, 13.0]	17.1 [15.6, 18.9]	9.5	45.7		
YOLOv10-X	14.4 [12.8, 16.1]	37.8 [30.4, 45.9]	4.3 [3.3, 5.4]	37.2 [32.0, 44.0]	2.0	36.9		
EfficientNet-B0	25.9 [20.9, 34.8]	81.4 [72.5, 91.1]	-0.9 [-1.8, -0.2]	48.6 [46.4, 50.5]	-1.2	42.8		
DenseNet-121	13.4 [10.8, 16.9]	20.9 [14.9, 28.1]	6.6 [5.0, 7.9]	32.4 [26.9, 38.3]	5.3	45.2		

Head input: tensors consumed by the task head. Median over images, interquartile range in brackets. All convolutions: per image, the median element-wise relative error over every convolution output. Worst SQNR: lowest per-layer median. INT8 with FP32 fallback, so the deviation is caused by quantization alone; the FP16 column gives the numerical floor of the comparison.

3.4 Deviation and detection errors

At the operating threshold of 0.5, 35% of the validation images contain a strict FP32/INT8 disagreement for Faster R-CNN and YOLOv10-S (27.7% for YOLOv10-X). Most of this is threshold flicker, though. If INT8 counterparts are accepted down to a score of 0.3, only 6–11% of the images remain, and only 0.6%, 2.9% and 1.3% of the FP32 detections genuinely vanish. At the image level, the head-input deviation does not predict these errors (AUC 0.52, 0.40 and 0.39). A trivial covariate, the number of FP32 detections, does much better (0.81, 0.71 and 0.77), and the deviation level itself depends on the architecture. The link does hold locally, however (Fig. C11). Measured in the box of the FP32 detection, that is, where the reference network located the object, the deviation separates vanishing detections with a cross-validated AUC of 0.60–0.67. Their share rises 2.7-, 11.8- and 8.9-fold from the lowest to the highest decile, and each doubling of the local deviation multiplies the odds of vanishing by 1.70 (95% CI 1.36–2.07), 3.22 (2.59–4.01) and 4.25 (3.13–5.95) after adjustment for score margin and box size (Eq. (17)). The data-derived thresholds of the local deviation are 2.1%, 3.8% and 9.8% for Faster R-CNN, YOLOv10-S and YOLOv10-X, and detections above them vanish 2.0, 2.4 and 2.9 times as often as those below (0.94 vs. 0.47%, 4.2 vs. 1.8% and 2.5 vs. 0.85%). The deviation is therefore informative for individual detections, but not as an image-level threshold. Detections that survive are displaced only slightly. Even so, the 95th-percentile distance error of a monocular pedestrian estimate is 5.2% for Faster R-CNN, against 1.2–1.8% for YOLOv10 (Table C5, Section B.4).

3.5 Robustness under adverse conditions

The INT8 Faster R-CNN keeps up with FP32 under all eight conditions (Fig. C12). Over 40 condition–severity pairs its mAP differs by between -1.0 and $+0.2$ points, while the corruptions themselves cost FP32 up to 34 points. Its internal deviation does grow under conditions that lower the image contrast (head-input SQNR from 21.7 to 14.5 dB; Fig. C14). For the YOLOv10 detectors, this growth shows up in the accuracy. On the 500 analysis images, the relative INT8 loss grows from 2.9% and 8.1% on clean images to 14.8% and 36.2% at the strongest contrast reduction (YOLOv10-X: 25.8 instead of 40.4 mAP). At the same time the head-input deviation, which needs no labels, roughly doubles, so it could signal the penalty in a shadow comparison with the FP32 model. FP8, with the same static scales, stays within 0.4–1.9% of FP32 for YOLOv10-X at every contrast severity (Fig. C12), as Corollary 2 predicts.

Looking at individual quantizers, the INT8 penalty follows the $20 \log_{10} c$ of Corollary 2 once the attenuation c is measured at the activations. In YOLOv10-X, clipping in the self-attention block adds another 11–15 dB, whereas no FP8 quantizer loses more than 0.5 dB (Fig. C13). None of the label-free INT8 remedies removed the penalty (robust calibration recovered at most 0.3 mAP, an FP16 input or top-ranked convolution at most 1.2 mAP). Max calibration even doubled it, because its coarser steps cost more than the clipping they avoid (Section B.5).

3.6 From diagnosis to repair

For DenseNet-121, the label-free analysis points to a placement error. Keeping its pre-activation batch normalisation in FP16 raises the INT8 top-1 accuracy from 57.4% to 61.6% (FP32: 62.0%) and the head-input SQNR on the calibration split from 6.6 to 17.5 dB, and the label-free criteria do select this variant (Table C3). The calibration method of the YOLOv10 detectors, on the other hand, cannot be chosen without labels (Section B.6).

Selective precision repairs damage that is concentrated in a few layers (Fig. C15, Table C17). In EfficientNet-B0, keeping just the first depthwise convolution in FP16 raises the top-1 accuracy from 24.8% to 43.9%. The iterative ranking reaches 62.3% with 8 and 66.0% with 20 of the 82 convolutions in FP16, against 45.8% and 57.2% for the one-shot ranking and 34.5–45.7% for random subsets of 20. The architectural heuristic from the literature, all 16 depthwise convolutions in FP16 [24, 37], reaches 65.0%, as much as the iterative ranking with 17 convolutions. It does so, however, at the highest latency and energy of all variants (0.95 ms and 31.0 mJ per image at batch size 8, against 0.72 ms and 23.3 mJ for the iterative ranking at 62.3%). The large gap between one-shot and iterative ranking reflects how far this network departs from the additive model ($\bar{\Gamma} = 2.89$). Along the depth (Fig. 7), eight FP16 convolutions lower the deviation in the first quarter of the network from 12.8% to 0.9%, but every variant above 54% top-1 is slower than pure FP16. YOLOv10-X recovers 5.1 of its 6.2 lost points with 20 ranked convolutions and remains faster than FP16. The convolutions ranked 13 to 20 are the ones that restore its duplicate suppression (15.8% duplicates with 12 and 0.4% with 20 FP16 convolutions; Table C7). YOLOv10-S, which spreads its noise over 79 quantizers of similar strength ($\bar{\Gamma} = 0.28$), gains nothing (43.8–44.0 mAP). A ranking predicted from FP32 statistics alone does not work (EfficientNet-B0 reaches 34.9% with 20 layers): the injected noise is predictable, but its propagation $\Gamma_{n \rightarrow h}$ has to be measured.

3.7 Illustrative fleet-level energy, cost and emissions

Our example fleet has 1 000 vehicles with six cameras at 10 Hz, which amounts to 1.26×10^{12} images per year. At the fleet batch size of 6, INT8 needs 70–91% less energy per image than FP32 and 36–50% less than FP16. With strict FP32, the Faster R-CNN backbone would consume 1 136 MWh per year and YOLOv10-X 885 MWh. INT8 brings this down to 99 and 104 MWh, which at the reference price and grid intensity saves 191 kEUR and 251 t CO₂ per year for Faster R-CNN and 144 kEUR and 189 t for YOLOv10-X (Faster R-CNN: 104–311 kEUR and 104–415 t over the price and grid ranges; Fig. 8, Table C2). FP16 already captures most of this saving; INT8 adds another 62 and 105 MWh per year on top of it. Counting only the energy above idle lowers all savings by about a fifth (Section B.7).

The cost model of Eq. (19) turns these savings into a decision. Relative to FP32, the deployed INT8 engines lose 0.010–0.038 road-user detections per image (FP8 0.003–0.044, FP16 at most 0.002), which means $1\text{--}5 \times 10^{10}$ single-frame critical errors per year for the INT8 fleet. The lower envelope of the cost lines (Fig. 8d) gives the cost-optimal precision. For YOLOv10-X, INT8 is optimal only for $\lambda < 0.72 \mu\text{EUR}$, FP8 up

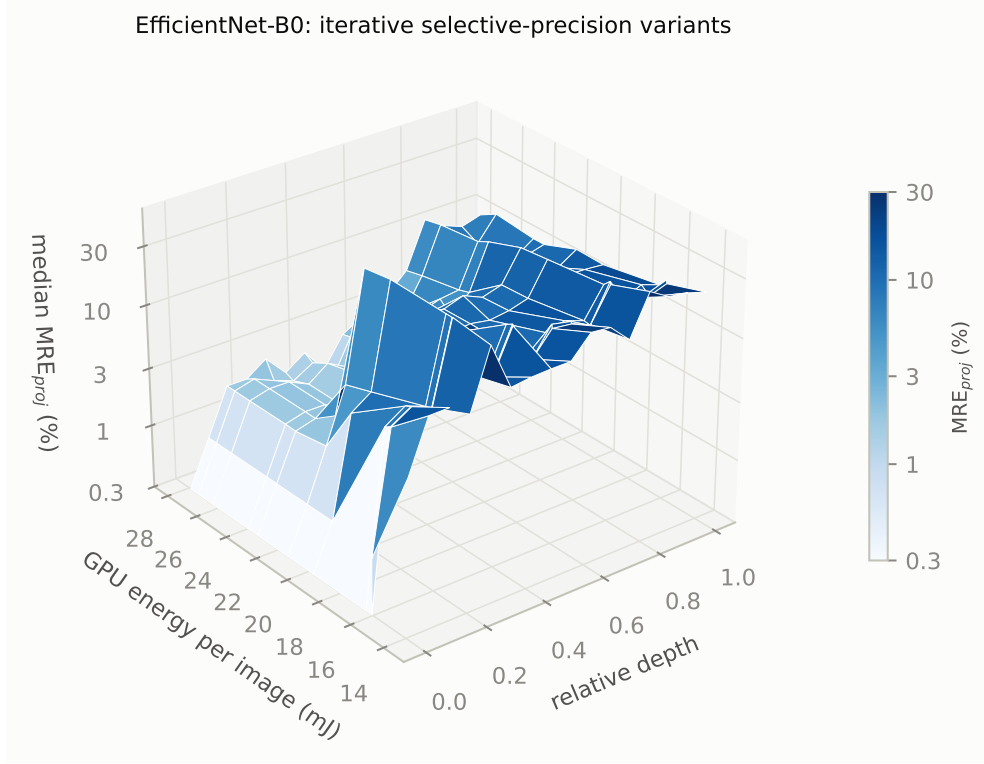


Fig. 7 Deviation, depth and energy of selective-precision variants of EfficientNet-B0. Median MRE_{proj} of every convolution against its relative depth (16 bins) and the GPU energy per image of the variant, for the fully quantized engine and the iterative ranking with $k = 1-20$ convolutions in FP16 (64 images each; batch size 8, shorter protocol without CUDA graphs, comparable only between variants). Logarithmic vertical axis and colour scale; top-1 accuracy in Table C17.

to $3.1 \mu\text{EUR}$, FP16 up to $492 \mu\text{EUR}$, and FP32 above that. For Faster R-CNN and YOLOv10-S, INT8 is optimal below $0.24 \mu\text{EUR}$ and FP16 above. These thresholds are small because the saving is spread over more than 10^{12} frames. In a real perception stack, on the other hand, tracking compensates for most single-frame misses near the score threshold. If such misses are practically free, INT8 captures the full saving; if they carry any measurable cost, FP8 or FP16 keeps most of it (94% and 87% of the INT8 saving for YOLOv10-X) at a fraction of the errors.

3.8 Predicting INT8 tolerance from the noise and propagation model

The analysis so far can be condensed into a procedure that is applied to a new network before it is deployed. Table C9 lists its formulas for INT8, FP8 and FP16, Table C11 applies it to all five networks, and Table C10 walks through every step for YOLOv10-S and YOLOv10-X; Fig. C7 relates the predictions to the measured accuracy. A short

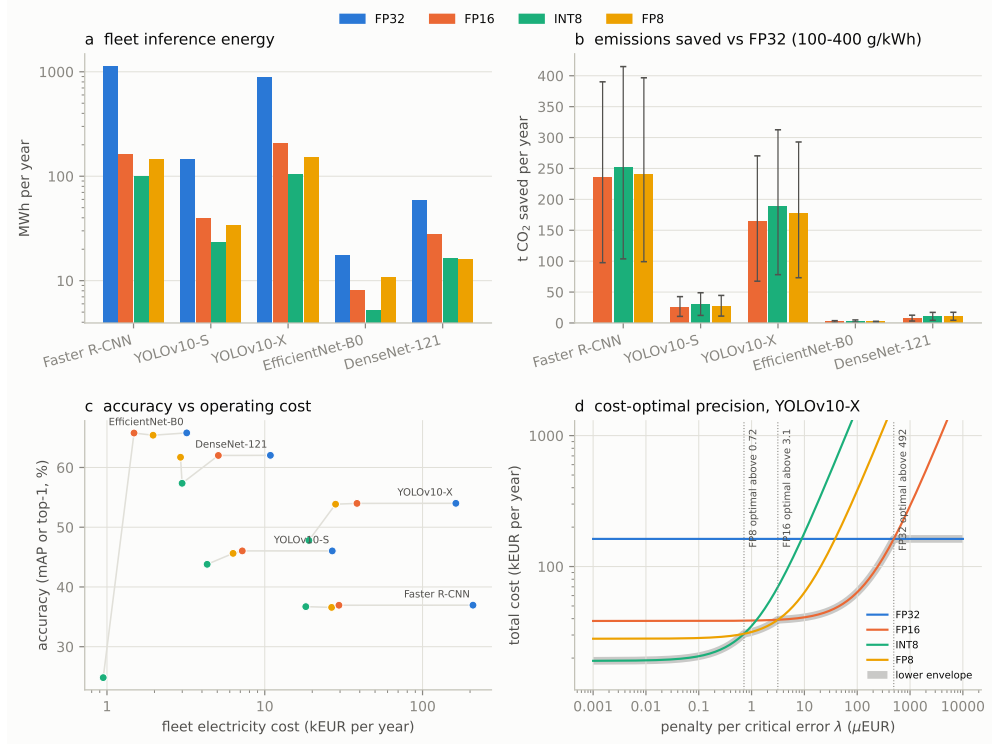


Fig. 8 AIBO fleet scenario. 1 000 vehicles, six cameras at 10 Hz, 16 h/day, measured energy at batch size 6, i.e. the synchronised frames of the six cameras. **a** Annual inference energy per precision. **b** CO₂ saved relative to FP32 (bars: 242 g/kWh; whiskers: 100–400 g/kWh). **c** Task accuracy against annual electricity cost. **d** Total annual cost of Eq. (19) for YOLOv10-X as a function of the penalty per critical error; the grey lower envelope marks the cost-optimal precision.

reminder on reading SQNR differences: 6 dB is a factor of two in the amplitude of the relative error, 20 dB a factor of ten, and a head-input SQNR of 0 dB means that the deviation is as large as the signal.

Step 1: noise of each quantizer, from FP32 statistics. The calibrated scales and the FP32 activations give the normalised range κ of every quantizer and, through Lemma 1, its injected noise. Over the 62–187 quantizers upstream of the head of each network, the median of Eq. (6) lies within 0.6–2.9 dB of the exact value (mean absolute error 1.1–2.8 dB). For the 79 quantizers of YOLOv10-S, for example (median $\kappa = 15.3$), it predicts 29.4 dB against an exact 28.0 dB. The FP8 prediction of Eq. (7), 31.5 dB for every quantizer, matches the exact values of all five networks within 0.02 dB. A single INT8 quantizer is therefore benign and predictable (26.7–32.5 dB). Where the networks differ is in how many quantizers they contain and in how the noise propagates.

Step 2: network-level screening without a quantized model. Summing the injected noise with unit propagation factors (Eq. (16)) predicts a head-input SQNR of 2.6–12.5 dB for INT8 and 10.1–14.6 dB for FP8. This prediction uses FP32 statistics only, and it already orders the networks by their relative INT8 loss with a single exchange

(Spearman $\rho = 0.8$, $n = 5$, exact one-sided $p = 0.07$; Fig. C7a). Faster R-CNN, with the highest value, loses 0.7% of its accuracy, whereas YOLOv10-X and EfficientNet-B0, with the lowest, lose 11.5% and 62.3%. A log-linear fit gives a tenfold loss per 6.2 dB ($R^2 = 0.77$). Predicting each network from the other four, however, misses by up to a factor of 5.9 (EfficientNet-B0). This step should therefore be read as a screening that separates safe from suspicious networks, not as a predictor of the loss. For FP8 the same computation gives at least 10 dB for every network, and indeed every FP8 engine stays within 1.0% of its FP32 accuracy.

Step 3: one measurement of the quantized head input. Once the head-input SQNR of the quantized network is measured, it orders the networks exactly by their relative INT8 loss ($\rho = 1.0$, exact one-sided $p = 0.008$; Fig. C7b), and the loss grows tenfold per 12.2 dB of lost SQNR ($R^2 = 0.96$). Predicted from the other four networks, the relative loss of each network is met within a factor of 1.3–2.2 (Table C11). The largest miss, EfficientNet-B0 with 28.7 instead of 62.3%, lies at the end of the range, where the network collapses. The difference between this measurement and the FP32-only prediction is the effective propagation factor, $\text{SQNR}_h = \text{SQNR}_{\text{add}} - 10 \log_{10} \bar{\Gamma}$. Faster R-CNN gains 8.9 dB by attenuating the injected noise ($\bar{\Gamma} = 0.13$), YOLOv10-S gains 5.6 dB, YOLOv10-X 1.6 dB and DenseNet-121 0.5 dB, whereas EfficientNet-B0 loses 4.6 dB because it amplifies the noise ($\bar{\Gamma} = 2.89$). Note that EfficientNet-B0 does not inject more noise than YOLOv10-X (3.7 against 2.6 dB). Its collapse is a property of the propagation, and only the measurement reveals it.

Step 4: depth and deployment conditions. For the chain-like detectors, the fixed point of Eq. (15) predicts the plateau of the layer SQNR within 0.3 dB (13.7 against 13.5 dB for YOLOv10-S, 5.7 against 6.0 dB for YOLOv10-X; Table C10). When the input changes after calibration, Corollary 2, combined with the measured attenuation at each quantizer input, predicts the change of the injected SQNR at contrast severity 5 within 0.3 dB (−1.2 against −1.2 dB and −0.5 against −0.8 dB), and it correctly predicts that FP8 shows no such change. The cumulative change of the layer SQNR is, however, 2.9–4.5 dB larger, because the shifted input also moves the operating point of the network. Corollary 2 thus predicts the direction of the effect and the difference between INT8 and FP8, but not its full size.

Step 5: choice of format and of layers. A network that passes the screening of step 2 and reaches a high head-input SQNR in step 3, such as Faster R-CNN (21.5 dB, relative loss 0.7%), can be deployed in INT8. For a network with a low head-input SQNR, step 1 tells us that FP8 removes the dependence on the range. Where FP8 kernels are not available, Proposition 3 selects the layers to keep in FP16 by their measured contribution; the propagation factors of single layers cannot be predicted from FP32 statistics (Section 3.6). The procedure is implemented in `scripts/42.predict_int8.py` of the code repository.

Which parts of a network drive the error. By Eq. (16), $\text{SQNR}_{\text{add}} = -10 \log_{10} N - 10 \log_{10} \langle \rho_n^2 \rangle$ holds exactly. The head-input fidelity therefore loses 3 dB each time the number N of quantizers on the way to the head doubles (62 for Faster R-CNN against 187 for YOLOv10-X and DenseNet-121, a difference of 4.8 dB). The mean injected noise, in turn, is dominated by the weakest quantizers, not by the typical one. EfficientNet-B0 has the highest median quantizer SQNR of the five networks

(32.5 dB), yet its mean noise corresponds to 24.2 dB, 8 dB lower, because the inputs of its depthwise convolutions carry channel ranges that differ by a factor of 61 under one per-tensor scale (Table C8). Which quantizers are weak follows from Lemma 1: a large normalised range κ , caused by outlier channels, gates or attention blocks, costs 20 dB per decade. The propagation factor then depends on the operators between the quantizers and the head. Sigmoid gates, max pooling after unquantized layers and the residual trunk of Faster R-CNN attenuate (Section 3.3), whereas non-folded batch normalisation amplifies, by a factor that its parameters predict (Eq. (A3)). Table C16 summarises these effects. Weights matter little for INT8. With one scale per output channel, their median SQNR is 37–43 dB in all seven networks analysed statically (Table C12), against 26–34 dB with a single scale per tensor. In FP8 they reach 32 dB, as much as the activations, and so they contribute equally. The size of a network does not predict its tolerance: Faster R-CNN (26.8 M parameters) is the most robust network and EfficientNet-B0 (5.3 M) the most fragile one. Finally, decoders and heads can turn small deviations into structural errors that the head-input SQNR does not show, such as a quantized TopK or the duplicate suppression of the NMS-free YOLOv10 head (Section 3.2).

Mathematical status. It is worth being explicit about what is proved and what is not. Lemma 1, Corollary 2 and Propositions 3 and 4 are derived under the stated assumptions, with proofs in Sections A.2 and A.3. Their constants were checked numerically: $10 \log_{10}(12 \cdot 127^2) = 52.87$ dB, $10 \log_{10}(8 \ln 2) = 7.44$ dB, and a Monte Carlo simulation of E4M3 rounding gives 31.53 dB against the predicted 31.52 dB. $\bar{\Gamma}$ is a definition, and the decomposition over N is an identity. The relation between head-input SQNR and relative loss, by contrast, is empirical. An ordinary least-squares fit of $\log_{10}$ of the relative loss on the head-input SQNR of the five networks gives a slope of -0.082 per dB (95% confidence interval -0.111 to -0.053 , $p = 0.003$), i.e. a relative loss proportional to $r_h^{1.64}$, with an exponent between 1.05 and 2.23. Two limiting cases bracket this exponent. Suppose a decision flips when the perturbation of its score exceeds its margin, and the margins have a finite density at zero; then the share of flipped decisions grows linearly with the perturbation amplitude (exponent 1). For a metric that is smooth at the FP32 operating point, where its first derivative vanishes, the loss grows quadratically instead (exponent 2). The measured exponent lies between the two, which is consistent with a mixture of threshold flips and changes of the ranking that enters the average precision. We have not derived this interpretation, and the fit rests on five networks and two accuracy metrics.

3.9 Static analysis and its validation on unseen networks

The static analysis of Section 2.13 takes 20–35 s per network on a CPU and needs no image (Table C12). On the five networks of this article, its INT8 SQNR_{add} orders the measured relative INT8 loss with one exchange (Spearman $\rho = 0.9$, exact one-sided $p = 0.04$), just like the FP32-statistics screening of Section 3.8. Its absolute values, however, are 8–12.5 dB higher than those obtained with the calibration of the toolchain (20.6 against 12.5 dB for Faster R-CNN, 14.9 against 2.6 dB for YOLOv10-X). The toolchain quantizes more tensors, sets its ranges by entropy calibration and meets heavier tails than the Gaussian model assumes. We also applied the analysis to

two networks that are not used elsewhere in this article, MobileNetV2 and ResNet-50, and to the two classifiers of the article. To validate it, we executed these networks on ImageNetV2 with simulated quantization on the CPU, placed at exactly the modelled tensors (Table C13; `44_validate_static.py`). Each tensor has one symmetric per-tensor scale, set on 128 calibration images to the MSE-optimal range for INT8 and to the maximum for FP8; weights are quantized per output channel, and 2000 disjoint images are used for evaluation. The validation run is only a check, and the analysis itself executes nothing. Since placement and calibration differ from the TensorRT toolchain, the check tests the method, not the deployment numbers of a particular runtime.

How accurate the static prediction is depends on the number format, as Lemma 1 leads one to expect. For FP8, the injected noise of every quantizer is predicted within 0.02–0.14 dB on all four networks (1–2% of the error amplitude), and the static SQNR_{add} equals the measured one within 0.05 dB (12.0, 11.5, 11.1 and 7.8 dB). The relative precision of a floating-point format does not depend on the distribution, so the Gaussian approximation simply does not matter here. For INT8, the prediction depends on the range and thus on the tails. It is accurate for MobileNetV2, whose quantized tensors are either bounded (ReLU6) or unrectified projections (median +0.4 dB, mean absolute error 1.1 dB, rank correlation 0.98 over its 52 tensors). For EfficientNet-B0, ResNet-50 and DenseNet-121, whose SiLU and unbounded ReLU outputs have heavier tails than the Gaussian model, it is 4–6 dB too optimistic (mean absolute error 5.0–6.3 dB, i.e. the error amplitude is underestimated by a factor of 1.8–2.1), and the ranking of single tensors is weak there (0.34–0.79). Even so, the static SQNR_{add} still orders the four networks by their simulated INT8 loss with one exchange.

Neither format allows the accuracy to be predicted statically, because the propagation factor is not visible in the parameters. The same FP8 noise of about 11 dB becomes 6.8 dB at the head of MobileNetV2 ($\bar{\Gamma} = 3.3$, loss 10.3%) but 12.0 dB at the head of ResNet-50 ($\bar{\Gamma} = 0.9$, loss 2.8%), and 7.8 dB becomes 17.0 dB at the head of DenseNet-121 ($\bar{\Gamma} = 0.12$, loss 0.6%). One measurement of the head input closes this gap. The relation of Section 3.8, calibrated on the TensorRT engines of the five other networks, predicts the relative loss of the simulated engines within the 95% bootstrap interval in six of the eight cases (ResNet-50: 1.5 against 1.1% for INT8 and 3.7 against 2.8% for FP8; DenseNet-121: 2.5 against 2.5% and 1.5 against 0.6%; EfficientNet-B0 FP8: 8.6 against 7.1%; MobileNetV2 FP8: 9.8 against 10.3%). The two misses are EfficientNet-B0 in INT8 (29.7 against 36.8%, a factor of 1.24) and MobileNetV2 in INT8 (1.4 against 0.2%, both below 2%). Table C14 gives the same quantities as relative errors in percent of the signal. For ResNet-50 in FP8, for example, the static analysis predicts an injected error of 2.7% per quantizer and 26.7% for the whole network, exactly as measured; after propagation the head input deviates by 25.1%, and the top-1 accuracy drops by 2.8% relative to FP32. In INT8 the static analysis predicts 1.4% per quantizer and 16.2% for the network, whereas 2.6% and 31.2% are measured; the network attenuates this to 14.3% at the head input (relative loss 1.1%). None of these deviations needs labels: only the accuracy loss does. To test the empirical parts of the analysis beyond the five calibration networks, we repeated the simulated check on 15

networks from four families (Table C15): residual (ResNet-18/34/50/101, ResNeXt-50, RegNetY-800MF), depthwise or inverted-residual (MobileNetV2/V3, MnasNet, ShuffleNetV2, EfficientNet-B0/B1), densely connected (DenseNet-121/169) and inception (GoogLeNet). The loss relation of Section 3.8, calibrated on the five TensorRT networks and not refitted, predicted the measured relative loss within the 95% bootstrap interval in 18 of the 25 cases that lie in its range of validity, with a median miss of a factor of 1.52 and at most 2.97 where the loss exceeded 1%; over these cases the head-input SQNR ranks the losses with a Spearman correlation of 0.83. Five cases lay outside the range of the calibration networks (ResNet-18 in INT8, 26.2 dB; ResNet-34 in INT8, 26.2 dB; ResNet-34 in FP8, 21.8 dB; GoogLeNet in INT8, 22.1 dB; EfficientNet-B1 in INT8, -4.9 dB) and are reported but not used. The propagation factor depended on the block type, but not strictly. Residual networks attenuated the noise or passed it on almost unchanged in both formats ($\bar{\Gamma} \leq 1$: 0.09–0.37 in INT8 and 0.27–0.97 in FP8), the densely connected ones attenuated it (0.32–0.44 and 0.11–0.12), whereas the depthwise networks had the largest factors (median 1.61 in INT8 and 2.19 in FP8) but ranged from attenuation to strong amplification (0.52–6.86 and 0.81–3.28). Block type is therefore an indicator, not a predictor, and $\bar{\Gamma}$ still has to be measured. This check is simulated on the CPU with quantizers at the batch-normalised tensors and uses 1 000 evaluation images for the further networks; it tests the method, not a particular runtime. As a further test outside torchvision, we applied both scripts to a custom 1.2 M-parameter network with a SiLU stem, depthwise blocks with squeeze-and-excitation and Hardswish, ReLU residual bottlenecks, one convolution without BN and a GroupNorm layer, whose BN statistics were collected on 256 ImageNetV2 images (untrained weights, so that only the noise can be compared; code in the repository). The FP8 noise per quantizer was predicted within 0.01 dB (median) and the network value within 0.02 dB (16.60 against 16.58 dB), whereas the INT8 noise was again too optimistic, by 6.5 dB, as for the other networks with SiLU and ReLU outputs; the coverage report listed the convolutions without BN and the GroupNorm layer.

The static analysis is therefore useful for decisions that do not depend on the propagation. It tells whether weights need per-channel scales (ResNet-50: 15.8 against 7.4 dB with one scale per tensor), how much noise FP8 injects into a given network, which batch normalisations amplify noise, which structures (depthwise inputs, imbalanced channels, gates, attention) deserve suspicion, and how a network ranks among measured references. Its INT8 noise estimate is reliable to about 1 dB for networks with bounded or unrectified activations and to about 5–6 dB otherwise. It does not give a fitted accuracy number, and for a good reason. A curve fitted from the static $SQNR_{\text{add}}$ to the measured INT8 loss of the five networks misses by up to a factor of ten in leave-one-out (YOLOv10-X: 114% predicted against 11.5% measured), because the propagation factor is not visible in the parameters, and for FP8 the measured losses do not even follow the static noise (all five networks stayed within 1.0%). What the analysis does give is an honest range. For INT8, the two measured networks next to the new one in the static ranking bracket its loss; for ResNet-50 these are Faster R-CNN and YOLOv10-S, giving 0.7–4.9%, and the simulated measurement found 1.1%. For FP8, the measured networks lost 0.3–1.0% with TensorRT and 0.6–10.3% in the simulated

check, the upper end belonging to a network that amplifies the noise (MobileNetV2, $\bar{\Gamma} = 3.3$); ResNet-50 lost 2.8%. In the same way, the deviation to be expected at the head input follows from the static SQNR_{add} and the range of propagation factors measured so far, $\text{SQNR}_h = \text{SQNR}_{\text{add}} - 10 \log_{10} \bar{\Gamma}$: for ResNet-50 in FP8, $\bar{\Gamma}$ between 0.12 and 3.3 gives a relative deviation of 9–48% of the signal, and 25.1% was measured; in INT8 the range is 6–42%, and 14.3% was measured. With one measurement of the quantized head input, which needs only unlabelled images, the relative loss can be predicted within a factor of 1.4 wherever it exceeds 1% (six of the eight cases), as long as the head-input SQNR lies within the range of the calibration networks (−1 to 22 dB).

4 Discussion

How do these results fit with the literature? The collapse of EfficientNet-B0 under per-tensor INT8 matches the known fragility of depthwise-separable networks [24, 37]. Lemma 1 and $\bar{\Gamma}$ add a quantitative reason: EfficientNet-B0 amplifies the injected noise instead of attenuating it. The accuracy advantage of FP8 confirms the analysis of Kuzmin et al. [29], while the speed advantage of INT8 for four of the five networks confirms the hardware argument of van Baalen et al. [30]; which format is preferable depends on the architecture and on the compiler. The dependence of INT8 accuracy on the range setting agrees with the finding that no calibration method is best for all networks [18] and with the known role of the calibration set [21]. Our results add the order of the calibration images as a further, hidden factor. The PEP-AI ranking can be seen as a gradient-free counterpart of Hessian-based mixed precision [26, 27]. The corruption sensitivity reported for quantized classifiers [38] shows up for the detectors only under contrast loss, which Corollary 2 explains. Finally, NMS-free detectors [43, 60] turn duplicate suppression into a learned function of small score differences, and INT8 can break it.

4.1 Technological limitations

All measurements were taken on a data-centre GPU. The relative effects follow from kernels and memory traffic, which embedded automotive GPUs share, but absolute latencies and energies have to be re-measured on the target hardware. TensorRT supports only static, symmetric, per-tensor activation scales, and this underlies both INT8 failure modes we observed: the collapse of EfficientNet-B0, whose depthwise layers see channel ranges that differ by a factor of 61 under one per-tensor scale, and the low-contrast penalty of Corollary 2. Per-channel or dynamic activation scales would behave differently, and the INT8 accuracies of the fragile networks are specific to the range-setting procedure (Table C19). The analytical models assume additive noise and first-order linearisation. Both fail for values below half a quantization step and under saturation or clipping, so Proposition 3 is a guide rather than a guarantee; this is why we also evaluated the measurement-based iterative variant. The rank correlation between $\bar{\Gamma}$ and the INT8 loss and the relation between head-input SQNR and relative loss rest on five networks and two accuracy metrics. The out-of-sample check covers 15 classifiers in simulated quantization only and should be extended to detectors, transformer architectures and other runtimes. The static analysis models only

batch-normalised tensors, as Gaussian channels. It therefore misses residual sums, tensors without batch normalisation and heavy activation tails, and it cannot see the propagation factor. We did not compare with post-training methods that adapt the weights on calibration data, such as AdaRound, BRECQ or QDrop [20, 22, 23], nor with cross-layer equalisation [24], which addresses the channel imbalance of depth-wise layers directly and would likely reduce the INT8 loss of EfficientNet-B0. The INT8 failures reported here therefore refer to the default toolchain, and some of them may be avoided by such methods. All evaluated networks are convolutional networks with batch normalisation. Transformer-based perception (DETR-type detectors, vision transformers) normalises with LayerNorm, which the static analysis does not model and which was not deployed here, so the conclusions should not be transferred to such networks without a test. COCO contains traffic scenes but is not a driving dataset (no BDD100K, nuScenes or Cityscapes evaluation), the adverse conditions are synthetic rather than recorded (as in ACDC), and for Faster R-CNN only the backbone and FPN were quantized. Finally, a single image-level deviation threshold cannot separate threshold flicker from genuine misses, so release criteria have to be applied locally, at the level of individual detections.

4.2 Practical applicability

PEP-AI quantifies where a quantized model deviates, whether the deviation predicts detection errors and how it changes under adverse conditions, each with an uncertainty, which is what the safety argument for a perception component requires [34, 35]. Even before any engine is built, the static analysis estimates the noise a network will receive (Sections 2.13 and 3.9), and the procedure of Section 3.8 then tells which networks need attention and how much accuracy INT8 will cost them. From the measurements we can give a recommendation for each architecture (Table C20). Faster R-CNN can be deployed in INT8 as it is. YOLOv10-X needs its duplicate suppression restored, either by a class-wise NMS for 0.16 ms per image or by selective precision. For YOLOv10-S, whose loss is spread over the whole network, only a different range setting or FP8 helps. DenseNet-121 recovers its accuracy with batch normalisation in FP16, but at nearly the cost of FP16, and EfficientNet-B0 is best served by FP16. A finding of direct practical value is the dependence on the calibration order: the same 1 000 calibration images gave EfficientNet-B0 24.8% or 41.7% top-1 depending only on which image came first, because the toolchain builds its histogram incrementally (Section A.1). For networks with a large propagation factor, INT8 accuracy should therefore be checked over several calibration orders and procedures and confirmed on labelled data. Where FP8 tensor cores are available, FP8 is the most robust choice for the networks that INT8 damages: it is nearly lossless, insensitive to the range setting and to contrast loss (Lemma 1), and, depending on kernel fusion, faster than FP16. At fleet scale this choice is worth up to 1 GWh, 191 kEUR and 251 t CO₂ per year for 1 000 vehicles, and the break-even analysis tells an operator how much residual risk per frame the saving can pay for [2, 10].

4.3 Future research directions

We see three natural extensions. The first concerns four-bit formats, which the evaluated GPU does not execute natively [63], and per-channel or dynamic activation scales. Lemma 1, Corollary 2 and $\bar{\Gamma}$ carry over to them and would predict, from FP32 statistics and a measured propagation factor, how far the precision can be reduced. The check on 15 networks (Table C15) shows that the block type indicates the propagation factor without determining it: residual and densely connected networks attenuated the noise or passed it on, whereas depthwise networks ranged from attenuation to strong amplification. Predicting $\bar{\Gamma}$ from finer structural features, such as the channel imbalance at the inputs of the depthwise layers or the presence of gates, would close the gap between the static analysis and the accuracy; it needs only unlabelled images and a CPU. The second concerns embedded automotive platforms, recorded adverse-weather driving data [40] and temporal consistency across video frames, where threshold flicker becomes an observable safety property. The third is spatially adaptive computation. The deviation is larger in the background, and for the Faster R-CNN backbone an ideal spatial early exit could skip at most about 31% of the computation (Section B.3). This is an upper bound that motivates such an accelerator design, but does not yet demonstrate it.

5 Conclusion

The main message of this work is that the error of a post-training quantized network separates into two parts. The first part, the noise that each quantizer injects, follows from FP32 statistics. By Lemma 1 it is $52.9 \text{ dB} - 20 \log_{10} \kappa$ for INT8 and a constant 31.5 dB for FP8 E4M3, and the injected noise of a network adds up to SQNR_{add} , which loses 3 dB each time the number of quantized tensors doubles and is dominated by the weakest tensors. The second part, how this noise propagates to the task head, is a property of the network. It is captured by a single factor $\bar{\Gamma}$, which ranged from 0.13 (Faster R-CNN, attenuating) to 2.89 (EfficientNet-B0, amplifying) and can only be seen by measuring the quantized network once. It was this propagation factor, and not the size of the network, that decided the INT8 tolerance: the 26.8 M-parameter Faster R-CNN lost 0.25 mAP, while the 5.3 M-parameter EfficientNet-B0 lost 41 top-1 points. The analysis also separates what depends on the distribution of the activations from what does not. Within the normal range of the format and without overflow, as in every network tested here, the noise of a floating-point format is independent of the distribution and of the calibrated range, so the FP8 noise of a network can be predicted from its parameters alone within 0.02–0.14 dB per quantizer, also for networks never seen before. Whether the FP8 accuracy follows, however, depends on the propagation, and a network that amplifies the noise can lose several percent even in FP8. The noise of INT8, on the other hand, depends on the range and therefore on the tails of the distribution. It is predicted within about 1 dB where the activations are bounded or unrectified, but 4–6 dB too optimistically for SiLU and unbounded ReLU outputs, and it degrades under contrast loss (Corollary 2) and under changes of the calibration sample that FP8 absorbs. The noise model, the propagation bounds and the selection rule are proved under stated assumptions and agree with the measurements to within a

few decibels. The relation between head-input fidelity and accuracy, a tenfold relative loss per 12 dB, is empirical and was fitted on five networks; in a simulated check on 15 networks it predicted the loss within the 95% interval in 18 of 25 cases, with a median miss of a factor of 1.5.

In practice, these findings give a procedure for choosing a number format before deployment. A static analysis of the FP32 weights and batch-normalisation parameters takes less than a minute on a CPU and needs no data; it gives the FP8 noise, flags the structures that are critical for INT8 and ranks a network among measured references. One measurement of the quantized head input on unlabelled images then predicts the relative INT8 loss within a factor of about two. Where INT8 fails, measured layer sensitivities locate the damage, from a single depthwise convolution in EfficientNet-B0 to the layers that implement the duplicate suppression of YOLOv10-X. Where the damage is concentrated, keeping these layers in FP16 restores most of the accuracy (EfficientNet-B0 from 24.8% to 66.0%, YOLOv10-X 5.1 of 6.2 lost points). For single detections, the deviation inside the box predicts whether a detection vanishes (odds ratio 1.7–4.2 per doubling), whereas the image-level deviation does not. INT8 is therefore safe for networks that attenuate the injected noise, and FP8 is the more robust choice for the others, provided their propagation factor is moderate.

The stakes of this choice are measurable. These observations refer to TensorRT 11.3 on a data-centre GPU and to general-purpose benchmarks; absolute latencies and energies on vehicle hardware will differ. On an NVIDIA H200, INT8 was 1.6–11.8 times faster than FP32 and 1.1–1.5 times faster than FP16, and it needed 72–91% less energy per image than FP32. In an illustrative fleet of 1000 vehicles with the H200 energy per image this would amount to up to 1 GWh and 251 t CO₂ per year, and the cost model gives the precision that minimises energy cost plus the penalty for quantization-induced errors.

Acknowledgements. This research was carried out at the Doctoral School of Informatics of the University of Debrecen.

Declarations

- Funding: This research received no specific funding.
- Competing interests: The authors declare no competing interests.
- Ethics approval and consent to participate: Not applicable.
- Consent for publication: Not applicable.
- Availability of data and materials: COCO 2017 and ImageNetV2 are publicly available. The tables, figures and per-layer and per-detection results of the reported run are included in the code repository (directory `reference_results`).
- Materials availability: Not applicable.
- Code availability: The complete pipeline (quantization, deployment, measurement and analysis scripts, configuration and figure generation) is available at <https://github.com/EgyipTomi425/EchteAI>; the code of the conference paper is preserved under the tag `citds-2026`. An archived release will be deposited upon acceptance.
- Authors' contributions: T.M.: conceptualisation, methodology, software, measurements, formal analysis, writing – original draft. A.H. and R.L.: conceptualisation,

supervision, writing – review and editing. All authors read and approved the final manuscript.

- Use of AI tools: The authors used a large language model (Claude, Anthropic) as an assistant while revising the text of the manuscript and while developing parts of the analysis scripts. The research questions, the experimental design and all measurements are the authors’ own. Every reported number is computed by the published scripts from the measured data, and all derivations, tables and figures were checked by the authors and can be reproduced with the code repository. The authors take full responsibility for the content of this article.
- Prior publication: This article extends the conference paper [41]. New in this version are the GPU deployment with strict-FP32, FP16, INT8 and FP8 TensorRT engines for five architectures, the complete-benchmark accuracy with confidence intervals, the latency, jitter, memory-traffic and energy measurements, the analytical noise and propagation model with its experimental tests, the operator-level amplification estimates, the statistical deviation–risk analysis, the robustness study, label-free diagnosis and repair including selective precision, the prediction procedure for INT8 tolerance, the static analysis with its validation on further networks, and the AIBO fleet scenario. Text reused from the conference paper is limited to the definition of the original metrics.

Appendix A Supplementary methods

A.1 Quantization and deployment

Entropy calibration [49] chooses the clipping range

$$\alpha^* = \arg \min_{\alpha} D_{\text{KL}}(P_{\alpha} \parallel \hat{P}_{\alpha}), \quad (\text{A1})$$

where P_{α} is the calibration histogram of $|x|$ truncated at α , with the mass beyond α added to its last bin, and $\hat{P}_{\alpha}$ its 128-level quantized version. In the ONNX PTQ toolchain used here (ModelOpt on ONNX Runtime), the histogram is built incrementally, one calibration image at a time: the first image defines 128 bins over its own range $[-a_1, a_1]$, later images only append bins of the same width, and the candidates of Eq. (A1) are the bin edges from a_1 to the global maximum; moreover, its reference quantization $\hat{P}_{\alpha}$ spreads 128 levels over $[-\alpha, \alpha]$, i.e. 64 per sign instead of the 127 of INT8. The selected range therefore satisfies $\alpha^* \geq a_1$ and depends on which image comes first. For comparison, an order-independent implementation of the original algorithm [49] first determines the global maximum a of $|x|$ over all calibration images, accumulates a 2048-bin histogram on $[0, a]$ and evaluates all candidates from 128 to 2048 bins; both passes are sums and maxima over images and thus invariant to their order. A fused integer convolution computes $y_c = s_x s_c \sum_i \bar{x}_i \bar{W}_{c,i} + b_c$ from 8-bit operands $\bar{x} = Q_{s_x}(x)/s_x$ and $\bar{W}_c = Q_{s_c}(W_c)/s_c$, accumulated in 32-bit integers. In the NMS-free YOLOv10 head the 300 output detections are selected by a TopK over the class scores; quantizing its input restricts the scores to 127 levels and discards every detection scoring below one quantization step. The box sides are decoded by the

distribution focal loss as the expectation $\sum_i p_i i$ over 16 bins, implemented as a fixed-weight convolution whose quantization scales the rounding error with the bin index and the stride. With the default placement YOLOv10-S reached 42.9 mAP; keeping the TopK in FP16 raised it to 43.3 and keeping the box decoder in FP16 as well to 43.8. The FP16 conversion keeps operations whose reference activations exceed the FP16-safe range in FP32. TensorRT 11.3 FP8 kernels failed at run time for dynamic input shapes larger than the optimisation shape (20 of the 55 input shapes of Faster R-CNN on COCO), so dynamic-shape FP8 engines were optimised for the largest shape; this affects only kernel selection, not the arithmetic. For every engine, the precision in which each convolution is actually executed was read back from the TensorRT engine inspector (kernel name and input data type), so that reduced precision is reported only where it is executed. The memory traffic per inference is estimated as

$$T = \sum_{\ell} \sum_{t \in \text{in}(\ell) \cup \text{out}(\ell)} |t| b(t) + \sum_w |w| b(w), \quad (\text{A2})$$

where ℓ runs over the fused layers, $|t|$ is the number of elements of a tensor, $b(t)$ the bytes per element of its stored data type and w the deployed weights, i.e. the DRAM traffic without cross-layer caching. The sensitivity of INT8 accuracy to the calibration sample was measured by calibrating each model on three seeded random halves of its calibration set.

A.2 Quantization noise model

Proof of Lemma 1 INT8, inside the range. By (A1) the rounding error $u = e$ is uniform on $[-s/2, s/2]$, so its mean square is

$$\mathbb{E} u^2 = \frac{1}{s} \int_{-s/2}^{s/2} u^2 du = \frac{1}{s} \cdot \frac{2}{3} \left(\frac{s}{2}\right)^3 = \frac{s^2}{12}.$$

This applies to the share $\Pr(|x| \leq \alpha)$ of the values that lie inside the range, which gives the first term of Eq. (5).

INT8, outside the range. A value with $|x| > \alpha$ is clipped to the largest level $\pm 127s = \pm \alpha$, so its error is $e = \text{sign}(x) \alpha - x$ and $e^2 = (|x| - \alpha)^2$. Taking the expectation over these values gives the second term of Eq. (5).

INT8 without clipping. If no value is clipped, only the first term remains and $\mathbb{E} e^2 = s^2/12$. With $s = \alpha/127$ and $\alpha = \kappa\sigma$,

$$\text{SQNR} = \frac{\mathbb{E} x^2}{\mathbb{E} e^2} = \frac{12 \sigma^2}{s^2} = \frac{12 \sigma^2 \cdot 127^2}{\kappa^2 \sigma^2} = \frac{12 \cdot 127^2}{\kappa^2},$$

and in decibels $10 \log_{10}(12 \cdot 127^2) - 20 \log_{10} \kappa = 10 \log_{10} 193\,548 - 20 \log_{10} \kappa = 52.87 \text{ dB} - 20 \log_{10} \kappa$, which is Eq. (6). The signal σ cancels: only the ratio κ of range to signal matters.

Floating point. Write $|x| = m 2^E$ with significand $m \in [1, 2)$ and integer exponent E . A format with p significand bits (including the implicit leading one) has 2^{p-1} representable numbers in each octave $[2^E, 2^{E+1})$, spaced by $\Delta = 2^{E-p+1}$; for E4M3 ($p = 4$) these are 8 numbers per octave. Round-to-nearest leaves an error that is uniform over one spacing, so by the same integral as above

$$\mathbb{E}[e^2 | x] = \frac{\Delta^2}{12} = \frac{2^{2E-2p+2}}{12} = \frac{x^2}{m^2} \cdot \frac{2^{-2p+2}}{12} = x^2 \frac{2^{-2p}}{3m^2},$$

where we used $2^E = |x|/m$. The relative error power $2^{-2p}/(3m^2)$ depends only on where x lies within its octave, not on the size of x . Averaging over all values, each weighted by its power x^2 ,

$$\mathbb{E} e^2 = \mathbb{E} \left[x^2 \frac{2^{-2p}}{3m^2} \right] = \mathbb{E} x^2 \cdot \frac{2^{-2p}}{3} \cdot \mathbb{E}_w [m^{-2}],$$

where $\mathbb{E}_w$ is the expectation under the power-weighted measure. By (A2), m then has the log-uniform density $1/(m \ln 2)$ on $[1, 2)$, and

$$\mathbb{E}_w [m^{-2}] = \int_1^2 \frac{1}{m^2} \cdot \frac{dm}{m \ln 2} = \frac{1}{\ln 2} \left[-\frac{1}{2m^2} \right]_1^2 = \frac{1}{\ln 2} \left(\frac{1}{2} - \frac{1}{8} \right) = \frac{3}{8 \ln 2}.$$

Hence $\mathbb{E} e^2 = \mathbb{E} x^2 2^{-2p}/(8 \ln 2)$, so that $\text{SQNR} = 8 \ln 2 \cdot 2^{2p}$, and in decibels $20p \log_{10} 2 + 10 \log_{10}(8 \ln 2) = 6.02p + 7.44$ dB, which is Eq. (7). For E4M3, $6.02 \cdot 4 + 7.44 = 31.52$ dB; for FP16, $6.02 \cdot 11 + 7.44 = 73.67$ dB. No κ or σ appears: the floating-point grid scales with the value itself. $\square$

Proof of Corollary 2 INT8. The scale s was fixed during calibration, so the noise power $s^2/12$ does not change, whereas the signal power changes from σ_{cal}^2 to $c^2 \sigma_{\text{cal}}^2$. Without clipping, $\text{SQNR} = 12 c^2 \sigma_{\text{cal}}^2 / s^2$, which is the calibrated value multiplied by c^2 , i.e. a change of $10 \log_{10} c^2 = 20 \log_{10} c$ dB: halving the signal ($c = 1/2$) costs 6 dB.

FP8. Multiplying x by $c = m_c 2^k$ shifts $\log_2 |x|$ by $\log_2 c$. A log-uniform significand means that the fractional part of $\log_2 |x|$ is uniform, and a uniform distribution on a circle stays uniform under any shift. The significand of cx therefore has the same distribution as that of x , and by Lemma 1 the SQNR is unchanged, as long as the scaled values stay in the normal range of the format, i.e. above $2^{-6}s$ for E4M3. $\square$

The calibration trade-off of Eq. (5) is shown in Fig. 2a. A small α lowers the granular noise of the bulk of the distribution but clips its tail; heavier tails move the optimum to larger κ . Max calibration sets κ to the extent of the tail and pays 20 dB per decade of κ ; entropy calibration clips the tail, which favours the bulk but distorts exactly the rare, large activations.

By Eq. (8), INT8 loses 6 dB per halving of the signal, whereas FP8 keeps its precision down to its smallest normal number, 89 dB below α (Fig. 2b). A loss of image contrast is such an attenuation. The contrast corruption maps every pixel to $\bar{x} + c_{\text{pix}}(x - \bar{x})$, with c_{pix} from 0.4 (severity 1) to 0.05 (severity 5), and fog compresses the image content by a factor between 0.4 and 0.25 before adding a smooth haze. Only the structural part of an activation scales with c_{pix} ; biases, mean responses and normalisation are unaffected, so the attenuation c that enters Eq. (8) is the measured ratio of the activation RMS at each quantizer, with $c_{\text{pix}} \leq c \leq 1$. Equation (8) thus predicts an INT8 penalty of $20 \log_{10} c$ at every quantizer, largest where the activation is dominated by image structure, and no penalty for FP8. A distribution shift can also enlarge an activation ($c > 1$); the signal then enters the clipping regime of Eq. (5). The two formats differ here as well: because the FP8 SQNR does not depend on κ , FP8 can be calibrated to the observed maximum at no cost in precision, whereas the INT8 range is kept tight by the granular term of Eq. (5), and entropy calibration places it just above the bulk of the calibration distribution. This prediction is tested quantizer by quantizer and at the level of task accuracy in Section 3.5.

A.3 Error propagation model

For three operator types the gain has a closed form that predicts the direction of the measurement. *Sigmoid gates*, as in SiLU and squeeze-and-excitation blocks, act element-wise with relative gain $|x \sigma'(x)/\sigma(x)| = (1 - \sigma(x))|x|$, which never exceeds $\max_{x>0} x/(1 + e^x) \approx 0.28$ for positive inputs: saturation attenuates the deviation of large activations. *Max pooling* passes on the deviation of the largest element of each window; because the noise of a per-tensor quantizer has the same power at every position, the selected large elements carry the smallest relative error, and $g < 1$. *Inference-mode batch normalisation*, $y_c = \gamma_c(x_c - \mu_c)/\sqrt{v_c + \epsilon} + \beta_c$, scales the deviation of channel c by $\gamma_c/\sqrt{v_c + \epsilon}$ but removes the channel mean μ_c from the signal. For noise of equal power σ_e^2 per element in every channel, as injected by a per-tensor quantizer, the deviation powers before and after the layer are proportional to $C\sigma_e^2$ and $\sigma_e^2 \sum_c \gamma_c^2/(v_c + \epsilon)$; with the running statistics μ_c, v_c as the moments of the input, the signal powers are proportional to $\sum_c(\mu_c^2 + v_c)$ and $\sum_c(\gamma_c^2 v_c/(v_c + \epsilon) + \beta_c^2)$, and the definition of g_n in Eq. (11) gives

$$g_{\text{BN}}^2 = \frac{1}{C} \sum_c \frac{\gamma_c^2}{v_c + \epsilon} \cdot \frac{\sum_c (\mu_c^2 + v_c)}{\sum_c (\gamma_c^2 v_c/(v_c + \epsilon) + \beta_c^2)}, \quad (\text{A3})$$

which exceeds one whenever μ_c^2/v_c is large, as for the non-negative concatenated features that enter every pre-activation batch normalisation of DenseNet. Equation (A3) depends only on the FP32 parameters and is compared with the measurement in Section 3.3.

Proof of Proposition 3 Additive decomposition. Eq. (10) is linear in the injected noises. Applying it node by node from the input to the head, every injected noise δ_n reaches the head through the product of the Jacobians along the paths from n to h , summed over all paths; call this $J_{n \rightarrow h}$. Hence

$$e_h = \sum_n J_{n \rightarrow h} \delta_n, \quad \mathbb{E}\|e_h\|^2 = \sum_n \sum_m \mathbb{E}\langle J_{n \rightarrow h} \delta_n, J_{m \rightarrow h} \delta_m \rangle = \sum_n \mathbb{E}\|J_{n \rightarrow h} \delta_n\|^2,$$

because for $n \neq m$ the noises are zero-mean and uncorrelated, so the cross terms vanish. Dividing by $\|x_h\|^2$ and multiplying and dividing each term by $\mathbb{E}\|\delta_n\|^2$ and $\|x_n\|^2$,

$$r_h^2 = \sum_n \underbrace{\frac{\mathbb{E}\|J_{n \rightarrow h} \delta_n\|^2}{\mathbb{E}\|\delta_n\|^2}}_{\Gamma_{n \rightarrow h}} \cdot \underbrace{\frac{\|x_n\|^2}{\|x_h\|^2} \cdot \frac{\mathbb{E}\|\delta_n\|^2}{\|x_n\|^2}}_{\rho_n^2},$$

which is Eq. (13). $\Gamma_{n \rightarrow h}$ says how strongly the network amplifies (> 1) or attenuates (< 1) the noise of node n on its way to the head, relative to the signal.

Selection. The Jacobians are evaluated at the FP32 operating point, so $\Gamma_{n \rightarrow h}$ does not depend on which other nodes are quantized. Keeping a set S in FP16 sets $\rho_n \approx 0$ for $n \in S$ (the FP16 noise is about 40 dB below the INT8 noise by Lemma 1) and removes exactly these terms from the sum. All terms are non-negative, so the remaining sum is smallest when the removed terms are the k largest ones: replacing any removed term by a larger one that was kept can only lower $r_h^2(S)$. $\square$

Proof of Proposition 4 Unrolled form. For $n = 1$, Eq. (11) gives $r_1^2 = g_1^2 r_0^2 + \rho_1^2$, which is Eq. (14). If Eq. (14) holds for n , then

$$r_{n+1}^2 = g_{n+1}^2 r_n^2 + \rho_{n+1}^2 = r_0^2 \prod_{m=1}^{n+1} g_m^2 + \sum_{k=1}^n \rho_k^2 \prod_{m=k+1}^{n+1} g_m^2 + \rho_{n+1}^2,$$

and the last term is the $k = n + 1$ term of the sum (with an empty product), so the formula holds for $n + 1$.

Bound. With $g_m \leq \bar{g}$ and $\rho_m \leq \bar{\rho}$, each product of j gains is at most $\bar{g}^{2j}$, so

$$r_n^2 \leq \bar{g}^{2n} r_0^2 + \bar{\rho}^2 \sum_{j=0}^{n-1} \bar{g}^{2j} = \bar{g}^{2n} r_0^2 + \bar{\rho}^2 \frac{1 - \bar{g}^{2n}}{1 - \bar{g}^2} = w r_0^2 + (1 - w) \frac{\bar{\rho}^2}{1 - \bar{g}^2}, \quad w = \bar{g}^{2n}.$$

Since $0 \leq w < 1$, this is a weighted average of r_0^2 and $\bar{\rho}^2/(1 - \bar{g}^2)$ and therefore never exceeds the larger of the two, at any depth n .

Fixed point. For constant $g < 1$ and ρ , a stationary value must satisfy $r^2 = g^2 r^2 + \rho^2$, i.e. $r_\infty^2 = \rho^2/(1 - g^2)$. The distance to it shrinks by g^2 per layer, $r_n^2 - r_\infty^2 = g^2 (r_{n-1}^2 - r_\infty^2) = g^{2n} (r_0^2 - r_\infty^2) \rightarrow 0$. In decibels, $\text{SQNR}_\infty = -10 \log_{10} r_\infty^2 = -20 \log_{10} \rho + 10 \log_{10} (1 - g^2)$, which is Eq. (15). For example, ρ corresponding to 30 dB and $g = 0.9$ give $30 + 10 \log_{10} 0.19 = 30 - 7.2 = 22.8$ dB.

Growth. If $g_m \geq g > 1$, all terms of Eq. (14) are non-negative, so $r_n^2 \geq r_0^2 \prod_m g_m^2 \geq g^{2n} r_0^2$. $\square$

A consequence of the additive model concerns the spatial structure of the deviation. A per-tensor quantizer adds noise of the same power at every position, so that before any non-linearity the expected absolute error does not depend on the position while the relative error scales as $1/|f(x)|$; for an object region O and the background B ,

$$\frac{\text{MRE}_O}{\text{MRE}_B} \approx \frac{\text{median}_O |e|}{\text{median}_B |e|} \cdot \frac{\text{median}_B |f|}{\text{median}_O |f|}, \quad (\text{A4})$$

where the first factor equals one for purely additive noise. Regions with stronger responses, typically objects, then show smaller relative deviation.

The sensitivity used for selective precision (Section 2.10) is the SQNR drop across a node, $s_n = 20 \log_{10} a_n$, which by Eq. (12) grows with the injected noise ρ_n relative to the incoming deviation. It ranks nodes like the contributions of Proposition 3 when their propagation factors $\Gamma_{n \rightarrow h}$ are comparable. Where the assumptions fail, for example through clipping or saturation, the error is no longer additive; the iterative strategy, which re-measures the network after every exclusion, does not rely on additivity, and the gap between the one-shot and the iterative ranking measures how far a network departs from the model. Figure C3 summarises the model and the measured quantities.

A.4 Deviation–risk analysis

Logistic models of the local deviation, the score margin, the logarithmic box area and their combinations are compared by five-fold cross-validated ROC AUC. The downstream consequence of box shifts is estimated with a flat-ground pinhole camera at height h_{cam} , which infers the distance of an object from the image row v_b of its

box bottom, $Z = f h_{\text{cam}} / (v_b - v_0)$. A bottom-edge shift $\Delta v = r f H / Z$, expressed as a fraction r of the box height of an object of physical height H , changes the estimate by

$$\frac{|\Delta Z|}{Z} \approx |r| \frac{H}{h_{\text{cam}}}, \quad (\text{A5})$$

independently of the focal length and the distance. The relative shifts r are measured between the FP32 and the deployed engines for all same-class matches ($\text{IoU} \geq 0.5$) of FP32 detections above the threshold on the 5 000 validation images; pedestrians ($H = 1.7$ m) and cars ($H = 1.5$ m) are evaluated for a camera height of 1.65 m, as in the KITTI setup [64]. The flat-ground model is an illustrative assumption; it isolates the contribution of quantization to the distance error of a monocular pipeline.

A.5 Diagnosis and repair

The label-free consistency criterion is evaluated on 256 calibration images $\mathcal{C}$,

$$\text{Cons}(v) = \begin{cases} \text{AP}_{[.5:.95]}(\mathcal{D}_v; \mathcal{D}_{\text{FP32}}^{s \geq 0.25}) & \text{detectors,} \\ \frac{1}{|\mathcal{C}|} \sum_{i \in \mathcal{C}} \mathbb{1}\{\hat{y}_v(i) = \hat{y}_{\text{FP32}}(i)\} & \text{classifiers,} \end{cases} \quad (\text{A6})$$

where $\mathcal{D}_v$ are the detections of variant v and the FP32 detections with a score of at least 0.25 serve as pseudo ground truth; each criterion selects $\arg \max_v$ of its value. The consistency criterion was added after the head-input SQNR had ranked the calibration variants of YOLOv10-S incorrectly; both are reported. The variants are derived from the calibrated, fully quantized model of the base configuration by removing the quantize–dequantize pairs at the activation and weight inputs of the excluded convolutions, and at their outputs unless these feed another quantized convolution; all variants therefore share the same calibration scales and differ only in the excluded layers. Where an exclusion would leave a precision combination for which TensorRT has no kernel, namely a depthwise convolution with INT8 input and a floating-point output, or with FP16 input and a fused INT8 output, the adjacent convolution is kept in FP16 as well, and the effective number of FP16 convolutions is reported. Accuracy, latency and energy are measured as functions of $k \in \{0, 1, 2, 3, 5, 8, 12, 20\}$ (up to 30 for EfficientNet-B0). An iterative variant re-measures the activations after each exclusion and removes the node that is then most sensitive, which accounts for the redistribution of the error. Selective precision is applied where the base configuration still loses more than 0.5 points of accuracy; Algorithm 1 states the procedure.

A.6 Statistical analysis

Uncertainty is quantified by nonparametric bootstrap resampling of the evaluation images. For an accuracy metric M (mAP or top-1) the difference between a precision p and FP32 is resampled in pairs: for $b = 1, \dots, 1\,000$ a sample I_b^* of the images is drawn with replacement and

$$\Delta_b^* = M_p(I_b^*) - M_{\text{FP32}}(I_b^*), \quad (\text{A7})$$

Algorithm 1 PEP-AI-guided selective precision

Require: FP32 model F ; calibrated, fully quantized graph $G_\emptyset$ of the base configuration; analysis images $\mathcal{I}$; budgets $\mathcal{K}$; mode $\in \{\text{one-shot, iterative}\}$

Ensure: variants G_S with accuracy, latency and energy for $|S| \in \mathcal{K}$

```
1:  $S \leftarrow \emptyset$ 
2:  $s_n \leftarrow \text{median}_{i \in \mathcal{I}} [\text{SQNR}_{\pi(n)}(i) - \text{SQNR}_n(i)]$  for every quantized convolution  $n$  of  $G_\emptyset$   $\triangleright F$  vs.  $G_\emptyset$ 
3: for  $k = 1, \dots, \max \mathcal{K}$  do
4:   if mode = iterative and  $k > 1$  then
5:     recompute  $s_n$  for  $F$  vs.  $G_S$   $\triangleright$  accounts for non-additive effects
6:   end if
7:    $S \leftarrow S \cup \{\arg \max_{n \notin S} s_n\}$ 
8:    $G_S \leftarrow G_\emptyset$  without the Q/DQ pairs at the activation and weight inputs of  $S$  and at those outputs of  $S$  that feed no quantized convolution  $\triangleright$  scales unchanged
9:   if  $k \in \mathcal{K}$  then measure accuracy, latency and energy of the engine built from  $G_S$ 
10:  end if
11: end for
```

and the 2.5th and 97.5th percentiles of Δ^* form the 95% confidence interval. For mAP, the per-image matching of COCOeval is computed once and only the accumulation over the resampled images is repeated, so every replicate is an exact COCO evaluation of I_b^* .

Appendix B Supplementary results

B.1 Deployment efficiency

FP16 and FP8 engines of Faster R-CNN reach the 700 W power limit at batch size 8, where the streaming-multiprocessor clock falls to 1.71 and 1.87 GHz, whereas the INT8 engine runs at 650 W and full clock. Energy was measured with CUDA-graph replays, the execution mode of the reported latencies; with direct enqueues, launch gaps lower the sustained throughput of the small engines by up to 26% and raise their energy per image by up to 25%, whereas the large engines agree within 2%. Without CUDA graphs, launch overhead adds 0.03–0.67 ms and, for DenseNet-121 with its 750 kernels, a jitter of up to 0.33 ms at batch size 1. FP8, although it reduces the arithmetic cost as much as INT8, is slower than INT8 wherever its convolutions are compiled by the Myelin backend with the activation functions as separate kernels (24–184 separate point-wise kernels against 1–39 for INT8), which roughly doubles the activation traffic of YOLOv10-X relative to INT8 (1 103 versus 559 MB). In DenseNet-121 the same backend is an advantage: the conventional engines implement each of its concatenations of all preceding feature maps as copies (536–563 reformat layers in FP32, FP16 and INT8), whereas the FP8 engine needs two. Finally, the INT8 engine of Faster R-CNN with FP32 fallback is faster than the deployed one with FP16 fallback (2.90 versus 3.46 ms at batch size 8), because the latter must convert its FP16 feature

maps to FP32 for the floating-point detection heads, which adds six kernels and 11% traffic. The measured speed-ups are thus properties of the compiled engines, and the engine inspector is needed to attribute them to the number format.

B.2 Calibration sample, order and implementation

The calibration subsets were drawn with fixed seeds and kept in their stored order. All EfficientNet-B0 subsets that begin with the second calibration image give 41.3–41.7% (halves and quarters alike), the quarter that begins with the eighth image 40.1%, and the full set, which begins with the first image, 24.8%; recalibrating on the full set reproduces this value exactly (24.82%), and DenseNet-121 behaves alike (57.36%). Two of the three COCO subsets begin with the same image as the full set and reproduce its accuracy (YOLOv10-S 43.82 and 43.83 versus 43.79 mAP), whereas the third begins with another image and reaches 45.31 and 50.45 mAP for YOLOv10-S and YOLOv10-X. Between the full-set and a half-set calibration of EfficientNet-B0, 29% of the activation scales change by more than 25%, some by a factor of 3.4 in either direction, whereas the three half sets agree with each other (correlation of the log-ratios 0.95–1.00). The dependence is therefore on the first image, not on the sample size, and it is strongest where the propagation factor is large: the robust Faster R-CNN is insensitive to every variant. Replacing in the full-set EfficientNet-B0 graph only the scale of the quantizer in front of the convolution that PEP-AI ranks first, 1 of 114 activation scales, by its half-set value raises the accuracy from 24.8% to 34.2%, more than half of the difference; the other 113 scales together give 38.2%, and all 114 reproduce the half-set result of 41.7%. The sensitivity to the calibration order is thus concentrated in the layer that the activation analysis ranks first.

The order-independent calibration sets the clipping ranges to a median of 44%, 54% and 84% of the toolchain ranges for YOLOv10-S, YOLOv10-X and EfficientNet-B0, whose quantized activations are mostly SiLU outputs (DenseNet-121: 81%, Faster R-CNN: 77%), and it clips several squeeze-and-excitation gates of EfficientNet-B0, sigmoid outputs of range $[0, 1]$ with a single value per channel and image, at 12–17% of their range. The toolchain’s lower bound of the first image’s range thus acts as an unintended regulariser that prevents such over-clipping. Quantizing only the inputs of convolutions and matrix multiplications, as in the study of Wu et al. [18], removes the gates from quantization: with the order-independent calibration EfficientNet-B0 then reaches 54.9% instead of 0.1%, whereas the toolchain calibration stays at 23.8%. Wu et al. report 72.1% INT8 top-1 accuracy against 76.9% in FP32 on the original ImageNet validation set for this placement; the remaining difference is consistent with the different weights and with the harder ImageNetV2 evaluation. Finally, the comparison of entropy and max calibration for the YOLOv10 detectors (Table C3) is confounded by the calibration order: a subset that begins with a different image reaches 45.3 and 50.5 mAP, more than max calibration (44.5 and 49.7).

B.3 Activation-level deviation

The chain recursion $r_n^2 = g^2 r_{n-1}^2 + \rho^2$ was fitted by least squares to the median relative errors of consecutive convolutions (Section 3.3); the same fit for FP8 matches its

plateau within 1.5 dB (Fig. C6). Equation (A4) explains the lower relative deviation inside objects. In the eight images per detector for which the averaged projections were stored, the absolute deviation is nearly the same inside and outside objects for Faster R-CNN (ratio 0.97) and YOLOv10-S (0.88), and the object regions carry 1.20 and 1.57 times stronger responses, which predicts relative-error ratios of 0.81 and 0.56 against measured 0.84 and 0.61. For YOLOv10-X the absolute deviation itself is halved inside objects (0.47), so that the saturation of the SiLU gates (Supplementary Section A.3) adds to the magnitude effect (ratio 0.34). EfficientNet-B0 loses most of its fidelity at its first depthwise convolution (43 $\rightarrow$ 13 dB). The median layer SQNR of FP8 is 3.9, 8.1, 10.5 and 16.6 dB above INT8 for YOLOv10-S, YOLOv10-X, DenseNet-121 and EfficientNet-B0, and 2.1 dB below it for Faster R-CNN, whose INT8 quantizers already operate at moderate ranges. A format with constant relative precision thus removes most of the architecture dependence that INT8 shows. At the operator level, convolutions, additions and concatenations attenuate mildly ($a_n = 0.77\text{--}0.97$), and max pooling attenuates in the SPPF block of YOLOv10 (0.67) but not after the quantized ResNet and DenseNet stems (1.2 and 4.6), where fresh rounding noise dominates; for the first dense layer of DenseNet-121, whose concatenated input has the largest mean relative to its spread, Eq. (A3) predicts an amplification of 3.49 against a measured 3.68. For the spatial early exit, the deepest ResNet stage and the FPN account for 53% of the operations of the Faster R-CNN backbone, and a median of 59% of the image area lies outside every annotated object in the 500 analysis images, which bounds the skippable computation at about 31%. Strictly, $\bar{\Gamma}$ characterises the network together with its quantizer placement: with the FP8 placement, which quantizes only convolution inputs, the INT8-derived factors predict the measured FP8 head-input SQNR of YOLOv10-S, EfficientNet-B0 and DenseNet-121 only to within 3.4–5.5 dB.

B.4 Deviation and detection errors

Crowded images offer more opportunities for a flip and, because deviation is lower inside objects, have a lower average deviation, which explains why the image-level deviation does not predict errors while the number of detections does. The median head-input deviation itself differs by a factor of seven between the architectures (2.0% for Faster R-CNN, 14.4% for YOLOv10-X; Table 6), so that any threshold has to be set per architecture; YOLOv10-X even disagrees on fewer images than YOLOv10-S, because its FP32 scores lie further from the threshold. Detections that disappear under INT8 have a higher local deviation inside their box (Faster R-CNN 2.00% vs. 1.79%, YOLOv10-S 4.10% vs. 3.71%, YOLOv10-X 9.87% vs. 8.42%), and local deviation alone separates genuinely vanishing detections with a cross-validated AUC of 0.60–0.67. For flicker around the threshold, the score margin dominates (AUC 0.74–0.92); for detections that genuinely vanish, local deviation is comparable to or stronger than the margin (YOLOv10-S 0.64 vs. 0.53) and consistently adds information to a model of margin and box size (AUC +0.006, +0.021 and +0.022 for Faster R-CNN, YOLOv10-S and YOLOv10-X).

Detections that survive quantization are displaced only slightly, but the tail depends on the architecture. For the deployed YOLOv10 engines, the bottom edge of a matched pedestrian box moves by a median of 0.17–0.23% of the box height (95th

percentile 1.1–1.7%), which by Eq. (A5) corresponds to a 95th-percentile distance error of 1.2–1.8%, or 24–35 cm at a distance of 20 m; cars behave alike (Table C5). The two-stage Faster R-CNN, whose mAP loss is the smallest, has the heaviest tail: its 95th-percentile shift is 5.0% of the box height (a distance error of 5.2%, about 1 m at 20 m), and 5.2% of its matched pedestrians exceed a 5% distance error, against at most 0.8% for YOLOv10. The FP16 engines stay below 0.6% at the 95th percentile. The difference is consistent with the post-processing: the NMS-free one-to-one head of YOLOv10 assigns each object to a single prediction, whereas in Faster R-CNN a small change of the features can alter the proposals and let non-maximum suppression keep a different candidate box. Average precision, which accepts any box with an IoU of at least 0.5, does not reveal this instability.

B.5 Robustness under adverse conditions

For the YOLOv10 detectors, the head-input deviation rises from 5.7% to 11.8% (YOLOv10-S) and from 14.4% to 23.5% (YOLOv10-X) at the strongest contrast reduction. The head-input SQNR falls in both formats, because a low-contrast input also shifts the operating point of the network, but INT8 starts from a much lower fidelity (4.2 against 9.8 dB for YOLOv10-X). For the INT8 Faster R-CNN, Fig. C14 resolves the condition dependence of the deviation along the depth of the network: under fog, the median deviation of the convolutions in each third of the network rises from 2.9–3.6% on clean images to 4.8–5.3% at severity 2 and 5.0–5.8% at severity 5, whereas under low light it stays at 3.0–3.4% in the second and third thirds at every severity and even falls in the first third (from 2.9% to 1.9–2.0%).

The low-contrast penalty of the YOLOv10 detectors could not be removed by the two label-free remedies suggested by these observations. Calibrating on a mixture in which half of the images carry a random adverse condition changed the mAP of both detectors by at most 0.3 points under every tested condition and severity (YOLOv10-X at contrast severity 5: 25.7 versus 25.8; FP32: 40.4). Keeping the input convolution in FP16, which for YOLOv10-X is also the top-ranked convolution of the PEP-AI analysis, recovered 1.2 of the 14.6 lost points at contrast severity 5 and at most 0.5 points elsewhere; for YOLOv10-S, keeping the input or the top-ranked convolution in FP16 changed the mAP by at most 0.6 points.

FP8 confirms the static-scale prediction of Eq. (8) from the other side. With the same static, calibrated scales, its penalty relative to FP32 does not grow with the contrast reduction: for YOLOv10-X it stays between 0.4% and 1.9% of the FP32 mAP at all five contrast severities, whereas the INT8 penalty rises from 8.1% on clean images to 13.6%, 21.1% and 36.2% at severities 3, 4 and 5 (40.4, 39.9 and 25.8 mAP for FP32, FP8 and INT8 at severity 5). Under fog, which compresses the image content by a factor of 0.25–0.4, the FP8 penalty stays below 1% while INT8 loses 10.7–11.8%. For YOLOv10-S, the FP8 penalty is 0.6–1.4% up to severity 3 and 4.5% at severity 4, against 4.0–4.7% and 11.8% for INT8; at severity 5, where the FP32 mAP itself has fallen to 9.3, both formats lose about one point.

Quantizer by quantizer, the INT8 mechanism is the one of Eq. (8), provided that the attenuation c is measured at the activations rather than taken from the pixels (Fig. C13). At contrast severities 1, 3 and 5 ($c_{\text{pix}} = 0.4, 0.2$ and 0.05), the median

RMS of the activations entering the INT8 quantizers of YOLOv10-S falls only to 0.97, 0.95 and 0.87 of its clean value, because biases and mean responses do not scale with contrast, and the median injected SQNR changes by -0.2 , -0.4 and -1.0 dB, as $20 \log_{10} c$ predicts; the stem quantizers, whose inputs are dominated by image structure ($c \approx 0.6$), lose up to 4.7 dB. In YOLOv10-X, the largest losses arise in the partial self-attention block, whose activations grow by up to 68% under low contrast and exceed the range set by entropy calibration ($\alpha = 5.0$, against 62.4 for FP8): clipping lowers the injected SQNR of these quantizers by 11–15 dB. No FP8 quantizer loses more than 0.5 dB. The measured head-input SQNR falls in both formats (YOLOv10-S: from 11.7 to 6.5 dB for INT8 and from 15.0 to 11.2 dB for FP8 at severity 5; YOLOv10-X: from 4.2 to 1.7 and from 9.8 to 7.8 dB), because a low-contrast input also moves the network to another operating point, which raises its propagation factors. INT8, however, starts from a much lower fidelity, so that for YOLOv10-X the same loss brings the head-input deviation close to the size of the signal itself, whereas FP8 retains a margin of almost 8 dB.

Figure C10 shows the effect on a single traffic scene: under contrast severity 4 the head-input SQNR of INT8 YOLOv10-X falls from 3.4 to 2.0 dB and the large van in the centre of the scene vanishes, whereas FP8 keeps all eleven detections at an unchanged 8.5 dB.

B.6 Diagnosis and repair

Table C3 lists the label-free criteria and the accuracy of every placement and calibration variant. Max calibration does not help DenseNet-121 (48.3%) or Faster R-CNN (-0.1 mAP), but it recovers part of the YOLOv10 loss (YOLOv10-S $43.8 \rightarrow 44.5$ mAP, YOLOv10-X $47.8 \rightarrow 49.7$ mAP) and most of the large-object loss (mAP_{large} $55.5 \rightarrow 62.4$ and $57.2 \rightarrow 67.5$); both label-free criteria nevertheless prefer entropy calibration, which resolves the bulk of small activations more finely and clips the rare large ones that carry the evidence of confident detections. Selecting the calibration method of these detectors therefore requires a small labelled validation set, whereas placement errors are identified correctly without labels. The four DenseNet variants are ranked identically by head-input SQNR and by accuracy except for the two best, which differ by 0.2 points. An exploratory score-fidelity criterion, defined after this observation, selects the better variant for YOLOv10-S but not for YOLOv10-X, where max calibration shifts the absolute scores while preserving their order.

For EfficientNet-B0, the one-shot ranking reaches 65.1% with 30 convolutions in FP16 and the iterative ranking 53.0% with only four. Random subsets of 20 convolutions are far less effective (mean 40.8%, range 34.5–45.7% for 20 layers), and the architectural heuristic of keeping all 16 depthwise convolutions in FP16 reaches 65.0% at a latency that exceeds every PEP-AI variant. The repair is effective but not free. Eight FP16 convolutions of EfficientNet-B0 cost 69% more energy per image than the fully quantized engine (shorter protocol of the search, without CUDA graphs). Every precision boundary adds conversion kernels, and on the H200 every variant above 54% top-1 is slower than the pure FP16 engine (0.66 ms at batch size 8), which is therefore the Pareto-optimal choice for EfficientNet-B0 on this platform; PEP-AI identifies this outcome without a search over layer subsets.

YOLOv10-X, whose propagation factor is closer to one ($\bar{\Gamma} = 0.69$), gains little from the first 12 layers (48.2 mAP) but 52.9 mAP with 20 ranked convolutions (26 in FP16), recovering 5.1 of the 6.2 lost points; the layers added between these budgets lie mainly in the P3 and P4 stages of the backbone (8 of 11) and in the class branch of the head. Even so, its latency at batch size 8 (6.3 ms) remains below that of the FP16 engine (6.9 ms). Thirty or forty ranked layers add nothing (53.0 mAP) but raise the latency to 6.8–7.1 ms, and FP8 reaches 53.9 mAP at 5.8 ms, so that on this GPU FP8 dominates every repaired INT8 variant of YOLOv10-X (Fig. C9).

PEP-AI therefore distinguishes two kinds of INT8 damage with different remedies: damage that is concentrated in identifiable layers (EfficientNet-B0, YOLOv10-X, and the batch-normalisation placement of DenseNet-121), which layer-level repair or a change of placement fixes, and damage that is spread over the network (YOLOv10-S), which calls for a different range setting or a higher-precision format.

Ranking the convolutions by the noise injected at their inputs, computed from FP32 statistics alone with unit propagation factors, gives 43.8–43.9 mAP for YOLOv10-S and leaves EfficientNet-B0 at 24.6–25.3% top-1 for up to 12 layers (Section 3.6).

B.7 Fleet scenario

The energy per image at the fleet batch size of 6 is up to 17% higher than at batch size 8, because smaller batches amortise less of the fixed cost. The INT8 savings of Faster R-CNN and YOLOv10-X correspond to an average of 178 and 134 W of GPU power per vehicle during operation; counting only the energy above idle, they amount to 153 kEUR and 201 t and to 115 kEUR and 152 t per year. For the small classifiers the absolute savings are an order of magnitude smaller (12 and 43 MWh). For DenseNet-121, FP8 is cost-optimal below a penalty of $0.03 \mu\text{EUR}$ per critical error, and for EfficientNet-B0 INT8 is optimal only below $0.001 \mu\text{EUR}$ because of its many top-1 disagreements; the upper limit of FP16 for Faster R-CNN is $89 \mu\text{EUR}$.

Appendix C Extended data

Table C1 Energy per operation at 45 nm (Horowitz, ISSCC 2014)

Operation	Energy (pJ)
8-bit integer add	0.03
32-bit integer add	0.1
16-bit float add	0.4
32-bit float add	0.9
8-bit integer multiply	0.2
32-bit integer multiply	3.1
16-bit float multiply	1.1
32-bit float multiply	3.7
32-bit SRAM read (8 kB)	5
32-bit DRAM read	640

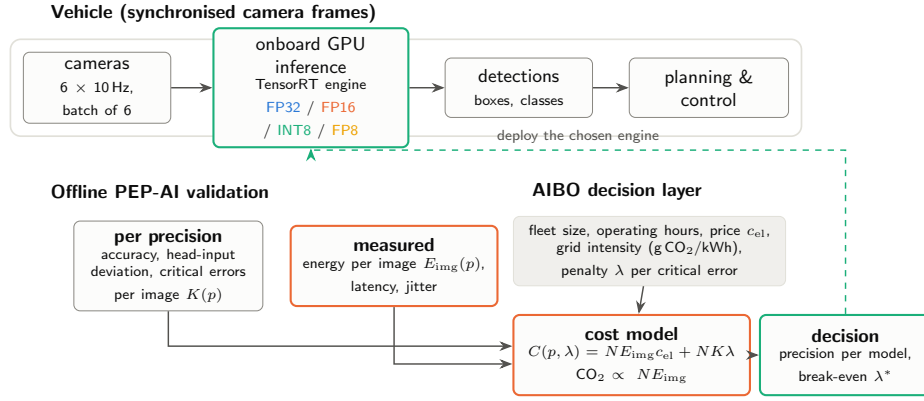


Fig. C1 AIBO view of quantized perception. Offline PEP-AI validation supplies accuracy, deviation, the critical errors per image $K(p)$ and the measured energy per image $E_{img}(p)$ of every precision; combined with fleet parameters, electricity price, grid intensity and a penalty λ per critical error, the cost model of Eq. (19) selects the precision of each deployed engine.

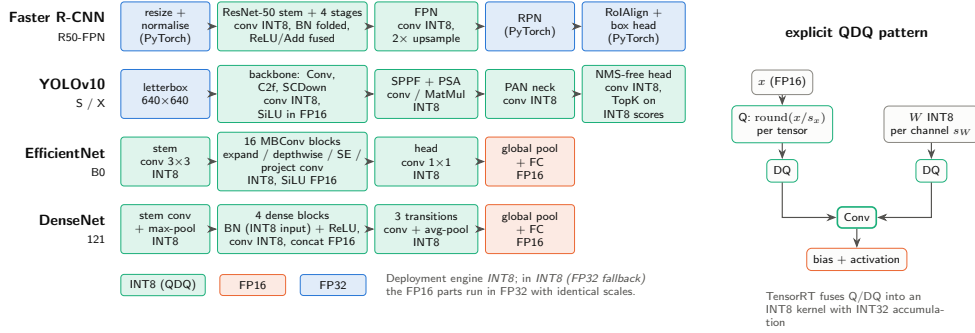


Fig. C2 Precision of the deployed INT8 engines. Precision of each architectural stage (left) and the explicit quantize–dequantize pattern that TensorRT fuses into integer kernels (right). Pre-processing and the Faster R-CNN proposal and ROI heads are not part of the quantized engine.

Table C2 AIBO fleet scenario: annual inference energy, cost, emissions and break-even error penalty per precision

Model	Precision	Energy (MW _h /yr)	Cost (kEUR/yr)	CO ₂ (t/yr)	Saved vs FP32		Critical errors (per 10 ³ images)	Break-even (µEUR/error)
					(kEUR/yr)	(t CO ₂ /yr)		
Faster R-CNN R50-FPN	FP32	1136	209	275	–	–	–	–
	FP16	161	30	39	179	236	1.6	88.8
	INT8	99	18	24	191	251	38.4	3.9
	FP8	144	27	35	182	240	44.4	3.3
YOLOv10-S	FP32	146	27	35	–	–	–	–
	FP16	39	7	9	20	26	0.0	–
	INT8	23	4	6	22	30	9.6	1.9
	FP8	34	6	8	20	27	4.6	3.5
YOLOv10-X	FP32	885	163	214	–	–	–	–
	FP16	209	38	51	124	164	0.2	492
	INT8	104	19	25	144	189	12.8	8.9
	FP8	153	28	37	134	177	2.8	38.1
EfficientNet-B0	FP32	17	3	4	–	–	–	–
	FP16	8	1	2	2	2	3.4	0.4
	INT8	5	1	1	2	3	718.8	0.0025
	FP8	11	2	3	1	2	76.7	0.013
DenseNet-121	FP32	59	11	14	–	–	–	–
	FP16	28	5	7	6	8	3.0	1.5
	INT8	16	3	4	8	10	239.3	0.026
	FP8	16	3	4	8	10	60.8	0.1

1 000 vehicles, 6 cameras at 10 Hz, 16 h/day; measured gross GPU energy per image at batch size 6 (NVIDIA H200), i.e. the synchronised frames of all cameras of a vehicle; electricity price 0.1837 EUR/kWh and grid intensity 242 g CO₂/kWh. Critical errors: FP32 road-user detections that vanish in the deployed engine (score-tolerant definition); classifiers: top-1 disagreements with FP32. Break-even: penalty per critical error at which the precision costs as much as FP32.

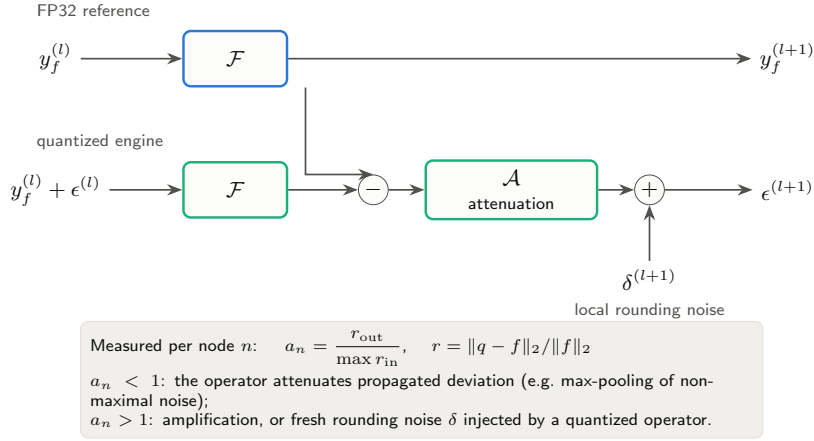


Fig. C3 Error propagation model of the conference paper. Model of Eq. 9 and the per-node amplification ratio used to estimate the attenuation operator $\mathcal{A}$ from measured activations.



Fig. C4 Adverse imaging conditions. The eight conditions applied to a COCO val2017 traffic scene (image 336232) at severity 3 of 5. Low light is gamma darkening with Poisson shot noise; the other conditions follow the COCO-C protocol.

Table C3 INT8 quantization variants and the variant selected by each label-free criterion

Model	Variant	Accuracy (%)	Head SQNR (dB)	FP32 consistency	Picked by
Faster R-CNN R50-FPN	default	36.7	22.0	0.869	SQNR, consistency, accuracy
	max calibration	36.6	18.1	0.826	
YOLOv10-S	default	43.8	11.7	0.961	SQNR, consistency
	max calibration	44.5	7.7	0.945	accuracy
YOLOv10-X	default	47.8	4.1	0.910	SQNR, consistency
	max calibration	49.7	2.2	0.909	accuracy
DenseNet-121	default	57.4	6.6	0.785	
	BN in FP16	61.6	17.5	0.953	SQNR
	max calibration	48.3	3.6	0.602	
	BN in FP16 + max calibration	61.8	16.5	0.957	consistency, accuracy

Accuracy: top-1 (classifiers) or box mAP (detectors) on the evaluation data. Head SQNR and FP32 consistency are computed on the calibration split only. *accuracy* marks the variant a test-set oracle would pick.

Table C4 Composition and memory traffic per inference of the batch-1 TensorRT engines (engine inspector)

Model	Precision	Kernels	Reformats	Convolutions executed in			Activations (MB)	Weights (MB)	Traffic vs FP32
				INT8	FP8	FP16			
Faster R-CNN R50-FPN	FP32	89	23	0	0	0	3021	107.3	1.00
	FP16	72	4	0	0	61	1355	53.7	0.45
	INT8	75	4	61	0	0	772	27.0	0.26
	FP8	90	3	0	60	1	1201	27.0	0.39
YOLOv10-S	FP32	214	37	0	0	0	909	29.0	1.00
	FP16	168	42	0	0	88	374	14.5	0.41
	INT8	173	57	86	0	1	168	7.4	0.19
	FP8	187	0	0	70	18	282	7.4	0.31
YOLOv10-X	FP32	421	68	0	0	0	4042	117.9	1.00
	FP16	326	60	0	0	176	1585	58.9	0.40
	INT8	303	81	174	0	1	559	29.9	0.14
	FP8	363	0	0	110	66	1103	30.0	0.27
EfficientNet-B0	FP32	165	1	0	0	0	194	21.1	1.00
	FP16	134	1	0	0	82	83	10.5	0.44
	INT8	120	3	81	0	1	22	6.8	0.14
	FP8	190	0	0	50	32	62	6.9	0.32
DenseNet-121	FP32	733	545	0	0	0	257	32.2	1.00
	FP16	725	536	0	0	121	163	16.1	0.62
	INT8	744	556	120	0	1	67	9.3	0.26
	FP8	194	2	0	119	2	65	9.3	0.26

Kernels: launched layers after fusion. Reformats: layout or data-type conversions, including the copies that implement concatenations. Convolutions executed in: arithmetic precision of each convolution from its kernel name and input data type (FP32 engines: none of the three). Traffic: bytes of every layer input and output in its stored data type plus the deployed weights, i.e. DRAM traffic without cross-layer caching.

Table C5 Monocular distance error caused by the FP32-to-deployed box shift (COCO val2017, 5 000 images)

Model	Precision	Class	Matched detections	Bottom-edge shift (% of box height)	Distance error (%)		> 5%
					p95	p99	
Faster R-CNN R50-FPN	FP16	person	14 046	0.01	0.08	0.66	0.3
	FP16	car	2 433	0.01	0.08	0.76	0.2
	INT8	person	13 557	0.38	5.02	14.22	5.2
	INT8	car	2 317	0.52	5.67	14.16	5.4
	FP8	person	13 348	0.50	5.90	16.87	6.4
	FP8	car	2 272	0.69	6.27	14.25	5.6
YOLOv10-S	FP16	person	6 508	0.05	0.22	0.43	0.0
	FP16	car	914	0.12	0.59	0.84	0.0
	INT8	person	6 084	0.17	1.15	3.41	0.6
	INT8	car	845	0.31	1.37	2.20	0.1
	FP8	person	6 219	0.23	1.27	2.56	0.4
	FP8	car	851	0.51	1.83	2.61	0.1
YOLOv10-X	FP16	person	7 624	0.05	0.27	0.53	0.0
	FP16	car	1 205	0.14	0.68	0.94	0.0
	INT8	person	7 273	0.22	1.74	4.64	0.9
	INT8	car	1 087	0.36	1.93	3.00	0.1
	FP8	person	7 405	0.16	0.81	1.68	0.1
	FP8	car	1 154	0.35	1.53	2.26	0.0

Same-class matches ($\text{IoU} \geq 0.5$) of FP32 detections with a score of at least 0.5. Distance error from Eq. (A5) for a camera height of 1.65 m and object heights of 1.7 m (person) and 1.5 m (car).

Table C6 Detection quality for road users (COCO val2017, 5 000 images)

Model	Precision	AP ₅₀ (%)		AP (%)		Recall at score ≥ 0.5 (%)		Precision (%)
		person	car	person	car	all $\geq 32^2$ px	large	
Faster R-CNN R50-FPN	FP32	81.9	67.9	51.6	41.0	77.5	89.3	93.0
	FP16	81.9	67.9	51.6	40.9	77.4	89.3	93.0
	INT8	81.8	67.6	51.5	40.8	77.7	89.4	93.2
	FP8	81.7	67.6	51.5	40.7	77.3	89.3	93.3
YOLOv10-S	FP32	81.7	66.7	58.6	44.9	53.6	75.4	88.4
	FP16	81.7	66.7	58.5	44.9	53.7	75.5	88.4
	INT8	81.4	65.9	58.2	44.2	51.5	73.4	87.9
	FP8	81.3	65.9	57.9	44.4	52.3	74.5	88.1
YOLOv10-X	FP32	86.4	75.7	65.1	54.1	62.9	81.8	93.6
	FP16	86.4	75.7	65.1	54.0	62.9	81.8	93.5
	INT8	78.9	68.5	59.7	49.0	62.0	80.7	91.5
	FP8	86.4	75.6	64.9	53.8	62.1	81.4	93.8

AP₅₀: average precision at IoU 0.5; AP: COCO AP@[.5:.95] of the class. Recall and precision: all road-user classes (person, bicycle, car, motorcycle, bus, truck) at the operating threshold (score ≥ 0.5) and IoU ≥ 0.5 , with the greedy score-ordered matching of COCOeval; $\geq 32^2$ px: COCO medium and large objects. Precision is a lower bound, because COCO does not annotate every visible object.

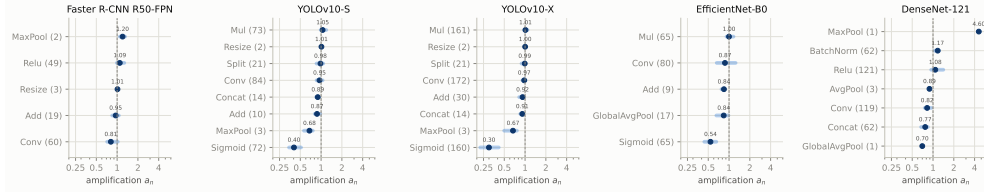
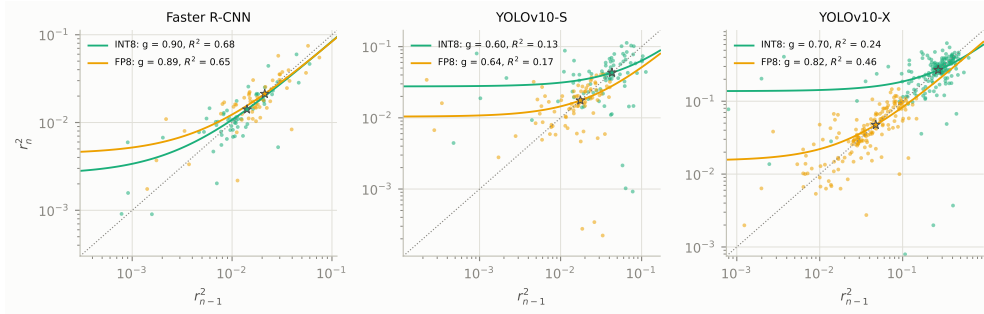
**Fig. C5** Measured amplification ratio a_n per operator type. Dot: median over the nodes of each type; bar: interquartile range. Values left of the dashed line indicate attenuation of the propagated deviation.**Fig. C6** Layer-to-layer recursion of the relative error. Layer-to-layer map of the relative error power $r_{n-1}^2 \mapsto r_n^2$ over consecutive convolutions (medians over the analysis images) for INT8 and FP8, with the least-squares fit of $r_n^2 = g^2 r_{n-1}^2 + \rho^2$ and its fixed point (star, Eq. (15)) on the identity line (dotted).

Table C7 Duplicate detections of the NMS-free YOLOv10 head (COCO val2017, 5 000 images)

Model	Precision	Detections ≥ 0.5	Duplicates (%)	mAP	mAP + NMS	mAP _{large}	mAP _{large} + NMS
YOLOv10-S	FP32	18417	0.48	46.0	46.2	63.2	63.3
	FP16	18441	0.46	46.0	46.2	63.2	63.3
	INT8	16594	0.55	43.8	44.1	55.5	56.3
	INT8 (FP32 fallback)	16600	0.54	43.8	44.1	55.3	56.1
	FP8	17619	0.48	45.6	45.8	63.1	63.3
YOLOv10-X	FP32	22027	0.25	54.0	54.1	70.5	70.5
	FP16	22051	0.26	54.0	54.1	70.4	70.4
	INT8	25057	15.24	47.8	52.4	57.2	65.7
	INT8 (FP32 fallback)	25013	15.17	47.9	52.4	57.4	65.9
	FP8	21437	0.26	53.9	53.9	70.4	70.3
	INT8, 12 ranked conv. in FP16	24503	15.82	48.2	52.9	57.6	66.0
	INT8, 20 ranked conv. in FP16	21002	0.40	52.9	52.9	66.1	66.2

Duplicates: detections with a score of at least 0.5 that overlap a higher-scoring detection of the same class with IoU ≥ 0.7 . + NMS: class-wise non-maximum suppression (IoU 0.7) appended to the engine output as post-processing; on the GPU it takes 0.16 ms per image at batch size 1 and 0.04 ms per image at batch size 8 (torchvision, 105–114 boxes per image). Ranked conv.: selective precision with the PEP-AI ranking (Table C17).

Table C8 Noise injected by the individual INT8 activation quantizers and its accumulation at the head input

Model	Quantizers	Injected SQNR (dB)	Model error (dB)	SQNR _{add} (dB)	SQNR _h (dB)	$\bar{\Gamma}$
Faster R-CNN R50-FPN	62	30.7 [29.5, 32.9]	2.8	12.5	21.5	0.13
YOLOv10-S	79	28.0 [25.9, 30.7]	1.7	6.0	11.6	0.28
YOLOv10-X	187	26.7 [25.0, 28.3]	1.3	2.6	4.3	0.69
EfficientNet-B0	114	32.5 [25.4, 37.9]	1.1	3.7	-0.9	2.89
DenseNet-121	187	30.5 [28.5, 33.3]	2.6	6.1	6.6	0.90

Quantizers upstream of the head-input tensors. Injected SQNR: exact SQNR of each quantizer applied with its calibrated scale to the FP32 activations of 32 analysis images (median and interquartile range). Model error: mean absolute difference between Eq. (6) and the injected SQNR for quantizers without clipping. SQNR_{add} and $\bar{\Gamma}$: Eq. (16); SQNR_h: measured head-input SQNR (500 images, Table 6).

Table C9 Noise and propagation formulas per number format

Quantity	INT8	FP8 (E4M3)	FP16	Source	Status
Scale and grid	$s = \alpha/127$, uniform, 255 levels	$s = \alpha/448$, $p = 11$ cand bits	no scale, $p = 11$	Eq. (1)	definition
Injected noise of one quantizer	$\text{SQNR} \approx 52.9 \text{ dB} - 20 \log_{10} \kappa$	$6.02p + 7.44 = 31.5 \text{ dB}$	$6.02p + 7.44 = 73.7 \text{ dB}$	Lemma 1	proved under (A1), (A2)
Dependence on the range $\kappa = \alpha/\sigma$	-20 dB per decade; clipping above α	none in the normal range	none	Eqs. (6), (7)	proved
Signal scaled by c after calibration	$\Delta \text{SQNR} = 20 \log_{10} c$	0 for $c\sigma \gg 2^{-6}s$	0	Corollary 2	proved, no clipping
Head-input error	$r_h^2 = \sum_n \Gamma_{n \rightarrow h} \rho_n^2$; ρ_n from the row above			Proposition 3	proved, linearised
Prediction from statistics	$\text{SQNR}_{\text{add}} = -10 \log_{10} \sum_n \rho_n^2$ (all $\Gamma = 1$)			Eq. (16)	bound-free estimate
Number N of quantizers	$\text{SQNR}_{\text{add}} = -10 \log_{10} N - 10 \log_{10} \langle \rho_n^2 \rangle$; -3 dB per doubling			Eq. (16)	identity
Propagation factor (one measurement)	$\bar{\Gamma} = 10^{(\text{SQNR}_{\text{add}} - \text{SQNR}_h)/10}$; < 1 attenuating, > 1 amplifying			Eq. (16)	definition
Plateau of a contracting chain	$\text{SQNR}_{\infty} = -20 \log_{10} \rho + 10 \log_{10} (1 - g^2)$ for $g < 1$			Proposition 4	proved
Gain of a non-folded BN	closed form from γ, β, μ, v			Eq. (A3)	derived, equal noise per channel
Layers to keep in FP16	the k largest contributions $\Gamma_{n \rightarrow h} \rho_n^2$			Proposition 3	proved, linearised
Relative loss vs head-input SQNR	$\times 10$ per 12.2 dB , i.e. $\propto r_h^{1.64}$ (95% CI 1.05–2.23)			Section 3.8	empirical, five net-works

α : clipping range, σ : root-mean-square value of the activation, $\kappa = \alpha/\sigma$, p : significant bits including the implicit one, ρ_n : relative noise injected at node n , $\langle \cdot \rangle$: mean over the N quantizers, SQNR_h : measured head-input SQNR, g : propagation gain of a chain, r_h : relative error of the head input. Proofs: Supplementary Sections A.2 and A.3. The empirical relation is a least-squares fit of $\log_{10}$ of the relative loss on the head-input SQNR with a t -based confidence interval.

Table C10 Worked example: the noise and propagation model against the measurement

Quantity (dB)	Source	YOLOv10-S			YOLOv10-X		
		predicted	measured	difference	predicted	measured	difference
INT8 noise of one quantizer, median SQNR	Eq. (6)	29.4	28.0	+1.4	27.2	26.7	+0.6
FP8 noise of one quantizer, median SQNR	Eq. (7)	31.5	31.5	0.0	31.5	31.5	0.0
Head-input SQNR from FP32 statistics ($\Gamma = 1$)	Eq. (16)	6.0	11.6	-5.6	2.6	4.3	-1.6
Plateau of the layer SQNR (chain fit)	Eq. (15)	13.7	13.5	+0.1	5.7	6.0	-0.2
INT8 injected SQNR change, contrast severity 5	Eq. (8)	-1.2	-1.2	0.0	-0.5	-0.8	+0.3
FP8 injected SQNR change, contrast severity 5	Eq. (8)	0.0	0.0	0.0	0.0	0.0	0.0
INT8 layer SQNR change (cumulative), severity 5	Eq. (8)	-1.2	-5.7	+4.5	-0.5	-3.4	+2.9

Medians over the quantizers upstream of the head input (32 analysis images) or over the layers (chain fit, 500 images; contrast, 100 images). INT8 with FP32 fallback. Head input: the difference equals $10 \log_{10} \Gamma$. Contrast rows: prediction $20 \log_{10} c$ with the measured attenuation c of each quantizer input (INT8) and 0 (FP8); the last row compares the injected change with the measured cumulative change of the layer SQNR, which also contains the shift of the operating point.

Table C11 Prediction of INT8 tolerance for the five networks

Model	N	INT8 quantizer (dB)		SQNR _{add} (dB)		SQNR _h (dB)	$\bar{\Gamma}$	INT8 loss (%)		FP8 loss (%)	
		Eq. (6)	exact	MAE	INT8			meas.	LOO _{add}	LOO _h	loss
Faster R-CNN R50-FPN	62	27.9	30.7	2.8	12.5	13.8	21.5	0.13	0.7	0.6	0.5
YOLOv10-S	79	29.4	28.0	1.7	6.0	13.5	11.6	0.28	4.9	8.3	3.8
YOLOv10-X	187	27.2	26.7	1.3	2.6	10.1	4.3	0.69	11.5	48.9	18.1
EfficientNet-B0	114	35.0	32.5	1.1	3.7	14.6	-0.9	2.89	62.3	10.5	28.7
DenseNet-121	187	29.4	30.5	2.6	6.1	10.8	6.6	0.90	7.5	7.2	11.2
											0.5

Quantizer SQNR: median over the quantizers upstream of the head input; Eq. (6) from the normalised range, exact from the calibrated scale applied to FP32 activations (32 images); MAE: mean absolute difference for quantizers without clipping. FP8: every quantizer within 0.02 dB of the 31.5 dB of Eq. (7). N : quantizers upstream of the head. SQNR_{add}: Eq. (16) with unit propagation factors (FP32 statistics only); SQNR_h: measured head-input SQNR of the INT8 engine (one measurement). Relative loss: accuracy loss of the deployed engine divided by the FP32 accuracy (mAP or top-1). LOO_{add}, LOO_h: leave-one-out prediction of $\log_{10}$ of the relative loss from a straight line fitted to the other four networks (all five: 6.2 dB per decade, $R^2 = 0.77$ for SQNR_{add}; 12.2 dB per decade, $R^2 = 0.96$ for SQNR_h).

Table C12 Static analysis of the FP32 parameters

Model	Params (M)	DW	N	Ch. spread		Weights		INT8 per t.	Act. INT8 (dB)	SQNR _{add}		BN gain	Meas. (dB)
				med.	max	per ch.	per t.			INT8	FP8		
Faster R-CNN R50-FPN	26.8	0	53	2.0	1539	38.3	29.1	38.7	20.6	14.3	—	—	12.5
YOLOv10-S	8.1	24	99	1.8	5	39.8	32.2	38.8	18.0	11.6	—	—	6.0
YOLOv10-X	31.7	70	185	1.8	851	39.2	31.1	38.4	14.9	8.9	—	—	2.6
EfficientNet-B0	5.2	16	49	3.3	29	42.6	33.0	34.3	16.2	14.6	—	—	3.7
DenseNet-121	7.9	0	121	2.0	1701	38.4	32.7	38.4	17.1	10.7	1.24 (62)	—	6.1
MobileNet V2 [†]	3.5	17	52	3.8	99	43.2	34.2	40.2	22.0	14.5	—	—	—
ResNet-50 [†]	25.5	0	53	2.6	21	37.4	26.1	38.0	19.0	14.3	—	—	—

Computed from the weights and the batch-normalisation buffers only (no image, no forward pass; `43.static.analysis.py`). DW: depthwise convolutions. N : activation tensors modelled from batch normalisation. Channel spread: 90th/10th percentile of the channel RMS under one per-tensor scale. Weights: median exact SQNR over the layers, BN folded, per channel and per tensor (FP8 weights: 31.7–32.3 dB for all networks). Act. INT8: median predicted injected SQNR (Gaussian channels, MSE-optimal range). SQNR_{add}: static, in dB. BN gain: median gain of Eq. (A3) over the non-foldable batch normalisations (their number in brackets). Meas.: SQNR_{add} from FP32 activations and the calibrated scales of the toolchain (Table C11).[†] not used elsewhere in this article (Table C13). Ranking of the five networks of the article by static INT8 SQNR_{add} against their relative INT8 loss: Spearman $\rho = 0.9$ (exact one-sided $p = 0.04$).

Table C13 Static analysis against simulated quantization

Model	Format	Quantizer SQNR, static – measured			SQNR _{add} (dB)	SQNR _h	$\bar{\Gamma}$	Top-1 (%)	Relative loss (%)	
		median (dB)	MAE (dB)	rank ρ	static measured				measured [95% CI]	predicted
MobileNetV2 [†]	INT8	+0.4	1.1	0.98	20.3	19.2	1.61	59.1	0.2 [−1.0, 1.3]	1.4
	FP8	0.0	0.14	–	12.0	12.0	6.8	53.0	10.3 [7.8, 12.6]	9.8
ResNet-50 [†]	INT8	+5.4	6.3	0.34	15.8	10.1	16.9	0.21	1.1 [0.0, 2.0]	1.5
	FP8	0.0	0.02	–	11.5	11.5	12.0	0.88	2.8 [1.3, 4.3]	3.7
ResNet-18 [†]	INT8	+5.3	5.3	0.18	21.7	18.2	26.2	0.16	−0.9 [−2.3, 0.4]	0.3
	FP8	0.0	0.04	–	15.5	15.5	21.2	0.27	−0.4 [−2.2, 1.4]	0.7
ResNet-34 [†]	INT8	+4.7	5.0	0.46	18.7	15.6	26.2	0.09	−1.2 [−2.7, 0.3]	0.3
	FP8	0.0	0.06	–	13.0	13.0	21.8	0.13	1.9 [0.0, 3.7]	0.6
ResNet-101 [†]	INT8	+4.9	5.1	0.47	12.7	8.9	19.4	0.09	0.3 [−1.2, 1.7]	0.9
	FP8	0.0	0.05	–	8.5	8.5	13.6	0.31	0.4 [−1.6, 2.4]	2.8
ResNeXt-50 [†]	INT8	+3.3	4.0	0.30	16.0	13.9	18.3	0.37	0.3 [−1.1, 1.8]	1.1
	FP8	0.0	0.05	–	11.5	11.5	11.6	0.97	1.6 [−0.3, 3.6]	4.0
RegNetY-800MF [†]	INT8	+3.9	4.9	0.77	15.0	10.9	16.0	0.31	0.8 [−1.1, 2.4]	1.7
	FP8	0.0	0.05	–	11.2	11.2	12.7	0.70	2.8 [0.8, 4.9]	3.2
GoogLeNet [†]	INT8	+4.8	5.6	0.54	18.4	14.0	22.1	0.16	0.7 [−0.9, 2.4]	0.6
	FP8	0.0	0.10	–	11.1	11.1	16.1	0.32	2.7 [0.3, 5.0]	1.7
MobileNetV3-L [†]	INT8	+3.2	3.3	0.89	13.3	11.4	6.4	3.20	3.6 [0.6, 6.6]	10.7
	FP8	0.0	0.20	–	11.8	11.8	7.4	2.69	5.1 [1.8, 8.3]	8.8
MnasNet [†]	INT8	+4.7	5.2	0.50	20.3	15.7	16.3	0.88	2.7 [0.3, 5.0]	1.7
	FP8	0.0	0.07	–	11.9	11.9	10.6	1.34	6.3 [3.4, 9.1]	4.8
ShuffleNetV2 [†]	INT8	+4.7	4.9	0.64	20.0	15.7	18.5	0.52	2.5 [0.4, 4.8]	1.1
	FP8	0.0	0.04	–	11.7	11.7	12.6	0.81	5.0 [2.0, 8.1]	3.3
EfficientNet-B1 [†]	INT8	+6.4	7.6	0.70	10.7	2.2	−4.9	5.08	99.1 [98.2, 99.7]	89.6
	FP8	0.0	0.16	–	9.7	9.6	6.4	2.13	6.0 [3.4, 8.8]	10.8
DenseNet-169 [†]	INT8	+6.4	6.2	0.43	12.4	7.6	11.2	0.44	1.8 [−0.7, 4.4]	4.3
	FP8	0.0	0.05	–	6.4	6.4	15.9	0.11	0.5 [−1.5, 2.6]	1.8
EfficientNet-B0	INT8	+4.0	5.0	0.79	15.2	9.3	1.0	6.86	36.8 [33.9, 39.6]	29.7
	FP8	0.0	0.11	–	11.1	11.1	7.6	2.25	7.1 [5.0, 9.0]	8.6
DenseNet-121	INT8	+6.2	6.1	0.36	14.1	9.2	14.2	0.32	2.5 [0.8, 4.1]	2.5
	FP8	0.0	0.04	–	7.8	7.8	17.0	0.12	0.6 [−0.9, 1.9]	1.5

ImageNetV2, 128 calibration and 2000 evaluation images, CPU simulation (44.validate.static.py): symmetric per-tensor quantizers on every batch-normalised tensor (INT8: MSE-optimal range; FP8 E4M3: maximum), weights per output channel. Top-1 of FP32: MobileNetV2[†] 59.2%, ResNet-50[†] 69.4%, ResNet-18[†] 55.1%, ResNet-34[†] 59.0%, ResNet-101[†] 68.0%, ResNeXt-50[†] 68.3%, RegNetY-800MF[†] 64.7%, GoogLeNet[†] 56.4%, MobileNetV3-L[†] 61.0%, MnasNet[†] 59.9%, ShuffleNetV2[†] 55.5%, EfficientNet-B1[†] 66.2%, DenseNet-169[†] 61.4%, EfficientNet-B0 65.8%, DenseNet-121 61.1%. Quantizer SQNR: static prediction from the BN parameters against the exact injected SQNR on 64 evaluation images. SQNR_{add} includes the weight quantizers. Relative loss: measured with a paired bootstrap interval, and predicted from the measured SQNR_h with the calibration of Table C11. The simulated placement and calibration differ from the TensorRT toolchain, so the INT8 numbers of EfficientNet-B0 and DenseNet-121 differ from Table 5. [†] not used elsewhere in this article.

Table C14 Static and measured deviation as relative error

Model	Format	Injected per quantizer (%)		Injected, whole network (%)		Head input (%)		Relative loss (%)	
		static	measured	static	measured	measured	measured	measured	measured
MobileNetV2 [†]	INT8	1.0	1.0	9.6	11.0	13.9	0.2	13.9	0.2
	FP8	2.6	2.6	25.0	25.1	45.5	10.3	45.5	10.3
ResNet-50 [†]	INT8	1.4	2.6	16.2	31.2	14.3	1.1	14.3	1.1
	FP8	2.7	2.7	26.7	26.7	25.1	2.8	25.1	2.8
ResNet-18 [†]	INT8	1.2	2.2	8.2	12.3	4.9	-0.9	4.9	-0.9
	FP8	2.6	2.7	16.7	16.8	8.7	-0.4	8.7	-0.4
ResNet-34 [†]	INT8	1.3	2.2	11.6	16.6	4.9	-1.2	4.9	-1.2
	FP8	2.6	2.7	22.4	22.4	8.1	1.9	8.1	1.9
ResNet-101 [†]	INT8	1.5	2.6	23.2	35.7	10.8	0.3	10.8	0.3
	FP8	2.6	2.7	37.5	37.6	21.0	0.4	21.0	0.4
ResNeXt-50 [†]	INT8	1.4	2.1	15.8	20.2	12.2	0.3	12.2	0.3
	FP8	2.6	2.7	26.5	26.6	26.2	1.6	26.2	1.6
RegNetY-800MF [†]	INT8	1.7	2.7	17.8	28.5	15.8	0.8	15.8	0.8
	FP8	2.7	2.7	27.6	27.7	23.1	2.8	23.1	2.8
GoogLeNet [†]	INT8	1.0	1.7	12.0	20.0	7.9	0.7	7.9	0.7
	FP8	2.6	2.7	27.8	27.9	15.7	2.7	15.7	2.7
MobileNetV3-L [†]	INT8	1.6	2.3	21.7	26.8	47.9	3.6	47.9	3.6
	FP8	2.6	2.7	25.8	25.8	42.4	5.1	42.4	5.1
MnasNet [†]	INT8	1.1	1.9	9.6	16.3	15.3	2.7	15.3	2.7
	FP8	2.6	2.6	25.4	25.4	29.4	6.3	29.4	6.3
ShuffleNetV2 [†]	INT8	1.0	1.8	10.0	16.5	11.9	2.5	11.9	2.5
	FP8	2.6	2.6	26.1	26.1	23.5	5.0	23.5	5.0
EfficientNet-B1 [†]	INT8	2.6	5.4	29.3	77.7	175.2	99.1	77.7	99.1
	FP8	2.6	2.6	32.8	33.0	48.1	6.0	48.1	6.0
DenseNet-169 [†]	INT8	1.2	2.6	23.9	41.7	27.7	1.8	27.7	1.8
	FP8	2.6	2.6	48.1	48.1	16.0	0.5	16.0	0.5
EfficientNet-B0	INT8	2.2	3.5	17.4	34.1	89.3	36.8	34.1	89.3
	FP8	2.6	2.7	27.8	27.9	41.9	7.1	41.9	7.1
DenseNet-121	INT8	1.2	2.5	19.7	34.5	19.6	2.5	19.6	2.5
	FP8	2.6	2.6	40.7	40.7	14.2	0.6	14.2	0.6

Relative error $r = \|\epsilon\|/\|f\| = 10^{-\text{SQNR}/20}$ in percent of the signal, from the SQNR values of Table C13. Injected per quantizer: median over the quantizers; whole network: SQNR_{add} including the weights, with unit propagation factors; head input: measured at the input of the last linear layer, i.e. after propagation. Static values need no image; measured values need unlabelled images, and only the relative loss needs labels. [†] not used elsewhere in this article.

Table C15 Out-of-sample check of the propagation factor and of the loss relation

Model	Block type	FP32 top-1 (%)	Format	SQNR _h (dB)	$\bar{\Gamma}$	Relative loss (%)		In range
						measured [95% CI]	predicted	
MobileNetV2 [†]	depthwise	59.2	INT8 FP8	17.1 6.8	1.61 3.28	0.2 [-1.0, 1.3] 10.3 [7.8, 12.6]	1.4 9.8	yes yes
ResNet-50 [†]	residual	69.4	INT8	16.9	0.21	1.1 [0.0, 2.0]	1.5	yes
ResNet-18 [†]	residual	55.1	FP8	12.0	0.88	2.8 [1.3, 4.3]	3.7	yes
ResNet-34 [†]	residual	59.0	INT8 FP8	26.2 21.2	0.16 0.27	-0.9 [-2.3, 0.4] -0.4 [-2.2, 1.4]	0.3 0.7	no yes
ResNet-101 [†]	residual	68.0	INT8	26.2	0.09	-1.2 [-2.7, 0.3]	0.3	no
ResNeXt-50 [†]	residual	68.3	FP8	21.8	0.13	1.9 [0.0, 3.7]	0.6	no
RegNetY-800MF [†]	residual	64.7	INT8	19.4	0.09	0.3 [-1.2, 1.7]	0.9	yes
GoogLeNet [†]	inception	56.4	FP8	13.6	0.31	0.4 [-1.6, 2.4]	2.8	yes
MobileNetV3-L [†]	depthwise	61.0	INT8	18.3	0.37	0.3 [-1.1, 1.8]	1.1	yes
MnasNet [†]	depthwise	59.9	FP8	11.6	0.97	1.6 [-0.3, 3.6]	4.0	yes
ShuffleNetV2 [†]	depthwise	55.5	INT8	16.0	0.31	0.8 [-1.1, 2.4]	1.7	yes
EfficientNet-B0	depthwise	66.2	FP8	12.7	0.70	2.8 [0.8, 4.9]	3.2	yes
DenseNet-169 [†]	dense	61.4	INT8	22.1	0.16	0.7 [-0.9, 2.4]	0.6	no
EfficientNet-B1 [†]	depthwise	65.8	FP8	16.1	0.32	2.7 [0.3, 5.0]	1.7	yes
DenseNet-121	dense	61.1	INT8	6.4	3.20	3.6 [0.6, 6.6]	10.7	yes
			FP8	7.4	2.69	5.1 [1.8, 8.3]	8.8	yes
			INT8	16.3	0.88	2.7 [0.3, 5.0]	1.7	yes
			FP8	10.6	1.34	6.3 [3.4, 9.1]	4.8	yes
			INT8	18.5	0.52	2.5 [0.4, 4.8]	1.1	yes
			FP8	12.6	0.81	5.0 [2.0, 8.1]	3.3	yes
			INT8	-4.9	5.08	99.1 [98.2, 99.7]	89.6	no
			FP8	6.4	2.13	6.0 [3.4, 8.8]	10.8	yes
			INT8	11.2	0.44	1.8 [-0.7, 4.4]	4.3	yes
			FP8	15.9	0.11	0.5 [-1.5, 2.6]	1.8	yes
			INT8	1.0	6.86	36.8 [33.9, 39.6]	29.7	yes
			FP8	7.6	2.25	7.1 [5.0, 9.0]	8.6	yes
			INT8	14.2	0.32	2.5 [0.8, 4.1]	2.5	yes
			FP8	17.0	0.12	0.6 [-0.9, 1.9]	1.5	yes

ImageNetV2, simulated quantization on the CPU (`44.validate_static.py`), 128 calibration images; evaluation on 2000 images for the networks of Table C13 and on 1000 images for the others. Predicted: from the measured SQNR_h with the relation of Section 3.8, calibrated on the five TensorRT networks. In range: SQNR_h within the range of the calibration networks (-0.9 to 21.5 dB); rows outside it are reported but not used in the summary. Summary over the rows in range: Spearman ρ between SQNR_h and the measured loss -0.83 ($n = 25$, $p = 3e - 07$); prediction within the 95% CI in 18 of 25 cases; for losses above 1%, median miss $\times 1.52$, largest $\times 2.97$. $\bar{\Gamma}$ by block type, median (range): INT8 dense: 0.38 (0.32-0.44, $n = 2$); INT8 depthwise: 1.61 (0.52-6.86, $n = 5$); INT8 residual: 0.26 (0.09-0.37, $n = 4$); FP8 dense: 0.12 (0.11-0.12, $n = 2$); FP8 depthwise: 2.19 (0.81-3.28, $n = 6$); FP8 inception: 0.32 (0.32-0.32, $n = 1$); FP8 residual: 0.70 (0.27-0.97, $n = 5$). [†] not used elsewhere in this article.

Table C16 How network structure and quantization choices affect the quantization error

Property	Effect	Formula	Evidence in this article	Status
<i>Network structure</i>				
Depth: number N of quantized tensors	−3 dB per doubling of N	$\text{SQNR}_{\text{add}} = -10 \log_{10} N - 10 \log_{10} \langle \rho_n^2 \rangle$	62 (Faster R-CNN) against 187 (YOLOv10-X) quantizers: 4.8 dB	identity
Weakest tensors (outlier channels, depthwise inputs, gates, attention)	dominate the mean noise; −20 dB per decade of κ	Lemma 1	EfficientNet-B0: median quantizer 32.5 dB, mean noise 24.2 dB	proved, measured
Channel imbalance under one scale	narrow channels are quantized coarsely	$10 \log_{10}(12 \sigma_c^2 / s^2)$ per channel	MobileNetV2: 20.9 dB for the narrowest, 42.5 dB for the widest channel	proved
Activation function	ReLU uses half of the grid; SiLU and unbounded ReLU outputs have heavy tails	—	static INT8 prediction too optimistic by 0.4 (MobileNetV2) to 6.2 dB (DenseNet-121)	measured
Operators between quantizer and head	sigmoid gates, max pooling after unquantized layers and residual trunks attenuate; non-folded BN amplifies	a_n (Eq. (12)), Eq. (A3)	$\bar{\Gamma} = 0.13$ (Faster R-CNN) to 2.89 (EfficientNet-B0); BN gain median 1.24, $\rho = 0.83$ with the measurement	closed form, measured
Block type	residual and densely connected networks attenuated or passed the noise on ($\bar{\Gamma} \leq 1$); depthwise networks ranged from attenuation to amplification	$\bar{\Gamma}$	15 networks: residual 0.09–0.37 (INT8), 0.27–0.97 (FP8); depthwise 0.52–6.86, 0.81–3.28	measured (simulated); indicator, not predictor
Parameter count, input size	no direct effect	—	Faster R-CNN (26.8 M) most robust, EfficientNet-B0 (5.3 M) most fragile	measured
Decoders and heads	structural errors not visible in the head-input SQNR	—	quantized TopK (−0.9 mAP); broken duplicate suppression of YOLOv10-X (15.2% duplicates)	measured
<i>Quantization choices</i>				
Number format	INT8 noise depends on the range and the tails, FP8 and FP16 noise does not	Eqs. (6), (7)	static FP8 prediction within 0.02–0.14 dB; FP8 loss at most 0.4 points	proved, measured
Weight scales	per channel needed for INT8; FP8 indifferent	Lemma 1 on w	INT8 37–43 dB per channel against 26–34 dB per tensor; FP8 about 32 dB either way	exact
Range setting (method, sample, order)	INT8 accuracy of fragile networks depends on it, FP8 does not	Eq. (5)	EfficientNet-B0 24.8% or 41.7% depending on the first calibration image	measured
Static scales under input shift	INT8 loses $20 \log_{10} c$ and clips for $c > 1$; FP8 unaffected	Corollary 2	YOLOv10-X relative INT8 loss 8.1% $\rightarrow$ 36.2% at contrast severity 5, FP8 at most 1.9%	proved, measured
Placement and selective precision	removing the largest contributions $\Gamma_{n \rightarrow h} \rho_n^2$ removes their noise	Proposition 3	DenseNet-121 BN in FP16: 57.4% $\rightarrow$ 61.6%; EfficientNet-B0, 20 layers: 24.8% $\rightarrow$ 66.0%	proved, measured

Status: *proved*, derived under the stated assumptions (Supplementary Section A.2); *identity*, exact by definition; *exact*, computed without approximation; *measured*, observed in this article; *hypothesis*, a pattern in few networks that remains to be tested. The relative accuracy loss follows from the head-input SQNR by the empirical relation of Section 3.8 (tenfold per 12 dB).

Table C17 Selective precision: accuracy, latency and energy versus the number k of convolutions kept in FP16

Model	Strategy	k (k_{eff})	Accuracy (%)	p50 bs8 (ms)	Energy bs8 (mJ/img)
EfficientNet-B0	INT8 (base)	0	24.8	0.48	13.8
	PEP-AI one-shot	1	43.9	0.55	16.0
	PEP-AI one-shot	8	45.8	0.59	17.4
	PEP-AI one-shot	20	57.2	0.80	25.6
	PEP-AI one-shot	30	65.1	0.87	27.9
	PEP-AI iterative	4	53.0	0.63	19.7
	PEP-AI iterative	8	62.3	0.72	23.3
	PEP-AI iterative	20	66.0	0.86	27.9
	FP32 noise ranking	8	25.1	0.56	16.4
	FP32 noise ranking	20 (22)	34.9	0.75	23.3
	random (3 seeds)	20	40.8 [34.5, 45.7]	0.76	23.7
	all depthwise	16	65.0	0.95	31.0
	FP16 (reference)	–	65.8		
	FP32 (reference)	–	65.8		
YOLOv10-S	INT8 (base)	0	43.8	1.48	64.6
	PEP-AI one-shot	1	43.9	1.73	83.2
	PEP-AI one-shot	8	43.9	1.80	87.9
	PEP-AI one-shot	12	43.9	1.89	94.1
	PEP-AI one-shot	20 (27)	43.9	1.99	104.5
	FP32 noise ranking	8 (12)	43.8	1.65	76.3
	FP32 noise ranking	20 (23)	43.9	2.04	106.2
	FP16 (reference)	–	46.0		
	FP32 (reference)	–	46.0		
YOLOv10-X	INT8 (base)	0	47.8	4.82	309.8
	PEP-AI one-shot	1	47.7	4.94	300.5
	PEP-AI one-shot	8	47.9	5.51	376.0
	PEP-AI one-shot	12	48.2	5.71	394.9
	PEP-AI one-shot	20 (26)	52.9	6.31	453.5
	FP16 (reference)	–	54.0		
	FP32 (reference)	–	54.0		

Accuracy: ImageNetV2 top-1 (EfficientNet-B0) or COCO val2017 mAP (YOLOv10). k_{eff} : FP16 convolutions including those added by the kernel-closure rule. Latency and energy from a shorter protocol than Table 5 (500 executions, 5s energy window); compare within this table. Random subsets: mean and range over three seeds.

Table C18 Sensitivity of the deployed INT8 accuracy to the calibration sample

Model	$\bar{\Gamma}$	Calibration images	Full set	Seed 1	Seed 2	Seed 3	Range	Max $ \Delta $
Faster R-CNN R50-FPN	0.13	256	36.71	36.72	36.70	36.71	36.70–36.72	0.01
YOLOv10-S	0.28	256	43.79	43.82	43.83	45.31	43.82–45.31	1.52
YOLOv10-X	0.69	256	47.79	47.79	47.97	50.45	47.79–50.45	2.66
EfficientNet-B0	2.89	500	24.82	41.70	41.53	41.31	41.31–41.70	16.88
		250	24.82	41.66	41.39	40.09	40.09–41.66	16.83
DenseNet-121	0.90	500	57.36	58.44	58.66	58.47	58.44–58.66	1.30
		250	57.36	58.31	58.52	60.44	58.31–60.44	3.09

Entropy calibration on seeded random subsets of the calibration set (COCO train2017: 512 images; ImageNetV2 calibration split: 1000 images), same exclusions and deployment engines as the main results; accuracy on the full evaluation sets (COCO val2017 box mAP, ImageNetV2 top-1, %). Full set: main result (Table 5); recalibrating on the full set with the same code reproduces it. Max $|\Delta|$: largest deviation of a subset calibration from the full-set result.

Table C19 INT8 accuracy under different range-setting procedures

Model	FP32	INT8, toolchain	entropy calibration		INT8, order-independent	FP8
			subsets (range)	max $ \Delta $		
Faster R-CNN R50-FPN	36.95	36.71	36.70–36.72	0.01	36.74	36.58
YOLOv10-S	46.03	43.79	43.82–45.31	1.52	35.80	45.62
YOLOv10-X	54.00	47.79	47.79–50.45	2.66	37.07	53.85
EfficientNet-B0	65.79	24.82	40.09–41.70	16.88	0.09	65.40
DenseNet-121	62.03	57.36	58.31–60.44	3.09	60.01	61.71
EfficientNet-B0, convolution inputs only	65.79	23.84	–	–	54.91	–

Toolchain: incremental entropy calibration of ModelOpt on ONNX Runtime (one image per step), full calibration set in stored order (main results) and seeded random subsets, which differ in their first image (Table C18). Order-independent: global 2048-bin histogram with NVIDIA's reference KL search (Supplementary Section A.1), applied to the same quantizers. Convolution inputs only: quantizers on the inputs of convolutions and matrix multiplications, as in [18]. COCO val2017 box mAP and ImageNetV2 top-1 (%); FP8: max calibration.

Table C20 Deployment options per network on the H200 (batch size 8)

Model	Configuration		Accuracy (%)	Δ vs FP32	p50 bs8 (ms)	Energy bs8 (mJ/img)	Energy vs FP32	Energy vs FP16
Faster R-CNN R50-FPN	FP16		36.9	0.0	5.09	442.7	-86%	0%
	INT8		36.7	-0.2	3.46	280.8	-91%	-37%
YOLOv10-S	FP16		46.0	0.0	1.83	108.8	-73%	0%
	INT8		43.8	-2.2	1.47	63.3	-85%	-42%
	FP8		45.6	-0.4	1.86	96.7	-76%	-11%
YOLOv10-X	FP16		54.0	0.0	6.90	581.6	-77%	0%
	FP8		53.9	-0.1	5.82	429.9	-83%	-26%
	INT8 + NMS		52.4	-1.6	5.16	295.3	-88%	-49%
EfficientNet-B0	FP16		65.8	0.0	0.66	20.6	-55%	0%
	FP8		65.4	-0.4	0.83	26.0	-43%	+26%
DenseNet-121	FP16		62.0	0.0	2.39	70.5	-54%	0%
	FP8		61.7	-0.3	1.23	40.9	-73%	-42%
	INT8, BN in FP16		61.6	-0.4	2.46	66.3	-56%	-6%

Accuracy: COCO val2017 box mAP or ImageNetV2 top-1 of the deployed engine. INT8 + NMS: class-wise NMS appended to the engine output (its GPU time is included in the latency; its energy is not). INT8, BN in FP16: the placement selected by the label-free criteria (Table C3). Energy: gross GPU energy per image with CUDA graphs.

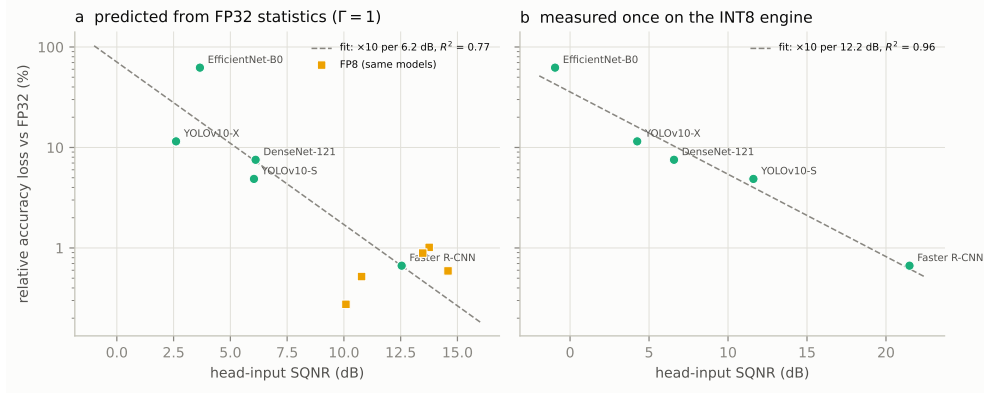


Fig. C7 Prediction of INT8 tolerance. Relative INT8 accuracy loss of the five networks against the head-input SQNR predicted from FP32 statistics with unit propagation factors (**a**; squares: FP8 engines of the same networks) and measured once on the INT8 engine with FP32 fallback (**b**); dashed: log-linear least-squares fits.

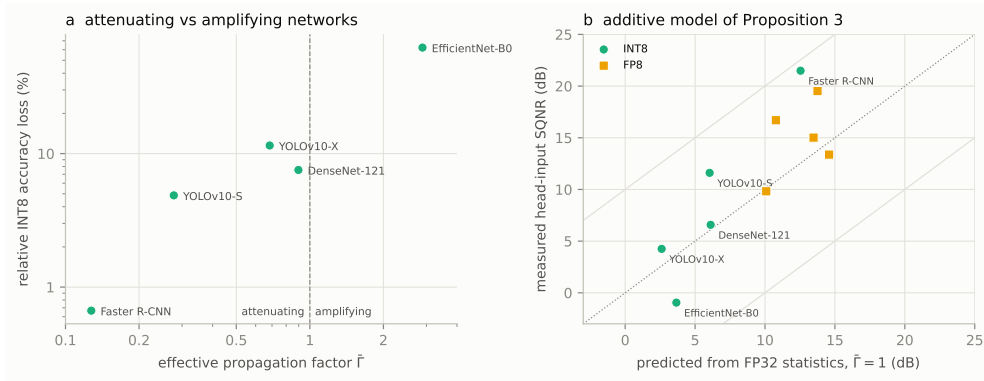


Fig. C8 Validation of the additive noise model. **a** Relative INT8 accuracy loss against the effective propagation factor $\bar{\Gamma}$ of Eq. (16); networks left of the dashed line attenuate the injected noise, those right of it amplify it. **b** Head-input SQNR predicted from FP32 statistics with unit propagation factors against the measured value for INT8 and FP8 (dotted: identity; grey: ± 10 dB).

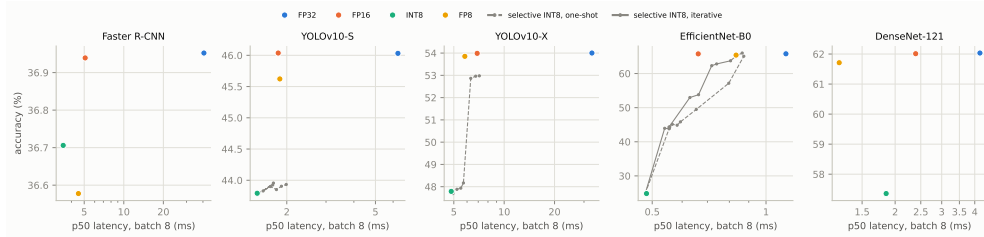


Fig. C9 Accuracy versus latency. Accuracy against the median latency at batch size 8 for FP32, FP16, INT8 and FP8 (final benchmark) and for the selective-precision INT8 variants (grey; one-shot and iterative PEP-AI ranking, shorter timing protocol).



Fig. C10 Qualitative example under reduced contrast. YOLOv10-X on COCO image 356612, clean (top) and at contrast severity 4 (bottom); the scene was selected automatically as the one with the most INT8-specific vanishing detections among the first 200 analysis images with at least four road users. Left: FP32 detections (score ≥ 0.5). Middle and right: INT8 (FP32 fallback) and FP8 detections over the head-input MRE_{proj} map (mean of the three pyramid levels); dashed: FP32 detections without a counterpart ($IoU \geq 0.5$, score ≥ 0.3).

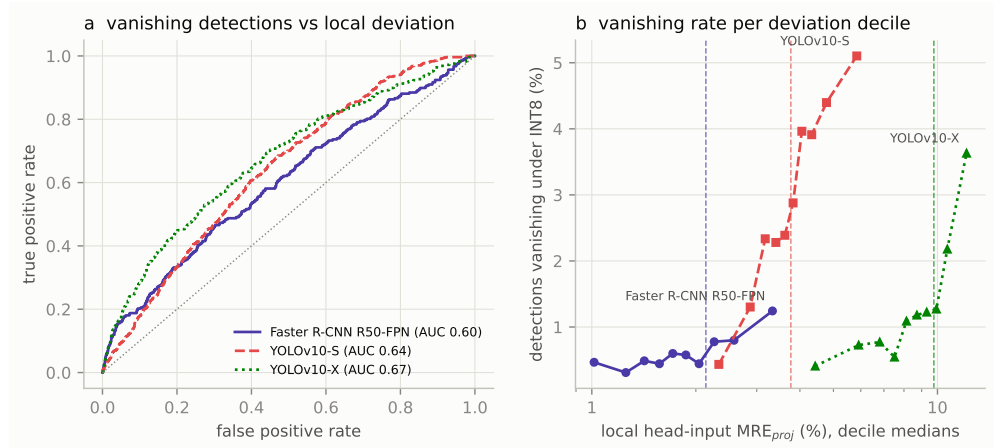


Fig. C11 Local deviation as a predictor of vanishing detections. Vanishing detections are FP32 detections above the 0.5 threshold without an INT8 counterpart down to a score of 0.3 (COCO val2017, 5 000 images, INT8 with FP32 fallback). **a** ROC curves of the median MRE_{proj} inside the detection box. **b** Share of vanishing detections per decile of the local deviation; dashed lines: data-derived threshold of each architecture (maximum Youden index).

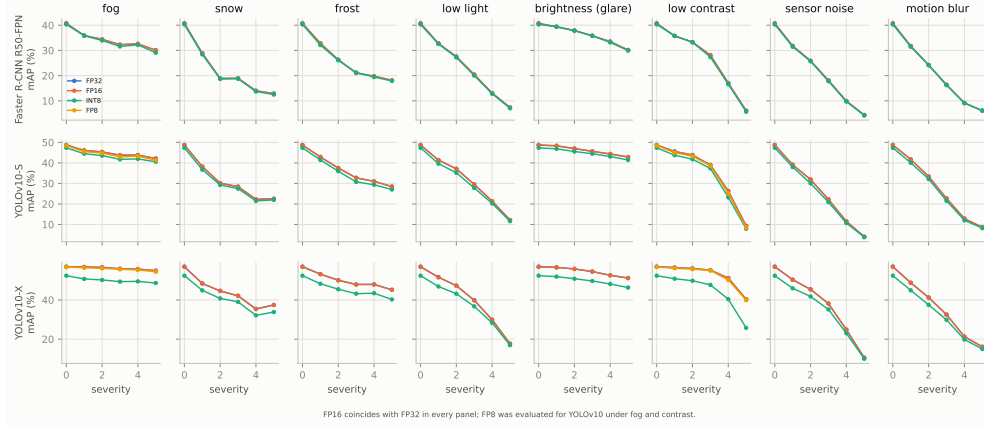


Fig. C12 Detection accuracy under adverse conditions. Box mAP of the FP32, FP16, INT8 and FP8 detectors under the eight adverse conditions at severities 0 (clean) to 5 (500 COCO val2017 images; FP8 evaluated for YOLOv10 under fog and contrast). FP16 coincides with FP32 in every panel, and FP8 stays close to FP32 (Supplementary Section B.5).

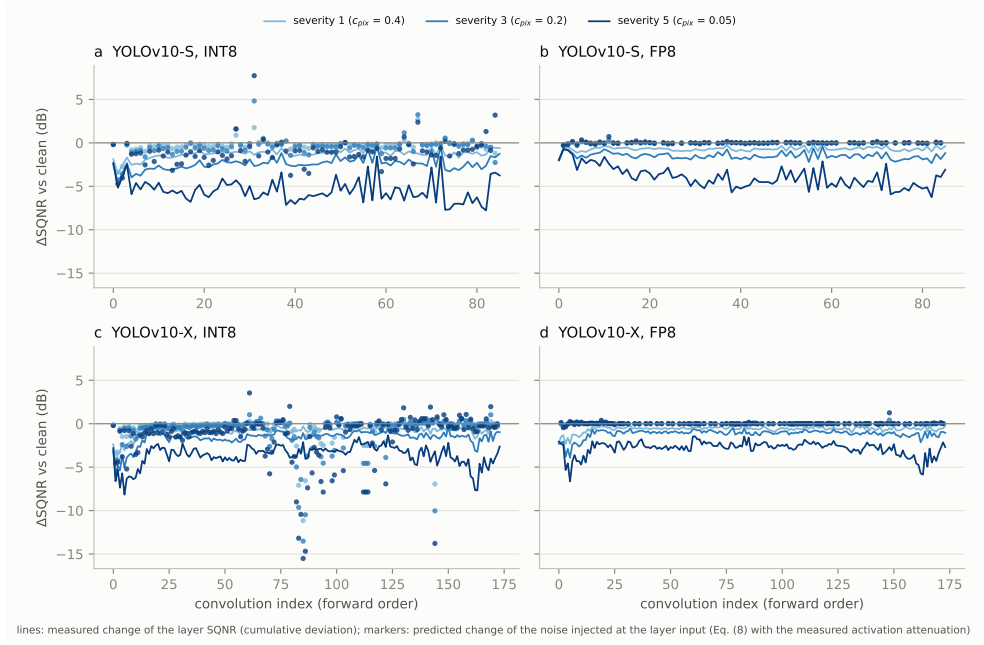


Fig. C13 Test of the static-scale prediction under reduced contrast. YOLOv10-S and YOLOv10-X in INT8 (FP32 fallback) and FP8. Lines: measured change of the SQNR of every convolution output relative to the clean images (median over the same 100 images; cumulative deviation). Markers: change of the noise injected at the input quantizer of each convolution, computed exactly from FP32 activations and the calibrated scales (32 images), i.e. Eq. (8) with the measured activation attenuation.

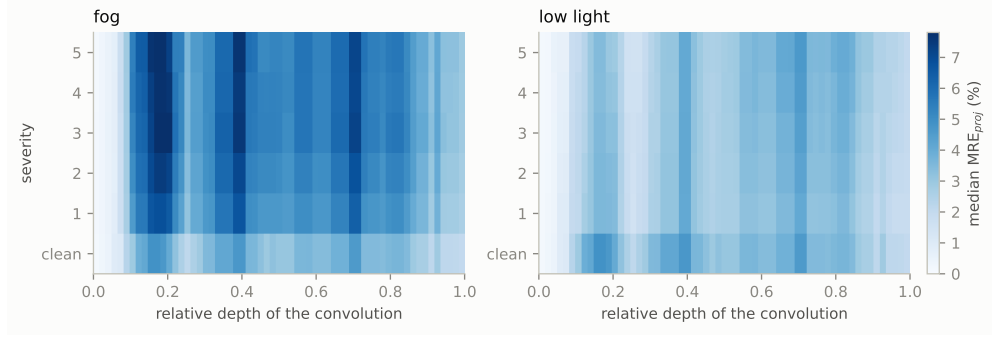


Fig. C14 Deviation of Faster R-CNN under fog and low light. Median MRE_{proj} of every INT8 Faster R-CNN convolution (backbone and FPN, ordered by depth) as a function of the severity of fog and low light, on a shared colour scale (the same 100 images at every severity; INT8 with FP32 fallback).

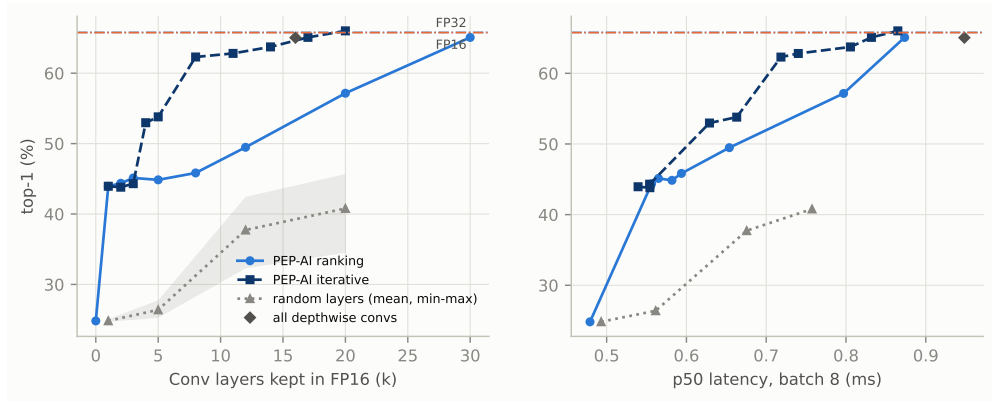


Fig. C15 Selective precision for EfficientNet-B0. Top-1 accuracy versus the number of convolutions kept in FP16 (left) and versus latency at batch size 8 (right) for the PEP-AI ranking, the iterative PEP-AI ranking, random subsets (mean and range over three seeds) and all depthwise convolutions. Dash-dotted lines: FP32 and FP16.

References

- [1] Veres, M., Moussa, M.: Deep learning for intelligent transportation systems: A survey of emerging trends. *IEEE Transactions on Intelligent Transportation Systems* **21**(8), 3152–3168 (2020)
- [2] Sudhakar, S., Sze, V., Karaman, S.: Data centers on wheels: Emissions from computing onboard autonomous vehicles. *IEEE Micro* **43**(1), 29–39 (2023)
- [3] Strubell, E., Ganesh, A., McCallum, A.: Energy and policy considerations for deep learning in NLP. In: *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 3645–3650 (2019)

- [4] Schwartz, R., Dodge, J., Smith, N.A., Etzioni, O.: Green AI. *Communications of the ACM* **63**(12), 54–63 (2020)
- [5] Wu, C.-J., Raghavendra, R., Gupta, U., Acun, B., Ardalani, N., Maeng, K., Chang, G., Behram, F.A., Huang, J., Bai, C., Gschwind, M., Gupta, A., Ott, M., Melnikov, A., Candido, S., Brooks, D., Chauhan, G., Lee, B., Lee, H.-H.S., Akyildiz, B., Balandat, M., Spisak, J., Jain, R., Rabbat, M., Hazelwood, K.: Sustainable AI: Environmental implications, challenges and opportunities. In: *Proceedings of Machine Learning and Systems (MLSys)*, vol. 4 (2022)
- [6] Lannelongue, L., Grealey, J., Inouye, M.: Green algorithms: Quantifying the carbon footprint of computation. *Advanced Science* **8**(12), 2100707 (2021)
- [7] Luccioni, S., Jernite, Y., Strubell, E.: Power hungry processing: Watts driving the cost of AI deployment? In: *Proceedings of the 2024 ACM Conference on Fairness, Accountability, and Transparency (FAccT)* (2024). <https://doi.org/10.1145/3630106.3658542>
- [8] Desislavov, R., Martínez-Plumed, F., Hernández-Orallo, J.: Trends in AI inference energy consumption: Beyond the performance-vs-parameter laws of deep learning. *Sustainable Computing: Informatics and Systems* **38**, 100857 (2023) <https://doi.org/10.1016/j.suscom.2023.100857>
- [9] Henderson, P., Hu, J., Romoff, J., Brunskill, E., Jurafsky, D., Pineau, J.: Towards the systematic reporting of the energy and carbon footprints of machine learning. *Journal of Machine Learning Research* **21**(248), 1–43 (2020)
- [10] European Commission: The European Green Deal. Communication from the Commission, COM(2019) 640 final, Brussels (2019)
- [11] European Commission: Sustainable and Smart Mobility Strategy – Putting European Transport on Track for the Future. Communication from the Commission, COM(2020) 789 final, Brussels (2020)
- [12] Duan, Y., Edwards, J.S., Dwivedi, Y.K.: Artificial intelligence for decision making in the era of Big Data – evolution, challenges and research agenda. *International Journal of Information Management* **48**, 63–71 (2019) <https://doi.org/10.1016/j.ijinfomgt.2019.01.021>
- [13] Jacob, B., Kligys, S., Chen, B., Zhu, M., Tang, M., Howard, A., Adam, H., Kalenichenko, D.: Quantization and training of neural networks for efficient integer-arithmetic-only inference. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 2704–2713 (2018)
- [14] Krishnamoorthi, R.: Quantizing deep convolutional networks for efficient inference: A whitepaper. arXiv preprint arXiv:1806.08342 (2018)

- [15] Nagel, M., Fournarakis, M., Amjad, R.A., Bondarenko, Y., van Baalen, M., Blankevoort, T.: A white paper on neural network quantization. arXiv preprint arXiv:2106.08295 (2021)
- [16] Gholami, A., Kim, S., Dong, Z., Yao, Z., Mahoney, M.W., Keutzer, K.: A survey of quantization methods for efficient neural network inference. In: Low-Power Computer Vision, pp. 291–326. Chapman and Hall/CRC, Boca Raton (2022). <https://doi.org/10.1201/9781003162810-13>
- [17] Horowitz, M.: Computing’s energy problem (and what we can do about it). In: IEEE International Solid-State Circuits Conference (ISSCC), pp. 10–14 (2014)
- [18] Wu, H., Judd, P., Zhang, X., Isaev, M., Micikevicius, P.: Integer quantization for deep learning inference: Principles and empirical evaluation. arXiv preprint arXiv:2004.09602 (2020)
- [19] NVIDIA: TensorRT. <https://developer.nvidia.com/tensorrt>. Version 11.3 (2026)
- [20] Nagel, M., Amjad, R.A., van Baalen, M., Louizos, C., Blankevoort, T.: Up or down? Adaptive rounding for post-training quantization. In: Proceedings of the 37th International Conference on Machine Learning (ICML), pp. 7197–7206 (2020)
- [21] Hubara, I., Nahshan, Y., Hanani, Y., Banner, R., Soudry, D.: Accurate post training quantization with small calibration sets. In: Proceedings of the 38th International Conference on Machine Learning (ICML). PMLR, vol. 139, pp. 4466–4475 (2021)
- [22] Li, Y., Gong, R., Tan, X., Yang, Y., Hu, P., Zhang, Q., Yu, F., Wang, W., Gu, S.: BRECK: Pushing the limit of post-training quantization by block reconstruction. In: International Conference on Learning Representations (ICLR) (2021)
- [23] Wei, X., Gong, R., Li, Y., Liu, X., Yu, F.: QDrop: Randomly dropping quantization for extremely low-bit post-training quantization. In: International Conference on Learning Representations (ICLR) (2022)
- [24] Nagel, M., van Baalen, M., Blankevoort, T., Welling, M.: Data-free quantization through weight equalization and bias correction. In: Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), pp. 1325–1334 (2019)
- [25] Zhao, R., Hu, Y., Dotzel, J., De Sa, C., Zhang, Z.: Improving neural network quantization without retraining using outlier channel splitting. In: Proceedings of the 36th International Conference on Machine Learning (ICML). PMLR, vol. 97, pp. 7543–7552 (2019)
- [26] Dong, Z., Yao, Z., Gholami, A., Mahoney, M.W., Keutzer, K.: HAWQ: Hessian

- AWare quantization of neural networks with mixed-precision. In: Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), pp. 293–302 (2019)
- [27] Yao, Z., Dong, Z., Zheng, Z., Gholami, A., Yu, J., Tan, E., Wang, L., Huang, Q., Wang, Y., Mahoney, M.W., Keutzer, K.: HAWQ-V3: Dyadic neural network quantization. In: Proceedings of the 38th International Conference on Machine Learning (ICML). PMLR, vol. 139, pp. 11875–11886 (2021)
 - [28] Banner, R., Nahshan, Y., Soudry, D.: Post training 4-bit quantization of convolutional networks for rapid-deployment. In: Advances in Neural Information Processing Systems, vol. 32 (2019)
 - [29] Kuzmin, A., van Baalen, M., Ren, Y., Nagel, M., Peters, J., Blankevoort, T.: FP8 quantization: The power of the exponent. In: Advances in Neural Information Processing Systems, vol. 35 (2022)
 - [30] van Baalen, M., Kuzmin, A., Nair, S.S., Ren, Y., Mahurin, E., Patel, C., Subramanian, S., Lee, S., Nagel, M., Soriaga, J., Blankevoort, T.: FP8 versus INT8 for Efficient Deep Learning Inference (2023). <https://doi.org/10.48550/arXiv.2303.17951>
 - [31] Sakr, C., Kim, Y., Shanbhag, N.: Analytical guarantees on numerical precision of deep neural networks. In: Proceedings of the 34th International Conference on Machine Learning (ICML), pp. 3007–3016 (2017)
 - [32] Lin, J., Gan, C., Han, S.: Defensive quantization: When efficiency meets robustness. In: International Conference on Learning Representations (ICLR) (2019)
 - [33] Hooker, S., Courville, A., Clark, G., Dauphin, Y., Frome, A.: What do compressed deep neural networks forget? arXiv preprint arXiv:1911.05248 (2019)
 - [34] International Organization for Standardization: ISO 21448:2022 Road vehicles – Safety of the intended functionality (2022)
 - [35] International Organization for Standardization: ISO/PAS 8800:2024 Road vehicles — Safety and artificial intelligence. Publicly Available Specification (2024)
 - [36] Mohseni, S., Wang, H., Xiao, C., Yu, Z., Wang, Z., Yadawa, J.: Taxonomy of machine learning safety: A survey and primer. ACM Computing Surveys **55**(8), 157 (2022) <https://doi.org/10.1145/3551385>
 - [37] Sheng, T., Feng, C., Zhuo, S., Zhang, X., Shen, L., Aleksic, M.: A quantization-friendly separable convolution for MobileNets. In: Workshop on Energy Efficient Machine Learning and Cognitive Computing for Embedded Applications (EMC2) (2018)

- [38] Xiao, Y., Zhang, T., Liu, S., Qin, H.: Benchmarking the robustness of quantized models. arXiv preprint arXiv:2304.03968 (2023)
- [39] Michaelis, C., Mitzkus, B., Geirhos, R., Rusak, E., Bringmann, O., Ecker, A.S., Bethge, M., Brendel, W.: Benchmarking robustness in object detection: Autonomous driving when winter is coming. arXiv preprint arXiv:1907.07484 (2019)
- [40] Zhang, Y., Carballo, A., Yang, H., Takeda, K.: Perception and sensing for autonomous vehicles under adverse weather conditions: A survey. *ISPRS Journal of Photogrammetry and Remote Sensing* **196**, 146–177 (2023) <https://doi.org/10.1016/j.isprsjprs.2022.12.021>
- [41] Menyhárt, T., Hajdu, A., Lakatos, R.: Quantization-induced error propagation: Activation analysis within the PEP-AI framework for sustainable high-efficiency and explainable AI. In: *Proceedings of CITDS 2026* (2026)
- [42] Ren, S., He, K., Girshick, R., Sun, J.: Faster R-CNN: Towards real-time object detection with region proposal networks. *IEEE Transactions on Pattern Analysis and Machine Intelligence* **39**(6), 1137–1149 (2017)
- [43] Wang, A., Chen, H., Liu, L., Chen, K., Lin, Z., Han, J., Ding, G.: YOLOv10: Real-time end-to-end object detection. In: *Advances in Neural Information Processing Systems*, vol. 37 (2024)
- [44] Tan, M., Le, Q.V.: EfficientNet: Rethinking model scaling for convolutional neural networks. In: *Proceedings of the International Conference on Machine Learning (ICML)*, pp. 6105–6114 (2019)
- [45] Huang, G., Liu, Z., van der Maaten, L., Weinberger, K.Q.: Densely connected convolutional networks. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 2261–2269 (2017)
- [46] Lin, T.-Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., Dollár, P., Zitnick, C.L.: Microsoft COCO: Common objects in context. In: *Proceedings of the European Conference on Computer Vision (ECCV)*, pp. 740–755 (2014)
- [47] Recht, B., Roelofs, R., Schmidt, L., Shankar, V.: Do ImageNet classifiers generalize to ImageNet? In: *Proceedings of the 36th International Conference on Machine Learning (ICML)*, pp. 5389–5400 (2019)
- [48] NVIDIA: TensorRT Model Optimizer. <https://github.com/NVIDIA/TensorRT-Model-Optimizer>. Version 0.47 (2026)
- [49] Migacz, S.: 8-bit Inference with TensorRT. NVIDIA GPU Technology Conference (2017)

- [50] Micikevicius, P., Stosic, D., Burgess, N., Cornea, M., Dubey, P., Grisenthwaite, R., Ha, S., Heinecke, A., Judd, P., Kamalu, J., Mellempudi, N., Oberman, S., Shoeybi, M., Siu, M., Wu, H.: FP8 formats for deep learning. arXiv preprint arXiv:2209.05433 (2022)
- [51] Micikevicius, P., Narang, S., Alben, J., Diamos, G., Elsen, E., Garcia, D., Ginsburg, B., Houston, M., Kuchaiev, O., Venkatesh, G., Wu, H.: Mixed precision training. In: International Conference on Learning Representations (ICLR) (2018)
- [52] NVIDIA: NVIDIA H200 Tensor Core GPU Datasheet. <https://resources.nvidia.com/en-us-data-center-overview-mc/en-us-data-center-overview/hpc-datasheet-sc23-h200> (2023)
- [53] NVIDIA: Working with Quantized Types – NVIDIA TensorRT Developer Guide. <https://docs.nvidia.com/deeplearning/tensorrt/10.x.x/inference-library/work-quantized-types.html> (2026)
- [54] Padilla, R., Passos, W.L., Dias, T.L.B., Netto, S.L., Silva, E.A.B.: A comparative analysis of object detection metrics with a companion open-source toolkit. *Electronics* **10**(3), 279 (2021) <https://doi.org/10.3390/electronics10030279>
- [55] Yang, Z., Adamek, K., Armour, W.: Accurate and convenient energy measurements for GPUs: A detailed study of NVIDIA GPU’s built-in power sensor. In: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC24) (2024). <https://doi.org/10.1109/SC41406.2024.00028>
- [56] Hendrycks, D., Dietterich, T.: Benchmarking neural network robustness to common corruptions and perturbations. In: International Conference on Learning Representations (ICLR) (2019)
- [57] Eurostat: Non-household electricity prices in 2nd half of 2025. Eurostat news release, 8 May 2026, <https://ec.europa.eu/eurostat/web/products-eurostat-news/w/ddn-20260508-2> (2026)
- [58] Ember: European Electricity Review 2024. <https://ember-energy.org/latest-insights/european-electricity-review-2024/> (2024)
- [59] Wang, C.-Y., Bochkovskiy, A., Liao, H.-Y.M.: YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 7464–7475 (2023)
- [60] Zhao, Y., Lv, W., Xu, S., Wei, J., Wang, G., Dang, Q., Liu, Y., Chen, J.: DETRs beat YOLOs on real-time object detection. In: Proceedings of the IEEE/CVF

Conference on Computer Vision and Pattern Recognition (CVPR), pp. 16965–16974 (2024)

- [61] TorchVision maintainers and contributors: TorchVision: PyTorch’s Computer Vision Library. <https://github.com/pytorch/vision> (2016)
- [62] Taori, R., Dave, A., Shankar, V., Carlini, N., Recht, B., Schmidt, L.: Measuring robustness to natural distribution shifts in image classification. In: Advances in Neural Information Processing Systems (NeurIPS), vol. 33, pp. 18583–18599 (2020)
- [63] Rouhani, B.D., Zhao, R., More, A., Hall, M., Khodamoradi, A., et al.: Microscaling Data Formats for Deep Learning (2023). <https://doi.org/10.48550/arXiv.2310.10537>
- [64] Geiger, A., Lenz, P., Urtasun, R.: Are we ready for autonomous driving? The KITTI vision benchmark suite. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 3354–3361 (2012). <https://doi.org/10.1109/CVPR.2012.6248074>